Date Published: April 01, 2026

*Visit the Zebra Product Support page for the most up-to-date version of this documentation.*

# Mcal functions

## Synopsis

The functions prefixed with Mcal make up the Calibration module. The Calibration module allows you to create a pixel coordinate to real-world coordinate mapping for your images, despite the presence of optical (for example, pincushion and barrel), aspect ratio, rotation, and perspective distortions in the images. Other Aurora Imaging Library modules can use this mapping to compensate for distortions in the images, and return results in real-world units. The mapping can also be used to physically correct the distortions in the images. The calibration mapping can be created using a simple physical grid image or a list of points.

## Functions

McalAlloc
McalAllocResult
McalAssociate
McalCalculateHandEye
McalControl
McalCopyResult
McalDraw
McalDraw3d
McalFixture
McalFree
McalGetCoordinateSystem
McalGrid
McalInquire
McalInquireSingle
McalList
McalRelativeOrigin
McalRestore
McalSave
McalSetCoordinateSystem
McalStream
McalTransformCoordinate
McalTransformCoordinate3dList
McalTransformCoordinateList
McalTransformImage
McalTransformResult
McalTransformResultAtPosition
McalUniform
McalWarp

# McalAlloc

## Synopsis

Allocate a camera calibration context, a fixturing offset object, a calculate hand-eye context, or a 3D draw calibration context.

## Syntax

```
AIL_ID McalAlloc(
    AIL_ID SysId,            //in
    AIL_INT64 Mode,          //in
    AIL_INT64 ModeFlag,      //in
    AIL_ID *CalibrationIdPtr //out
)
```

## Description

This function allocates a camera calibration context, a fixturing offset object, a calculate hand-eye context, or a 3D draw calibration context.

When allocating a camera calibration context, use McalGrid() or McalList() to define the pixel-to-world mapping for the camera calibration context. For a uniform pixel-to-world mapping, instead of using McalGrid() or McalList(), use McalUniform(). To associate the camera calibration context to an image buffer or digitizer, use McalAssociate().

When the context/object is no longer required, release it using McalFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the context's lifetime and you must not manually free it.

## Parameters

SysId INQ

Specifies the system on which to allocate the camera calibration context, fixturing offset object, calculate hand-eye context, or 3D draw calibration context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT_HOST | Specifies the default Host system of the current Aurora Imaging Library application. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *System identifier* | Specifies a valid system identifier, previously allocated using MsysAlloc(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Mode

Specifies the type of context/object to allocate, and when allocating a camera calibration context, specifies the camera calibration mode.

To allocate a camera calibration context, this parameter must be set to one of the following values. See the Camera calibration modes section of Chapter 28: Calibrating your camera setup for more information about the camera calibration mode best suited for your application.

> Note that during development and at runtime, 2D and 3D calibration support requires the presence of different Aurora Imaging Library licenses that grant access to either the 2D or 3D calibration package. Access to both is granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to the required package separately.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| To allocate a camera calibration context and the camera calibration mode that should be used INQ | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DEFAULT | Same as M_LINEAR_INTERPOLATION. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_3D_ROBOTICS | Specifies a camera calibration context for a 3D-based camera calibration that uses a Tsai-based camera model in robotics mode. This mode is a true 3D camera calibration technique that determines the distance and orientation between the modeled camera and the image plane. It also estimates the position and orientation of the robot base coordinate system with respect to the absolute coordinate system. This allows for results to be inquired with respect to any defined plane.<br><br>This calibration mode can be used for a camera setup where the camera is mounted on the last joint of a robot arm, or where the camera is stationary and the robot arm is moving. Use the ModeFlag parameter to specify your camera setup.<br><br>Tsai-based robotics mode requires at least three calls to McalGrid() or McalList() with M_ACCUMULATE before performing a full calibration with M_FULL_CALIBRATION. Before each call, you must move the robot arm to a different position, retrieve the tool's new position with respect to the robot base from the robot encoder, and using this information, set the position of the tool coordinate system with respect to the robot base coordinate system using McalSetCoordinateSystem(). Calling McalGrid() or McalList() with M_ACCUMULATE more than three times can greatly improves the accuracy of the camera calibration.<br><br>Since this mode requires that the image has perspective distortion, you should not use this mode when using a camera with a telecentric lens; telecentric lenses negate perspective effects. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_3D_ROBOTICS_ZHANG_BASED | Specifies a camera calibration context for a 3D-based camera calibration that uses a Zhang-based camera model in robotics mode.<br><br>This mode is similar to M_3D_ROBOTICS but makes use of an M_ZHANG_BASED camera model. With this mode, the camera's intrinsic parameters are determined separately from the position and orientation estimations of the robot system. This typically results in a more accurate camera calibration model compared to the M_3D_ROBOTICS mode. This calibration mode can be used for a camera setup where the camera is mounted on the last joint of a robot arm, or where the camera is stationary and the robot arm is moving. Use the ModeFlag parameter to specify your camera setup.<br><br>Zhang-based robotics mode requires at least three calls to McalGrid() or McalList() with M_ACCUMULATE before performing a full calibration with M_FULL_CALIBRATION. Before each call, you must move the robot arm to a different position, retrieve the tool's new position with respect to the robot base from the robot encoder, and using this information, set the position of the tool coordinate system with respect to the robot base coordinate system using McalSetCoordinateSystem(). Calling McalGrid() or McalList() with M_ACCUMULATE more than three times can greatly improves the accuracy of the camera calibration.<br><br>Before each call, you must also move the grid or camera such that the grid is not viewed at the same plane by the camera as a previous call to McalGrid() or McalList() with M_ACCUMULATE. The pose provided in the last call to M_ACCUMULATE will be used to automatically set the position of the absolute and camera coordinate systems. It is recommended to accumulate calibration data at various perceived depths, and at the edges of the working area of your application to accurately estimate the focal length and the radial distortion coefficients.<br><br>Since this mode requires that the image has perspective distortion, you should not use this mode when using a camera with a telecentric lens; telecentric lenses negate perspective effects. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_LINEAR_INTERPOLATION | Specifies a camera calibration context for a 2D-based camera calibration that uses piecewise linear interpolation mode. This mode fits a piecewise linear interpolation function to the specified set of image coordinates and their real-world equivalents.<br><br>This mode can compensate for any type of distortion. It is very accurate for points located inside the working area. However, it is less accurate for points outside the working area. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PERSPECTIVE_TRANSFORMATION | Specifies a camera calibration context for a 2D-based camera calibration that uses perspective transformation mode. This mode best fits a global perspective transformation function to the set of image coordinates and their real-world equivalents.<br><br>This mode can compensate for rotation, translation, scale, and perspective distortions. For such distortions, the perspective transformation mode is accurate for points inside and outside the working area. This mode cannot compensate for non-linear distortions such as lens distortions. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TSAI_BASED | Specifies a camera calibration context for a 3D-based camera calibration that uses Tsai-based mode. This mode is based on a technique developed by Roger Y. Tsai. It is a true 3D camera calibration technique that determines the distance and orientation between the modeled camera and the image plane. This allows for results to be inquired with respect to any defined plane.<br><br>To use this mode, the optical axis of your camera must be at least 30º away from the axis perpendicular to the camera calibration plane. If the optical axis is perpendicular to the camera calibration plane, you must perform two calibrations: the first calibration must be performed with the camera inclined with respect to the camera calibration plane, using McalGrid() or McalList() with M_FULL_CALIBRATION; the second calibration must be performed after repositioning the camera, such that its optical axis is perpendicular to the camera calibration plane, using McalGrid() or McalList() with M_DISPLACE_CAMERA_COORD. Instead of calibrating a second time, you can, alternatively, move the camera coordinate system using McalSetCoordinateSystem() if the movement is known with great precision.<br><br>Since this mode requires that the image has perspective distortion, you should not use this mode when using a camera with a telecentric lens; telecentric lenses negate perspective effects. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNIFORM_TRANSFORMATION | Specifies a camera calibration context for a 2D-based camera calibration that uses uniform transformation mode.<br><br>This mode can only map a linear translation, rotation, and scaling between the pixel and world coordinate systems. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_ZHANG_BASED | Specifies a camera calibration context for a 3D-based camera calibration that uses Zhang-based mode. This mode is based on a technique developed by Z. Zhang. It is a true 3D camera calibration technique that determines the distance and orientation between the modeled camera and the image plane. This allows for results to be inquired with respect to any defined plane.<br><br>To use this mode, it is required to make at least three calls to McalGrid() or McalList() with M_ACCUMULATE before performing a full calibration with M_FULL_CALIBRATION. Calling McalGrid() or McalList() with M_ACCUMULATE more than three times can greatly improves the accuracy of the camera calibration.<br><br>Before each call, move the grid or camera such that the grid is not viewed at the same plane by the camera as a previous call to McalGrid() or McalList() with M_ACCUMULATE. The pose provided in the last call to M_ACCUMULATE will be used to automatically set the position of the absolute and camera coordinate systems. It is recommended to accumulate calibration data at various perceived depths, and at the edges of the working area of your application to accurately estimate the focal length and the radial distortion coefficients. You can inquire the values of the second and fourth order radial distortion coefficients using McalInquire() with M_DISTORTION_RADIAL_1 and M_DISTORTION_RADIAL_2, respectively.<br><br>Note that the Zhang-based calibration mode calculates the image coordinates of the principal point, which can be inquired using McalInquire() with M_PRINCIPAL_POINT_X and M_PRINCIPAL_POINT_Y.<br><br>Since this mode requires that the image has perspective distortion, you should not use this mode when using a camera with a telecentric lens; telecentric lenses negate perspective effects. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

To allocate a fixturing offset object, set this parameter to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For setting a fixturing offset | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_FIXTURING_OFFSET | Specifies to allocate a fixturing offset object, used to store a preestablished positional and angular offset of the relative coordinate system from the object to be processed or analyzed when fixturing.<br><br>To set up the fixturing offset object, use McalFixture() with M_LEARN_OFFSET. To use the fixturing offset object, call McalFixture() with M_MOVE_RELATIVE, the reference location, and the fixturing offset object.<br><br>> Note that you cannot use a fixturing offset object to calibrate an image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

To allocate a 3D draw calibration context, set this parameter to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For allocating a 3D draw calibration context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DRAW_3D_CONTEXT | Specifies to allocate a 3D draw calibration context, for drawing 3D annotations using McalDraw3d(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

To allocate a calculate hand-eye context, set this parameter to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For allocating a calculate hand-eye context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_CALCULATE_HAND_EYE_CONTEXT | Specifies to allocate a calculate hand-eye context, for use with McalCalculateHandEye(), to calculate the transformations between elements of a robotics setup that uses a precalibrated camera/3D sensor (required for hand-eye calibration). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ModeFlag

Specifies the mode in which to calibrate when using a robotics mode (M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED). For all other contexts, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of robot setup to use | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_MOVING_CAMERA. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MOVING_CAMERA | Specifies a moving camera setup consisting of a camera/3D sensor mounted on the last joint of a robotic arm (eye-in-hand calibration).<br><br>When using McalGrid() or McalList(), the grid should be stationary. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_STATIONARY_CAMERA | Specifies a stationary camera setup consisting of a stationary camera/3D sensor, and a moving robotic arm (eye-to-hand calibration).<br><br>When using McalGrid(), the grid should be mounted on the robotic tool.<br><br>> Note that specifying this camera setup changes the default value of M_CALIBRATION_PLANE to M_RELATIVE_COORDINATE_SYSTEM. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

CalibrationIdPtr

Specifies the address of the variable in which to write the camera calibration context, fixturing offset object, calculate hand-eye context, or 3D draw calibration context identifier or specifies the data type that the function should use to return the identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For retrieving the identifier or specifying how to return it | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NULL | Specifies that you will use this function's return value to obtain the identifier of the allocated camera calibration context, fixturing offset object, calculate hand-eye context, or 3D draw calibration context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_UNIQUE_ID | Specifies that you will use this function's return value to obtain the identifier of the allocated camera calibration context, fixturing offset object, calculate hand-eye context, or 3D draw calibration context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CAL_ID* is returned instead of a standard Aurora Imaging Library identifier.<br><br>> This setting is only available when using C++11 (or later).<br><br>An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the camera calibration context, fixturing offset object, calculate hand-eye context, or 3D draw calibration context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Address in which to write the 3D draw calibration context identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D draw calibration context.<br><br>If allocation fails, **M_NULL** is written as the identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Address in which to write the calculate hand-eye context identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the allocated calculate hand-eye context.<br><br>If allocation fails, **M_NULL** is written as the identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Address in which to write the camera calibration context identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the allocated camera calibration context.<br><br>If allocation fails, **M_NULL** is written as the identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Address in which to write the fixturing offset object identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the allocated fixturing offset object.<br><br>If allocation fails, **M_NULL** is written as the identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Return value

The returned value is the camera calibration context, fixturing offset object, calculate hand-eye context, or 3D draw calibration context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CAL_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalAllocResult

## Synopsis

Allocate a calculate hand-eye result buffer.

## Syntax

```
AIL_ID McalAllocResult(
    AIL_ID SysId,          //in
    AIL_INT64 ResultType,  //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ResultCalIdPtr //out
)
```

## Description

This function allocates a calculate hand-eye result buffer, on the specified system, to store results from an McalCalculateHandEye() operation.

When the calculate hand-eye result buffer is no longer required, release it using McalFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the calculate hand-eye result buffer's lifetime and you must not manually free it.

## Parameters

SysId

Specifies the system on which to allocate the result buffer. This parameter should be set to one of the following values:

| For specifying the system | |
|---|---|
| Value | Description |
| M_DEFAULT_HOST | Specifies the default Host system of the current Aurora Imaging Library application. |
| *System identifier* | Specifies a valid system identifier, previously allocated using MsysAlloc(). |

ResultType

Specifies the type of result buffer to allocate.

| For specifying the type of result buffer to allocate | |
|---|---|
| Value | Description |
| M_CALCULATE_HAND_EYE_RESULT | Specifies to allocate a calculate hand-eye result buffer used to store McalCalculateHandEye() results.<br>> To retrieve the transformation matrices that are calculated using McalCalculateHandEye(), use McalCopyResult(). |

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ResultCalIdPtr

Specifies the address of the variable in which to write the calculate hand-eye result buffer identifier.

| For retrieving the identifier or specifying how to return it | |
|---|---|
| Value | Description |
| M_NULL | Specifies that you will use this function's return value to obtain the identifier of the allocated calculate hand-eye result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned. |

| | |
|---|---|
| M_UNIQUE_ID | Specifies that you will use this function's return value to obtain the identifier of the allocated calculate hand-eye result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CAL_ID* is returned instead of a standard Aurora Imaging Library identifier.<br><br>> This setting is only available when using C++11 (or later).<br><br>An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the calculate hand-eye result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application. |
| *Address in which to write the calculate hand-eye result buffer identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the allocated calculate hand-eye result buffer.<br><br>If allocation fails, **M_NULL** is written as the identifier. |

## Return value

The returned value is the calculate hand-eye result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CAL_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalAssociate

## Synopsis

Associate/disassociate a camera calibration context with/from an image or digitizer.

## Syntax

```
void McalAssociate(
    AIL_ID SrcCalibrationOrAilId,    //in
    AIL_ID TargetImageOrDigitizerId, //out
    AIL_INT64 ControlFlag            //in
)
```

## Description

This function associates a camera calibration context with an image or digitizer. This function can also be used to disassociate a camera calibration context from an image or digitizer. Note that you do not have to first disassociate a camera calibration context from an image or digitizer to associate it with a different camera calibration context.

When you grab an image with a calibrated digitizer, the camera calibration context currently associated with the digitizer gets associated with the grabbed image.

When you associate a camera calibration context with an image using **McalAssociate**() or by grabbing with a calibrated digitizer, the image receives a copy of the camera calibration context's current coordinate systems and a reference to the camera calibration context for all other settings. When you associate a camera calibration context with a digitizer, the digitizer only receives a reference to the camera calibration context.

When associating a camera calibration context with an image, its child buffers, if any, will also be associated with the same camera calibration context. However, any previous camera calibration information of the child buffers will be overwritten by the new context associated with the parent.

If you copy or process the calibrated image, the operation will typically copy the image's camera calibration settings to the destination image. If multiple source images are used for the operation, the destination image will only receive a copy of the camera calibration information if the source images have the same camera calibration information. Most functions propagate the calibration settings, while others clear this information. For more information, see the Camera calibration propagation section of Chapter 28: Calibrating your camera setup.

If you associate a camera calibration context with an image buffer containing an ROI, all graphic elements in the 2D graphics list that were created in world units (using MgraControl() with M_INPUT_UNITS set to M_WORLD) will be associated with the same callibration information as the image; all other graphic elements in the 2D graphics list, as well as any generated raster information, will not be calibrated. See MbufSetRegion() for more information.

## Parameters

SrcCalibrationOrAilId

Specifies the camera calibration context to associate. This parameter can be set to one of the following. If an image buffer, digitizer or result buffer is passed to this parameter, the camera calibration context associated with that object is used; camera calibration settings saved with the image, digitizer or result are also copied.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the camera calibration context to associate with | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NULL | Specifies to remove the association between the camera calibration context and the image or digitizer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibration context identifier* | Specifies the camera calibration context to associate with the target image or digitizer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Image buffer identifier* | Specifies the image whose camera calibration information and reference to a camera calibration context will be associated with the target image or digitizer.<br><br>The image buffer must be a 1- or 3-band buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Processing or analysis module result buffer identifier* | Specifies the processing or analysis module result whose camera calibration information and reference to a camera calibration context will be associated with the target image or digitizer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

TargetImageOrDigitizerId

Specifies the identifier of the image or digitizer with which to associate the camera calibration context.

The image buffer must be a 1- or 3-band buffer.

ControlFlag

Specifies the function's control flag. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the function's control flag | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Sets the function's control to the default. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalCalculateHandEye

## Synopsis

Calculate the transformations between elements of a robotics setup that uses a precalibrated camera/3D sensor (required for hand-eye calibration).

## Syntax

```
void McalCalculateHandEye(
    AIL_ID CalculateHandEyeContextCalId,     //in
    const AIL_ID *HandMatrix3dgeoIdArrayPtr, //in
    const AIL_ID *EyeMatrix3dgeoIdArrayPtr,  //in
    AIL_ID CalculateHandEyeResultCalId,      //out
    AIL_INT NumPoses,                        //in
    AIL_INT64 ControlFlag                    //in
)
```

## Description

This function calculates the transformations between elements of a robotics setup that uses a precalibrated camera/3D sensor. You can use this function to calculate the transformations required to perform operations such as robotic hand-eye calibration. The function solves the system of matrices (AX=ZB) for the X and Z transformation matrices (note that X and Z do not refer to the X and Z-axes). You must provide both the A and B transformation matrices. Note that transformation matrix A is typically obtained using the robotic software.

The following are example descriptions for the AX=ZB transformation matrices; the matrices do not have to follow the descriptions listed below.

| | Transformation matrix [1] | | | |
|---|---|---|---|---|
| | **A**<br>HandMatrix3dgeoIdArrayPtr<br>(Must be provided) | **X**<br>CalculateHandEyeResultCalId<br>(Calculated and returned) | **Z**<br>CalculateHandEyeResultCalId<br>(Calculated and returned) | **B**<br>EyeMatrix3dgeoIdArrayPtr<br>(Must be provided) |
| **Eye-in-hand calibration**<br>Camera attached to robotic arm | Transform points from the tool coordinate system to the robot base coordinate system | Transforms points from the camera coordinate system to the tool coordinate system | Transforms points from the reference object coordinate system to the robot base coordinate system | Transform points from the camera coordinate system to the reference object coordinate system |
| **Eye-to-hand calibration**<br>Camera stationary and robotic arm moves | Transforms points from the robot base coordinate system to the tool coordinate system | Transforms points from the camera coordinate system to the robot base coordinate system | Transforms points from the reference object coordinate system to the tool coordinate system | Transforms points from the camera coordinate system to the reference object coordinate system |

[1] In this table, the camera can be a 2D camera calibrated using a 3D camera calibration mode or a precalibrated 3D sensor.

This function is especially useful to establish the transformation to apply to points from a precalibrated 3D sensor, and to get their coordinates relative to the robot base coordinate system. Apply the transformation using M3dimMatrixTransformList(). Note that when using a 2D camera, it is more efficient to calibrate the camera in a robotics mode (McalAlloc() with M_3D_ROBOTICS or

M_3D_ROBOTICS_ZHANG_BASED) and use McalTransformCoordinate3dList() to retrieve their coordinates in the robot base coordinate system.

To accurately calculate the X and Z matrices to calibrate the robotics system, you must pass information about a minimum of 3 poses (each pose consists of an A matrix and a corresponding B matrix) to both HandMatrix3dgeoIdArrayPtr and EyeMatrix3dgeoIdArrayPtr, respectively. While you must pass a minimum of 3 poses, more poses improves the accuracy of the calibration. For each pose, you must change the position and orientation of the tool holding the camera or grid with respect to the robot base. More specifically, you must rotate the tool along at least two non-parallel axes.

You can retrieve your X or Z transformation matrix using McalCopyResult() with M_MATRIX_X or M_MATRIX_Z, respectively.

## Parameters

CalculateHandEyeContextCalId

Specifies the identifier of the calculate hand-eye context. The calculate hand-eye context must be allocated using McalAlloc() with M_CALCULATE_HAND_EYE_CONTEXT.

HandMatrix3dgeoIdArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoses)

Specifies the address of the array containing the identifiers of the transformation matrices that all represent the A transformation matrix in AX = ZB from the various poses.

You must allocate the transformation matrix object(s) using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

> Note that the number of transformation matrix objects in the array passed to HandMatrix3dgeoIdArrayPtr must be the same as EyeMatrix3dgeoIdArrayPtr.

EyeMatrix3dgeoIdArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoses)

Specifies the address of the array containing the identifiers of the transformation matrices that all represent the B transformation matrix in AX = ZB from the various poses.

You must allocate the transformation matrix object(s) using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

> Note that the number of transformation matrix objects in the array passed to HandMatrix3dgeoIdArrayPtr must be the same as EyeMatrix3dgeoIdArrayPtr.

CalculateHandEyeResultCalId

Specifies the identifier of the calculate hand-eye result buffer in which to write the resulting X and Z transformation matrices. This result buffer must be allocated using McalAllocResult() with M_CALCULATE_HAND_EYE_RESULT.

NumPoses

Specifies the number of robot poses provided to calculate the transformations between elements of a robotics setup (hand-eye calibration). The number of poses specifies the number of matrices in the arrays.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |

| | |
|---|---|
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalControl

## Synopsis

Control the camera calibration information of a camera calibration context, calibrated image, or 3D draw calibration context.

## Syntax

```
void McalControl(
    AIL_ID ContextCalOrCalibratedAilObjectId, //out
    AIL_INT64 ControlType,                    //in
    AIL_DOUBLE ControlValue                   //in
)
```

## Description

This function allows you to control a specified setting of a camera calibration context, calibrated image, or 3D draw calibration context.

If you control a camera calibration setting of a calibrated image with an M_VECTOR_AND_RASTER ROI, the raster information will be discarded, causing the ROI to become an M_VECTOR ROI. See MbufSetRegion() for more information.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **McalControl()** internally calls the *AIL_DOUBLE* version of this function (McalControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (McalControlInt64()).

## Parameters

ContextCalOrCalibratedAilObjectId

Specifies the identifier of the camera calibration context, calibrated image, or 3D draw calibration context.

ControlType

Specifies the setting to change.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

## Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For a camera calibration context
- For a 3D-based camera calibration context
- For a calibrated image
- For a calibrated and corrected image
- For a 3D draw calibration context

The following ControlType and ControlValue parameter settings can be specified for a camera calibration context:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ControlType | | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| | ControlValue | | | | | | | | | | | | | | | | |
| M_CALIBRATION_PLANE | | Sets the plane in which the calibration points are defined. INQ | | | | | | | | | | | | | | | |

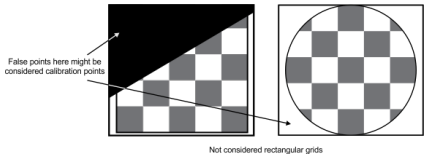
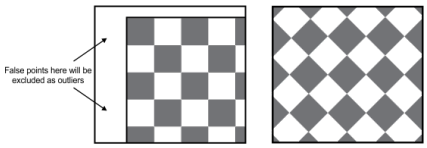
| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_DEFAULT | Specifies the default value.<br><br>For most calibration modes, the default value is M_ABSOLUTE_COORDINATE_SYSTEM. For an M_STATIONARY_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, the default is M_RELATIVE_COORDINATE_SYSTEM. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ABSOLUTE_COORDINATE_SYSTEM | Specifies that the calibration points are defined in the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_RELATIVE_COORDINATE_SYSTEM | Specifies that the calibration points are defined in the relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CALIBRATION_ERROR_SCALE_FACTOR | | Sets the scale factor used when drawing arrows, representing the camera calibration error, using McalDraw() with M_DRAW_CALIBRATION_ERROR. This scale factor is either applied directly to the arrows, or applied in conjunction with Aurora Imaging Library's automatic scaling factor, depending on whether M_DRAW_CALIBRATION_ERROR_SCALE_MODE is M_ABSOLUTE or M_AUTO. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* > 0 | Specifies the scale factor.<br><br>The default value is 1. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CALIBRATION_ERROR_SCALE_MODE | | Sets the scale mode used when drawing arrows, representing the camera calibration error, using McalDraw() with M_DRAW_CALIBRATION_ERROR. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ABSOLUTE | Specifies to use M_DRAW_CALIBRATION_ERROR_SCALE_FACTOR as a scaling factor that directly multiplies the arrows' horizontal and vertical displacements.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_AUTO | Specifies that Aurora Imaging Library determines the scale mode. To do this, Aurora Imaging Library makes internal assumptions to establish an ideal scaling factor that optimizes the space taken by the arrows in the destination (DstImageBufOrListGrald) while reducing the chance of overlapping arrows. When using M_AUTO, Aurora Imaging Library multiplies the arrows' horizontal and vertical displacements by both M_DRAW_CALIBRATION_ERROR_SCALE_FACTOR and the internally established scale factor. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FOREGROUND_VALUE | | Sets whether the circles in a circle grid, used with McalGrid(), are lighter or darker than the background. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Determines the appropriate setting automatically. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_FOREGROUND_BLACK | Specifies that the grid's circles are darker than the background. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_FOREGROUND_WHITE | Specifies that the grid's circles are lighter than the background. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_GRID_FIDUCIAL | Specifies that McalGrid() will look for a fiducial in the grid. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Same as M_NONE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DATAMATRIX | Specifies that a Data Matrix code is used as a fiducial in a chessboard grid.<br><br>This control type is only used for camera calibrations performed using McalGrid() set to M_CHESSBOARD_GRID.<br><br>You must set **McalControl**() with M_GRID_PARTIAL to M_ENABLE and M_GRID_HINT_ANGLE_X, M_GRID_HINT_PIXEL_X, and M_GRID_HINT_PIXEL_Y to M_NONE. Setting M_GRID_FIDUCIAL to M_DATAMATRIX in any other circumstance will generate an error when calling McalGrid(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies that there is no fiducial in the grid. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_HINT_ANGLE_X | Specifies the hint angle used to establish the direction of the X-axis of the absolute (or relative) coordinate system, when calibrating with a partial chessboard grid.<br><br>To determine the X-axis of the absolute coordinate system when using a partial grid, McalGrid() first determines the two grid lines intersecting at the reference calibration point. The X- and Y-axes are always aligned with these grid lines. By default, the X-axis of the absolute coordinate system is the grid line intersecting the reference calibration point and closest to the image's X-axis, with the positive X-axis pointing right. To specify the other grid line and/or direction for the positive X-axis, you must specify a hint angle. When you specify a hint angle, the positive X-axis will be the grid line intersecting the reference calibration point and closest in angle to the hint angle.<br><br>This control type is only used for camera calibrations performed using McalGrid() set to M_CHESSBOARD_GRID, and **McalControl**() with M_GRID_PARTIAL set to M_ENABLE. Setting this control type in any other circumstance will generate an error when calling McalGrid(). INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies that no hint angle is used. The positive X-axis is the right-pointing grid line closest to the horizontal.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the hint angle, measured counter-clockwise. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

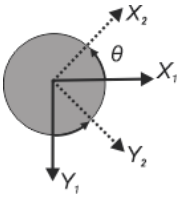
| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_GRID_HINT_PIXEL_X | Specifies the X-coordinate of the hint pixel. The hint pixel is used to help McalGrid() determine the location of the grid's reference calibration point. Specify an X- and Y-coordinate for the hint pixel that is close to the grid's reference calibration point in the image.<br><br>For complete grids, the corner calibration point closest to the origin of the pixel coordinate system (top-left corner of the image) is the default reference calibration point. If your grid's reference calibration point is not near the origin of the pixel coordinate system (for instance, if the grid is rotated in the image), you must specify a hint pixel. For complete grids, the corner calibration point closest to the hint pixel will be the grid's reference calibration point.<br><br>For complete grids, given that the number of rows and columns are specified, there can be either 2 valid corners (for a rectangle grid) or 4 corners (for a square grid), so the approximate X-coordinate does not have to be very precise.<br><br>For partial chessboard grids (M_GRID_PARTIAL set to M_ENABLE), the calibration point closest to the center of the image is the default reference calibration point. If you need your grid's reference calibration point in another place, you must specify a hint pixel; the calibration point closest to the hint pixel will be the grid's reference calibration point.<br><br>This control type is only used for camera calibrations performed using McalGrid(). INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Same as M_NONE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies not to use a hint pixel.<br><br>When using a complete grid, the calibration point closest to the top-left corner of the image is used as the grid's reference calibration point.<br><br>When using a partial grid, the calibration point closest to the center of the image is used as the grid's reference calibration point.<br><br>> Note that when you set M_GRID_HINT_PIXEL_X to M_NONE, you must also set M_GRID_HINT_PIXEL_Y to M_NONE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate of the hint pixel, in the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_GRID_HINT_PIXEL_Y | Specifies the Y-coordinate of the hint pixel. The hint pixel is used to help McalGrid() determine the location of the grid's reference calibration point. Specify an X- and Y-coordinate for the hint pixel that is close to the grid's reference calibration point in the image.<br><br>For complete grids, the corner calibration point closest to the origin of the pixel coordinate system (top-left corner of the image) is the default reference calibration point. If your grid's reference calibration point is not near the origin of the pixel coordinate system (for instance, if the grid is rotated in the image), you must specify a hint pixel. For complete grids, the corner calibration point closest to the hint pixel will be the grid's reference calibration point.<br><br>For complete grids, given that the number of rows and columns are specified, there can be either 2 valid corners (for a rectangle grid) or 4 corners (for a square grid), so the approximate Y-coordinate does not have to be very precise.<br><br>For partial chessboard grids (M_GRID_PARTIAL set to M_ENABLE), the calibration point closest to the center of the image is the default reference calibration point. If you need your grid's reference calibration point in another place, you must specify a hint pixel; the calibration point closest to the hint pixel will be the grid's reference calibration point.<br><br>This control type is only used for camera calibrations performed using McalGrid(). INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Same as M_NONE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies not to use a hint pixel.<br><br>When using a complete grid, the calibration point closest to the top-left corner of the image is used as the grid's reference calibration point.<br><br>When using a partial grid, the calibration point closest to the center of the image is used as the grid's reference calibration point.<br><br>> Note that when you set M_GRID_HINT_PIXEL_Y to M_NONE, you must also set M_GRID_HINT_PIXEL_X to M_NONE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate of the hint pixel, in the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_PARTIAL | Specifies whether McalGrid() can assume that the chessboard grid is complete. A complete grid is one the has the exact number of rows and columns specified when calling McalGrid(). INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_DISABLE | Specifies that McalGrid() will only calibrate the camera setup when a complete grid is found in the image.<br><br>For complete grids, the grid's default reference calibration point is the calibration point at the corner of the grid, closest to the top-left corner of the image. To specify the calibration point at another corner of the grid, for instance to indicate that the physical grid is rotated, you must specify a hint pixel using M_GRID_HINT_PIXEL_X and M_GRID_HINT_PIXEL_Y.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies that McalGrid() can calibrate the camera setup when a partial grid is found in the image.<br><br>When you enable partial grids, the calibration point closest to the center of the image is the grid's default reference calibration point. You can specify an alternative reference calibration point by specifying a hint pixel using M_GRID_HINT_PIXEL_X and M_GRID_HINT_PIXEL_Y.<br><br>When you enable partial grids, you can specify how to handle potential outlying false calibration points using M_GRID_SHAPE.<br><br>When enabling partial grids, you must specify that the grid is an M_CHESSBOARD_GRID type grid when calling McalGrid(), which includes both chessboard grids and fiducial grids. Enabling partial grids with an M_CIRCLE_GRID type grid will result in an error. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_SHAPE | | Specifies how to handle potential outlying calibration points, using assumptions about the grid's shape in the image.<br><br>When M_GRID_PARTIAL is set to M_DISABLE, you must set M_GRID_SHAPE to M_RECTANGLE.<br><br>McalGrid() will try to find the outside edges of a partial chessboard grid if it is assumed to be rectangular. This affects whether potential calibration points are included in the camera calibration or are excluded as outliers. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_RECTANGLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_ANY | Specifies to include all potential calibration points in the image; McalGrid() will not look for the boundary of the real-world grid. In this case, the shape of the partial grid is not bound.<br><br>Set this value when the partial grid in the image is not a rectangle. This can be either when the real-world grid is a rectangle, but is occluded by an object in the image, as in the image on the left, or when the real-world grid is not a rectangle, as in the image on the right.<br><br>  <br><br><br>With no fixed shape, no potential calibration point can be excluded as an outlier because it is not clear where the boundary of the partial grid is. This could lead to outlying false calibration points being included in the camera calibration, which would reduce the calibration's precision. In this case, it is highly recommended to check all calibration points extracted from the image using McalGrid() are valid, using McalDraw() with M_DRAW_IMAGE_POINTS. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_RECTANGLE | Specifies to exclude potential calibration points in the image that McalGrid() determines are outside the boundaries of the partial grid. In this case, the shape of the partial grid is a rectangle.<br><br>Set this value if the real-world grid is rectangular and the image of the partial grid is not occluded by any objects in the image. Note that a partial grid is still considered rectangular if the boundary of the partial grid is obscured by the end of the image.<br><br>  <br><br><br>In this case, McalGrid() will look for the boundaries of the partial grid in the image. This allows the possibility of excluding false calibration points outside of the grid. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LINK_TOOL_AND_HEAD | | Sets a rigid link between the head (camera or relative) coordinate system and the tool coordinate system.<br><br>When a rigid link exists between the tool and the head, moving one automatically moves the other accordingly. However, this link can be broken using M_LINK_TOOL_AND_HEAD set to M_DISABLE. You can then set the tool coordinate system to a known location without affecting the head coordinate system. After moving the tool coordinate system, you can then re-establish the link so that you move the head coordinate system by moving the tool coordinate system.<br><br>Note that for M_LINK_TOOL_AND_HEAD, the head represents a camera or a reference/calibration object depending on the type of robotic setup. When the robotic setup is M_MOVING_CAMERA, the head refers to a camera attached to the last joint of the robotic arm. When the robotic setup is M_STATIONARY_CAMERA, the head refers to a reference/calibration object attached to the last joint of the robotic arm. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DEFAULT | Specifies the default value.<br><br>Same as M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies to remove the link between the two coordinate systems, allowing both to be moved independently. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies to link the two coordinate systems, allowing both to be moved together. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LOCALIZATION_NB_ITERATIONS_MAX | Specifies the maximum number of iterations to attempt to fit the provided points when calculating the new position of the camera or relative coordinate system, when using McalList() and McalGrid() with M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD respectively. If the specified number of iterations is large, the computation could take longer to run, but the algorithm will be more robust. If M_LOCALIZATION_NB_OUTLIERS_MAX is set to 0, only 1 iteration is required and no more will be performed. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies the default value; the default value is 250. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 1 | Specifies the number of expected outliers. Only integer values are accepted. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LOCALIZATION_NB_OUTLIERS_MAX | Specifies the maximum number of possible outliers that can occur in the dataset used by McalList() or McalGrid() with M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD. If the specified number of outliers is large, more iterations are required to determine which points provide the best solution; therefore, computation can take longer, but the function will be more robust to outliers. Also, the function could fail if the value of M_LOCALIZATION_NB_ITERATIONS_MAX is too low. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies the default value; the default value is 0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0 | Specifies the number of expected outliers. Only integer values are accepted. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_POSITION_X | Sets the X-position of the tool coordinate system in the absolute coordinate system. The X-position is initialized to 0.0. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_POSITION_Y | Sets the Y-position of the tool coordinate system in the absolute coordinate system. The Y-position is initialized to 0.0. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_TOOL_POSITION_Z | Sets the Z-position of the tool coordinate system in the absolute coordinate system. The Z-position is initialized to 0.0.<br><br>When using the linear interpolation or perspective transformation camera calibration mode, M_TOOL_POSITION_Z must be set to M_DEFAULT. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies the default value; the default value is 0.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Z-coordinate, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_ROTATION_Z | Sets the angle at which to rotate the tool coordinate system about its Z-axis.<br><br>A positive angle rotates the tool coordinate system in the counter-clockwise direction (from the positive X-axis to the negative Y-axis of the tool coordinate system). The rotation angle is initialized to 0.0.<br><br>  <br><br><br>Note that this control is only available for 2D-based camera calibration modes (M_UNIFORM_TRANSFORMATION, M_LINEAR_INTERPOLATION, or M_PERSPECTIVE_TRANSFORMATION). INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies the default value; the default value is 0.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the angle, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRANSFORM_CACHE | Sets whether to use a cache to accelerate the McalTransformImage() function. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies not to use a cache. Not using a cache saves memory but slows down subsequent calls to McalTransformImage(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies to use a cache.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

The following ControlType and ControlValue parameter settings can be specified only for a 3D-based camera calibration context ( M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED):

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a 3D-based camera calibration context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ControlType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| ControlValue | | | | | | | | | | | | | | | | |
| M_CCD_ASPECT_RATIO | Sets the width-to-height ratio of the individual elements of the CCD. If this value is not set and is required by the camera calibration mode, the ratio will default to 1.0. INQ | | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1.0 | Specifies that the width and height of the CCD element are equal.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* > 0.0 | Specifies the value of the width of a CCD element divided by its height. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISTORTION_RADIAL_1 | | Sets the value of the second order radial distortion coefficient used in the camera calibration algorithm. Radial distortion refers to image distortions caused by the camera's lens.<br><br>Use this to override the value determined by the previous call to McalGrid() or McalList() if you want to use calibration values from another source. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the value of the second order radial distortion coefficient.<br><br>The default value is 0.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISTORTION_RADIAL_2 | | Sets the value of the fourth order radial distortion coefficient used in the camera calibration algorithm. Radial distortion refers to image distortions caused by the camera's lens. This radial distortion coefficient is only supported for M_ZHANG_BASED and M_3D_ROBOTICS_ZHANG_BASED camera calibration modes.<br><br>Use this to override the value determined by the previous call to McalGrid() or McalList() if you want to use calibration values from another source. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the value of the fourth order radial distortion coefficient.<br><br>The default value is 0.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FOCAL_LENGTH | | Sets the effective focal length of the pinhole camera model used in the camera calibration.<br><br>Use this to override the value determined by the previous call to McalGrid() or McalList() if you want to use calibration values from another source. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* > 0.0 | Specifies the effective focal length of the pinhole camera model, expressed in horizontal pixels.<br><br>The default value is 1.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PRINCIPAL_POINT_X | | Sets the X-coordinate of the intersection of the camera's optical axis and the image plane. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Specifies half of the image's width, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the X-coordinate, in pixels. This only has an effect if M_PRINCIPAL_POINT_Y is not set to M_DEFAULT. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PRINCIPAL_POINT_Y | | Sets the Y-coordinate of the intersection of the camera's optical axis and the image plane. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Specifies half of the image's height, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the Y-coordinate, in pixels. This only has an effect if M_PRINCIPAL_POINT_X is not set to M_DEFAULT. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

The following ControlType and ControlValue parameter settings can be specified for a calibrated image:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a calibrated image | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ControlType | | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| | ControlValue | | | | | | | | | | | | | | | | | |
| M_CALIBRATION_CHILD_OFFSET_X | | Sets the X-offset of a child buffer, relative to its highest order calibrated parent buffer. INQ | | | | | | | | | | | | | | | | |
| | *Value* | Specifies the X-offset, relative to the child buffer's highest order calibrated parent buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_CHILD_OFFSET_Y | | Sets the Y-offset of a child buffer, relative to its highest order calibrated parent buffer. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the Y-offset, relative to the child buffer's highest order calibrated parent buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

The following ControlType and ControlValue parameter settings can be specified for a calibrated image:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a calibrated and corrected image | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ControlType | | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| | ControlValue | | | | | | | | | | | | | | | | | |
| M_GRAY_LEVEL_SIZE_Z | | Sets the difference in height corresponding to a difference of one gray level in a fully-corrected depth map. INQ | | | | | | | | | | | | | | | | |
| | M_INVALID_SCALE | Specifies that the image is not a depth map.<br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* != 0.0 | Specifies the height, in world units, corresponding to a difference of one gray level. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_POS_Z | | Sets the real-world Z-offset of a gray level of 0 in the corrected image, useful in depth maps. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the base height of a gray level of 0.<br>The default value is 0.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

The following ControlType and ControlValue parameter settings can be specified for a 3D draw calibration context:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a 3D draw calibration context | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ControlType | | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| | ControlValue | | | | | | | | | | | | | | | | | |
| M_DRAW_ABSOLUTE_COORDINATE_SYSTEM | | Sets whether to draw the absolute coordinate system's axes. INQ | | | | | | | | | | | | | | | | |
| | M_DEFAULT | Same as M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_DISABLE | Specifies not to draw the absolute coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the absolute coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CAMERA_COORDINATE_SYSTEM | | Sets whether to draw the camera coordinate system's axes. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the camera coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the camera coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CAMERA_COORDINATE_SYSTEM_NAME | | Sets the name to draw for the camera coordinate system; the initial value is "Camera". INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | AIL_TEXT("*String*") 1 | Specifies the name of the camera coordinate system.<br>> Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*String*")). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_COORDINATE_SYSTEM_LENGTH | | Sets the length at which to draw the specified coordinate system's axes. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Specifies the default value; the default value is 50.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* > 0.0 | Specifies the length (in world units) at which to draw the axes of the specified coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_FRUSTUM | | Sets whether to draw the frustum of the camera's view. The frustum is the truncated pyramid of vision that originates at the camera's position and, when drawn, ends at the relative XY plane. When enabled, M_DRAW_FRUSTUM draws a wireframe frustum, using the color specified with M_DRAW_FRUSTUM_COLOR. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the frustum. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the frustum. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_FRUSTUM_COLOR | | Sets the frustum's color. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_COLOR_YELLOW. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_RGB888(<br>AIL_INT Red,<br>AIL_INT Green,<br>AIL_INT Blue<br>) | Specifies an RGB value to draw in an 8-bit, 3-band buffer. The red, green, and blue values must be values between 0 and 255, inclusive. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Parameters** | | | | | | | | | | | | | | | |
| | | Red<br><br>Specifies the red band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | | Green<br><br>Specifies the green band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | | Blue<br><br>Specifies the blue band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | M_COLOR_BLACK | Specifies the color black. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLUE | Specifies the color blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BRIGHT_GRAY | Specifies the color bright gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_CYAN | Specifies the color cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_BLUE | Specifies the color dark blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_CYAN | Specifies the color dark cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_GREEN | Specifies the color dark green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_MAGENTA | Specifies the color dark magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_RED | Specifies the color dark red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_YELLOW | Specifies the color dark yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GRAY | Specifies the color gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GREEN | Specifies the color green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_BLUE | Specifies the color light blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GRAY | Specifies the color light gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GREEN | Specifies the color light green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_WHITE | Specifies the color light white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_MAGENTA | Specifies the color magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_COLOR_RED | Specifies the color red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_WHITE | Specifies the color white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_YELLOW | Specifies the color yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NO_COLOR | Specifies no color. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_COORDINATE_SYSTEM | | Sets whether to draw the relative coordinate system's axes. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the relative coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the relative coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_COORDINATE_SYSTEM_NAME | | Sets the name to draw for the relative coordinate system; the initial value is "Relative". INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | AIL_TEXT("*String*") 1 | Specifies the name of the relative coordinate system.<br>> Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*String*")). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE | | Sets whether to draw the relative XY plane. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the relative XY plane. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the relative XY plane. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE_COLOR_FILL | | Sets the relative XY plane's fill color. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_AUTO_COLOR. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_RGB888(<br>AIL_INT Red,<br>AIL_INT Green,<br>AIL_INT Blue<br>) | Specifies an RGB value to draw in an 8-bit, 3-band buffer. The red, green, and blue values must be values between 0 and 255, inclusive. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Parameters** | | | | | | | | | | | | | | | |
| | | Red<br>Specifies the red band, as a value between 0 and 255. | | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Green<br><br>Specifies the green band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | | Blue<br><br>Specifies the blue band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | M_AUTO_COLOR | Specifies either the color white or the texture image.<br><br>If a texture image is specified (using McalDraw3d() with RelXYPlaneTextureImageBufId), the texture image is drawn on the laser plane. Otherwise, the plane is drawn with M_COLOR_WHITE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLACK | Specifies the color black. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLUE | Specifies the color blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BRIGHT_GRAY | Specifies the color bright gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_CYAN | Specifies the color cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_BLUE | Specifies the color dark blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_CYAN | Specifies the color dark cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_GREEN | Specifies the color dark green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_MAGENTA | Specifies the color dark magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_RED | Specifies the color dark red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_YELLOW | Specifies the color dark yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GRAY | Specifies the color gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GREEN | Specifies the color green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_BLUE | Specifies the color light blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GRAY | Specifies the color light gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GREEN | Specifies the color light green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_WHITE | Specifies the color light white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_MAGENTA | Specifies the color magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_RED | Specifies the color red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_COLOR_WHITE | Specifies the color white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_YELLOW | Specifies the color yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NO_COLOR | Specifies no color. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_TEXTURE_IMAGE | Specifies to use the image passed to McalDraw3d() with RelXYPlaneTextureImageBufId, when drawing the relative XY plane. The texture image is typically a 2D image of the 3D scene. For example, you can specify the image used for calibration. If no texture image is specified using McalDraw3d(), setting M_DRAW_RELATIVE_XY_PLANE_COLOR_FILL to M_TEXTURE_IMAGE will cause an error. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE_COLOR_OUTLINE | | Sets the relative XY plane's outline color. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_COLOR_WHITE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_RGB888(<br>AIL_INT Red,<br>AIL_INT Green,<br>AIL_INT Blue<br>) | Specifies an RGB value to draw in an 8-bit, 3-band buffer. The red, green, and blue values must be values between 0 and 255, inclusive. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Parameters** | | | | | | | | | | | | | | | |
| | | Red<br><br>Specifies the red band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | | Green<br><br>Specifies the green band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | | Blue<br><br>Specifies the blue band, as a value between 0 and 255. | | | | | | | | | | | | | | | |
| | M_COLOR_BLACK | Specifies the color black. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLUE | Specifies the color blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BRIGHT_GRAY | Specifies the color bright gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_CYAN | Specifies the color cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_BLUE | Specifies the color dark blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_CYAN | Specifies the color dark cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_GREEN | Specifies the color dark green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_MAGENTA | Specifies the color dark magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_COLOR_DARK_RED | Specifies the color dark red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_YELLOW | Specifies the color dark yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GRAY | Specifies the color gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GREEN | Specifies the color green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_BLUE | Specifies the color light blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GRAY | Specifies the color light gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GREEN | Specifies the color light green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_WHITE | Specifies the color light white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_MAGENTA | Specifies the color magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_RED | Specifies the color red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_WHITE | Specifies the color white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_YELLOW | Specifies the color yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NO_COLOR | Specifies no color. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE_OPACITY | | Sets the relative XY plane's opacity. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Specifies the default value; the default value is 20.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | 0.0 <= *Value* <= 100.0 | Specifies the relative XY plane's opacity. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_ROBOT_BASE_COORDINATE_SYSTEM | | Sets whether to draw the robot base coordinate system's axes. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the robot base coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the robot base coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_TOOL_COORDINATE_SYSTEM | | Sets whether to draw the tool coordinate system's axes. INQ | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Same as M_DISABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_DISABLE | Specifies not to draw the tool coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the tool coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

## Type-specific versions of the function when using a C compiler under 64-bit

```
void McalControlInt64 (AIL_ID ContextCalOrCalibratedAilObjectId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

### Parameters

ContextCalOrCalibratedAilObjectId

See ContextCalOrCalibratedAilObjectId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void McalControlDouble (AIL_ID ContextCalOrCalibratedAilObjectId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

### Parameters

ContextCalOrCalibratedAilObjectId

See ContextCalOrCalibratedAilObjectId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalCopyResult

## Synopsis

Copy a group of results from a calculate hand-eye result buffer into a transformation matrix object.

## Syntax

```
void McalCopyResult(
    AIL_ID CalculateHandEyeResultCalId, //in
    AIL_INT64 ResultIndex,              //in
    AIL_ID Matrix3dgeoId,               //out
    AIL_INT64 CopyType,                 //in
    AIL_INT64 ControlFlag               //in
)
```

## Description

This function copies a group of results from a calculate hand-eye result buffer into a transformation matrix object.

You can only copy calculate hand-eye results after calling McalCalculateHandEye().

## Parameters

CalculateHandEyeResultCalId

Specifies the identifier of the source calculate hand-eye result buffer from which to copy. The calculate hand-eye result buffer must have been allocated using McalAllocResult() with M_CALCULATE_HAND_EYE_RESULT, and must contain the results of a call to McalCalculateHandEye().

ResultIndex

Specifies a pose or an average of all poses from the source calculate hand-eye result buffer that the transformation matrix object will correspond to.

| For specifying the pose | |
|---|---|
| Value | Description |
| M_LAST | Specifies to use the last pose from the source calculate hand-eye result buffer. |
| M_MEAN | Specifies the use the average of all the poses from the source calculate hand-eye result buffer.<br>This is the default value. |
| 0 <= *Value* < *Num poses* | Specifies to use a specific pose from the source calculate hand-eye result buffer. |

Matrix3dgeoId

Specifies the identifier of a transformation matrix object in which to copy. The matrix object must have been allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. The type of result that is copied depends on the copy operation (CopyType).

CopyType

Specifies the type of copy operation to perform.

| Specifies the type of copy operation | |
|---|---|
| Value | Description |

| | |
|---|---|
| M_MATRIX_X | Specifies to copy the transformation matrix X into the transformation matrix object. What this transformation matrix can be used for depends on what was passed as matrices to A (HandMatrix3dgeoIdArrayPtr) and B (EyeMatrix3dgeoIdArrayPtr) for the system of matrices AX = ZB when McalCalculateHandEye() was called. For example, if the matrix that transforms points from the tool coordinate system to the robot base coordinate system was passed as A, and the matrix that transforms points from the camera coordinate system to the reference object coordinate system was passed as B, then matrix X would transform points from the camera coordinate system to the tool coordinate system.<br><br>> Note that X and Z in this equation do not relate to the X and Z-axes. |
| M_MATRIX_Z | Specifies to copy the transformation matrix Z into the transformation matrix object. What this transformation matrix can be used for depends on what was passed as matrices to A (HandMatrix3dgeoIdArrayPtr) and B (EyeMatrix3dgeoIdArrayPtr) for the system of matrices AX = ZB when McalCalculateHandEye() was called. For example, if the matrix that transforms points from the tool coordinate system to the robot base coordinate system was passed as A, and the matrix that transforms points from the camera coordinate system to the reference object coordinate system was passed as B, then matrix Z would transform points from the reference object coordinate system to the robot base coordinate system.<br><br>> Note that X and Z in this equation do not relate to the X and Z-axes. |

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalDraw

## Synopsis

Draw specific features of results obtained from a camera calibration operation.

## Syntax

```
void McalDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ContextCalOrImageBufId, //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT Index,                 //in
    AIL_INT64 ControlFlag          //in
)
```

## Description

This function draws specific features of results, obtained from a camera calibration operation, in the destination image buffer or 2D graphics list.

## Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the 2D graphics context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used.<br>Note that there is a different default 2D graphics context for each thread. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *2D graphics context identifier* | Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ContextCalOrImageBufId

Specifies the camera calibration context, calibrated image, corrected image, or fixturing offset object from which to extract the information to draw.

When specifying a calibrated image or corrected image, the image cannot be associated with a region of interest (ROI), otherwise an error will occur. The image must be a 1- or 3-band buffer.

If passing a camera calibration context, it must have been previously allocated on the required system using McalAlloc(). The camera calibration context should have been successfully calibrated using McalList() or McalGrid().

When working in M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration mode, it is possible to use **McalDraw**() with M_DRAW_IMAGE_POINTS, and for fiducial grids M_DRAW_FIDUCIAL_BOX, between two successful calls to McalGrid() / McalList(), to draw the calibration points or the fiducial box of one camera calibration pose before being fully calibrated.

If passing a fixturing offset object, it must have been previously allocated on the required system using McalAlloc() with M_FIXTURING_OFFSET. In addition the fixturing offset object should have been set up using McalFixture() with M_LEARN_OFFSET.

If ContextCalOrImageBufId is set to **M_NULL**, the coordinate system of the destination image is used.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. If a buffer is specified, it must be a 1- or

3-band, unsigned, 8-bit image buffer. By drawing into the display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

If you specify a 2D graphics list, the 2D graphics list must have been previously allocated using MgraAllocList().

Operation

Specifies the type of operation to perform.

For a camera calibration context, calibrated image, or corrected image, you can set the Operation parameter to one of the following values. Note that if no combination constant is specified, the coordinate system will be drawn as if M_DRAW_ALL were specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

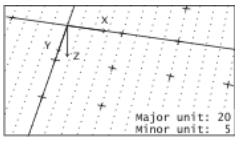
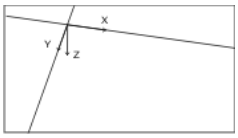
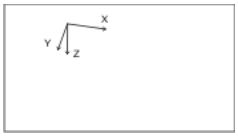
| For a camera calibration context, calibrated image, or corrected image | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DRAW_ABSOLUTE_COORDINATE_SYSTEM + | Draws the absolute coordinate system of the specified source camera calibration context or calibrated image (ContextCalOrImageBufId). If ContextCalOrImageBufId is set to **M_NULL**, the function draws the absolute coordinate system of the destination image, DstImageBufOrListGraId. In the case of a 2D graphics list being passed as the destination, the function will use the camera calibration information associated with the image in which the graphics in the 2D graphics list are rendered; when the 2D graphics list is associated with a display, it uses the camera calibration information of the image currently selected to the display.<br><br>An error is reported if ContextCalOrImageBufId is **M_NULL** and the image passed to DstImageBufOrListGraId is not a calibrated image. In the case of a 2D graphics list being passed as the destination, the error will occur when the graphics in the 2D graphics list are rendered in an uncalibrated image or display.<br><br>If both ContextCalOrImageBufId and DstImageBufOrListGraId have camera calibration information associated with them, the camera calibration information associated with ContextCalOrImageBufId will be used to define the position of the coordinate system, while DstImageBufOrListGraId will be used to define the mapping of the coordinates between world and pixel units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_PIXEL_COORDINATE_SYSTEM + | Draws the pixel coordinate system of the destination image, DstImageBufOrListGraId with the origin placed at the center of the top-left pixel. ContextCalOrImageBufId must be set to **M_NULL**.<br><br>In the case of a 2D graphics list being passed as the destination, the function will draw the pixel coordinate system of the image in which the graphics in the 2D graphics list are rendered; when the 2D graphics list is associated with a display, it draws the pixel coordinate system of the image currently selected to the display. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DRAW_RELATIVE_COORDINATE_SYSTEM + | Draws the relative coordinate system of the specified source camera calibration context or calibrated image (ContextCalOrImageBufId). If ContextCalOrImageBufId is set to **M_NULL**, the function draws the relative coordinate system of the destination image, DstImageBufOrListGraId. In the case of a 2D graphics list being passed as the destination, the function will use the camera calibration information associated with the image in which the graphics in the 2D graphics list are rendered; when the 2D graphics list is associated with a display, it uses the camera calibration information of the image currently selected to the display.<br><br>An error is reported if ContextCalOrImageBufId is **M_NULL** and the image passed to DstImageBufOrListGraId is not a calibrated image. In the case of a 2D graphics list being passed as the destination, the error will occur when the graphics in the 2D graphics list are rendered in an uncalibrated image or display.<br><br>If both ContextCalOrImageBufId and DstImageBufOrListGraId have camera calibration information associated with them, the camera calibration information associated with ContextCalOrImageBufId will be used to define the position of the coordinate system, while DstImageBufOrListGraId will be used to define the mapping of the coordinates between world and pixel units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for the values listed in For a camera calibration context, calibrated image, or corrected image.

You can add one of the following values to the above-mentioned values to specify how to draw the coordinate system.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying how to draw the coordinate system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DRAW_ALL | Draws the coordinate system using all the features listed in this table. The behavior of M_DRAW_AXES is chosen over M_DRAW_FRAME. The following is an example of the annotations.<br><br>  <br><br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_AXES | Draws the X-axis and Y-axis lines with arrow heads and axis labels. If the Z-axis can be seen, it is drawn with an arrow and label. The following is an example of the annotations.<br><br>  <br> | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_FRAME | Draws three arrows (instead of full axes) and axis labels. The following is an example of the annotations.<br><br>  <br> | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DRAW_LEGEND | Draws the legend in the image at the bottom-right. The major unit describes the distance in real-world units between two major marks, and the minor unit describes the distance in real-world units between two minor marks. The following is an example of the annotations. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_MAJOR_MARKS | Draws crosses at each major tick mark. The following is an example of the annotations. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_MINOR_MARKS | Draws dots at each minor tick mark. The following is an example of the annotations. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_DRAW_ABSOLUTE_COORDINATE_SYSTEM; M_DRAW_RELATIVE_COORDINATE_SYSTEM.

You can add the following value to the above-mentioned values to specify a new location for the axes to cross if drawing the world coordinate (0,0) causes an arrow not to be visible.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying where the axes will be displayed | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_ALWAYS_SHOW_AXES | Specifies to choose a new location for the axes to cross if, when trying to draw the lines and arrows at world coordinate (0,0), part of the axes would not otherwise be visible . | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a camera calibration context, you can set the Operation parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DRAW_FIDUCIAL_BOX | Draws a box around the fiducial located during calibration, for fiducial grids with a Data Matrix code (McalGrid() with M_GRID_FIDUCIAL set to M_DATAMATRIX). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_IMAGE_POINTS + | Draws the calibration points, used with McalGrid() or McalList(), to calibrate the source camera calibration context (ContextCalOrImageBufId). It draws the points at the original pixel positions specified (or established) for the calibration points during camera calibration; it does not use the mapping established during camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_VALID_REGION | Draws a contour of the rectangle surrounding the calibrated area when using piecewise linear camera calibration mode (M_LINEAR_INTERPOLATION). The points transformed in this calibrated area are never extrapolated, only interpolated. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DRAW_VALID_REGION_FILLED | Draws a filled rectangle on the calibrated area when using piecewise linear camera calibration mode (M_LINEAR_INTERPOLATION). The points transformed in this calibrated area are never extrapolated, only interpolated. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_WORLD_POINTS + | Draws the real-world positions of the calibration points, used with McalGrid() or McalList(), to calibrate the source camera calibration context (ContextCalOrImageBufId).<br><br>To draw the real-world positions, it transforms these positions to pixel units using the camera calibration context associated with destination image, or using ContextCalOrImageBufId if the destination image is not calibrated. It does not use the original pixel positions specified (or established) for the calibration points during camera calibration; it uses the mapping established during camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_DRAW_IMAGE_POINTS; M_DRAW_WORLD_POINTS.

You can add the following value to the above-mentioned values to specify to draw arrows representing the calibration error.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying to draw the calibration error | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DRAW_CALIBRATION_ERROR | Specifies that instead of drawing crosshairs at calibration points, this draws an arrow between the original pixel calibration points and the real-world positions of the calibration points transformed into pixel units using the camera calibration context (associated with destination image, or specified using ContextCalOrImageBufId if the destination image is not calibrated). For example, if you specify M_DRAW_IMAGE_POINTS, the arrow is drawn from the calibration points of M_DRAW_IMAGE_POINTS to the corresponding calibration points of M_DRAW_WORLD_POINTS. In practice, the error is too small to see, so it can be scaled up using McalControl() with M_DRAW_CALIBRATION_ERROR_SCALE_MODE or M_DRAW_CALIBRATION_ERROR_SCALE_FACTOR. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a fixturing offset object, you can set the Operation parameter to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a fixturing offset object | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DRAW_FIXTURING_OFFSET | Specifies to draw the reference location that would have been used to place the current relative coordinate system of the destination image, given the specified fixturing offset object. **McalDraw**() draws a line from the current origin of the relative coordinate system to the inferred reference position. It also draws a small arrow at this position, which illustrates the inferred reference angle.<br><br>If the destination image is not calibrated, **McalDraw**() behaves as if it were calibrated with a uniform scale of 1 and an absolute and relative coordinate system equal to the pixel coordinate system. Note, however, that the destination image remains uncalibrated.<br><br>In the case of a 2D graphics list being passed as the destination, the function will use the camera calibration information associated to the image at the time of annotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Index

Specifies the index of the pose of an M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context from which the calibration points or fiducial box should be drawn. If this information is not required or supported, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the index of the pose | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Specifies to draw the calibration points of the last pose, for M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration contexts. For all other cases, implements the default behavior. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0 <= *Value* < *Number of poses* | Specifies the index of a camera calibration pose. Use McalInquire() with M_NUMBER_OF_CALIBRATION_POSES to determine the number of poses.<br><br>This setting must be used only when drawing calibration points of a camera calibration context in M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED mode. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalDraw3d

## Synopsis

Draw 3D annotations based on a calibrated or corrected image.

## Syntax

```
AIL_INT64 McalDraw3d(
    AIL_ID OperationDraw3dContextCalId, //in
    AIL_ID ContextCalOrImageBufId,      //in
    AIL_INT64 SrcIndex,                 //in
    AIL_ID DstList3dgraId,              //out
    AIL_INT64 DstParentLabel,           //in
    AIL_ID RelXYPlaneTextureImageBufId, //in
    AIL_INT64 ControlFlag               //in
)
```

## Description

This function draws 3D annotations (for example, the camera's coordinate system) based on a 3D draw calibration context, in a 3D graphics list. Set the draw operations and options for the draw using McalControl(). Example operations include drawing the relative XY plane and the frustum of the camera's view.

## Parameters

OperationDraw3dContextCalId

Specifies the identifier of the 3D draw calibration context that specifies the annotations to draw and how to draw them. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the 3D draw calibration context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Specifies that the default 3D draw calibration context of the current Aurora Imaging Library application is used. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *3D draw calibration context identifier* | Specifies a valid 3D draw calibration context identifier, which you have allocated using McalAlloc() with M_DRAW_3D_CONTEXT. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ContextCalOrImageBufId

Specifies the identifier of the camera calibration context or calibrated image.

If passing a camera calibration context, it must have been previously allocated on the required system using McalAlloc().

When specifying a calibrated image, the image cannot be associated with a region of interest (ROI), otherwise an error will occur. The image must be a 1- or 3-band buffer.

SrcIndex

Reserved for future expansion and must be set to **M_DEFAULT**.

DstList3dgraId

Specifies the identifier of the 3D graphics list in which to draw. You can specify a 3D graphics list that you have previously allocated using M3dgraAlloc(), or you can specify the identifier of the 3D display's internal graphics list (inquired using M3ddispInquire() with M_3D_GRAPHIC_LIST_ID).

DstParentLabel

Specifies the label of the 3D graphic in the 3D graphics list to be used as the annotation's parent.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the parent label | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_ROOT_NODE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROOT_NODE | Specifies the top-most node of the 3D graphics list. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0 | Specifies the label of the 3D graphic in the 3D graphics list. Label 0 is the 3D graphics list's root node. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

RelXYPlaneTextureImageBufId

Specifies the identifier of the image buffer containing the texture image, which you can apply to the relative XY plane to enhance its visibility. The texture image is typically a 2D image of the 3D scene. For example, you can specify the image used for calibration.

The image buffer must be an 8-bit, unsigned, 1- or 3-band buffer, and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Set this parameter to **M_NULL** if not used.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

## Return value

Returns the parent label of the graphics added to the graphics list.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalFixture

## Synopsis

Move the relative coordinate system or learn the fixturing offset.

## Syntax

```
void McalFixture(
    AIL_ID DstCalibratedAilObjectId, //out
    AIL_ID FixturingOffsetCalId,     //in
    AIL_INT64 Operation,             //in
    AIL_INT64 LocationType,          //in
    AIL_ID CalOrLocSourceId,         //in
    AIL_DOUBLE Param1,               //in
    AIL_DOUBLE Param2,               //in
    AIL_DOUBLE Param3,               //in
    AIL_DOUBLE Param4                //in
)
```

## Description

This function allows you to move the relative coordinate system to a location (position and angle) that is at a fixed offset from an object that you want to analyze. Alternatively, this function can write the values required for the move to the specified 3D transformation matrix object. To specify the location, you must pass the calculated reference location for the object and, optionally, a preestablished offset to apply to this reference location. To pass the preestablished offset, you must pass a fixturing offset object that contains the information. Use a separate call to this function to set up the fixturing offset object.

When moving the relative coordinate system (M_MOVE_RELATIVE), you can specify the reference location of the object in one of two ways, based upon some previous analysis. You can have **McalFixture**() use, for example, the position and angle of a model occurrence from a specified Aurora Imaging Library Pattern Matching or Model Finder result buffer. Alternatively, you can explicitly specify a location with respect to the relative coordinate system of the Aurora Imaging Library object passed to CalOrLocSourceId; internally, **McalFixture**() converts this location so that it is with respect to the absolute coordinate system, and then uses the calculated location to place the new relative coordinate system.

> Note that if you know the new position and angle for the relative coordinate system in the absolute coordinate system, use McalSetCoordinateSystem() or McalRelativeOrigin() to move it explicitly. Alternatively, you can specify the default uniform camera calibration context (M_DEFAULT_UNIFORM_CALIBRATION) as the coordinate system in which the location is expressed (CalOrLocSourceId). This camera calibration context's relative coordinate system is always aligned with its absolute coordinate system (no angle, no offset, no scale). Therefore, the specified location is consequently interpreted as being relative to the absolute coordinate system.

When setting up the fixturing offset object (M_LEARN_OFFSET), **McalFixture**() learns the offset between a specified location and the relative coordinate system. You can specify the location using the same techniques as those available when moving the relative coordinate system. In addition, you can set it to the reference location of a specified model in its original model source image. The function then saves the offset between the location and its relative coordinate system, in the specified fixturing offset object. Typically, you have obtained the location from a training image, whereby the location is a typical reference location and the relative coordinate system is at a required offset from this location.

**McalFixture**() assumes that the reference location and preestablished offset use the same absolute coordinate system as the image/calibrated context in which to save the new relative coordinate system, even if their mapping to the pixel coordinate system is different.

For more information on fixturing, see Chapter 29: Fixturing in Aurora Imaging Library.

Note that only 1- and 3-band image buffers are supported.

If you adjust the relative coordinate system of a calibrated image associated with an M_VECTOR_AND_RASTER ROI, the raster information will be discarded, causing the ROI to become an M_VECTOR ROI. See MbufSetRegion() for more information.

## Parameters

DstCalibratedAilObjectId

Specifies the identifier of the destination camera calibration context, 3D transformation matrix object, the 3D reconstruction result

buffer, or the image for which to displace the relative coordinate system. If the Operation parameter is set to M_LEARN_OFFSET, this parameter should be set to **M_NULL**.

> Note that if an image is passed but is not associated with a camera calibration context, it is automatically considered to be associated with a uniform camera calibration context where there is no scaling, rotation, or offset present. Therefore, each pixel in the image is considered to represent the same area in real-world units and the image is assumed to have no non-linear distortions.

FixturingOffsetCalId

Specifies the identifier of the fixturing offset object to set up (M_LEARN_OFFSET) or to apply to the reference location when moving the relative coordinate system (M_MOVE_RELATIVE). The fixturing offset object must have been previously allocated using McalAlloc(). If you don't need to apply a fixturing offset when moving the relative coordinate system, set this parameter to **M_NULL**.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

LocationType

Specifies how the location used for the operation is specified.

See the Parameter associations section for possible values that can be specified.

CalOrLocSourceId

Specifies the identifier of the Aurora Imaging Library object containing the location for the operation, or containing the coordinate system in which the location is expressed.

When this parameter is not used, it should be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies information about the location used for the operation. Its definition is dependent on the specified operation and the location type.

When this parameter is not used, it should be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies information about the location used for the operation. Its definition is dependent on the specified operation and the location type.

When this parameter is not used, it should be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies information about the location used for the operation. Its definition is dependent on the specified operation and the location type.

When this parameter is not used, it should be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies information about the location used for the operation. Its definition is dependent on the specified operation and the location type.

When this parameter is not used, it should be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

## Parameter associations

The table below lists possible values for the **Operation**, **LocationType**, **CalOrLocSourceId**, **Param1**, **Param2**, **Param3**, and **Param4** parameters.

- For specifying the location used for the operation

> Any parameter that is not used should be set to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the location used for the operation | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Operation | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| LocationType | | | | | | | | | | | | | | | | |
| CalOrLocSourceId | | | | | | | | | | | | | | | | |
| Param1 | | | | | | | | | | | | | | | | |
| Param2 | | | | | | | | | | | | | | | | |
| Param3 | | | | | | | | | | | | | | | | |
| Param4 | | | | | | | | | | | | | | | | |
| M_LEARN_OFFSET | Learns the offset between the specified location and the coordinate system in which it is expressed, and saves this offset in the specified fixturing offset object. INQ | | | | | | | | | | | | | | | |
| M_MODEL_MOD | Specifies to use the location of a Model Finder model's reference axis (that is, the model's reference location) in the model source image. Note that this location is with respect to the model source image's top-left corner (the top-left corner of the original image from which the model was defined), and not the model image itself.<br><br>To set the reference location of the model, use MmodControl() with M_REFERENCE_X, M_REFERENCE_Y, and M_REFERENCE_ANGLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the Model Finder context identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the index of the model within the context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MODEL_PAT | Specifies to use the reference location of a Pattern Matching model in its source image.<br><br>To set the reference location of the model, use MpatControl() with M_REFERENCE.... | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the Pattern Matching context identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies a camera calibration context or calibrated image that defines the pixel-to-world mapping of the Pattern Matching model's source image.<br><br>> This is needed because this association is not stored with the model.<br><br>If the Pattern Matching model was defined from an uncalibrated image, set this parameter to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the index of the model within the context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_POINT_AND_ANGLE | Specifies to use the specified point and angle as the location. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the relative coordinate system in which the point and angle are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT_UNIFORM_CALIBRATION | Specifies to use the relative coordinate system of a uniform camera calibration where the absolute and relative coordinate systems are equal to the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibrated image ID* | Specifies the identifier of the calibrated image.<br>> If an image is passed but is not associated with a camera calibration context, it will be treated as if associated with a uniform camera calibration context where there is no scaling, rotation, or offset present. Therefore, each pixel in the image will be considered to represent the same area in real-world units and it will be assumed that no non-linear distortions are present within the image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibration context ID* | Specifies the identifier of the camera calibration context which defines the relative coordinate system in which the point and angle are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the X-coordinate of the point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Y-coordinate of the point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the angle, in degrees. The angle is measured from the positive X-axis towards the negative Y-axis of the specified relative coordinate system, as described in the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_POINT_AND_DIRECTION_POINT | Specifies to use two points to define the reference location. The first point defines the position, and the angle of the vector going from the first point to the second point defines the angle.<br>If there is no fixturing offset object, the origin of the new relative coordinate system is placed at the first point and the angle of the X-axis of the new relative coordinate system is set to the angle of the vector going from the first point to the second point. The Y-axis is 90° from the X-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the relative coordinate system in which the two points are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT_UNIFORM_CALIBRATION | Specifies to use the relative coordinate system of a uniform camera calibration where the absolute and relative coordinate systems are equal to the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | *Calibrated image ID* | Specifies the identifier of the calibrated image.<br><br>> If an image is passed but is not associated with a camera calibration context, it will be treated as if associated with a uniform camera calibration context where there is no scaling, rotation, or offset present. Therefore, each pixel in the image will be considered to represent the same area in real-world units and it will be assumed that no non-linear distortions are present within the image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | | *Calibration context ID* | Specifies the identifier of the camera calibration context which defines the relative coordinate system in which the point and angle are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Param1** | | Specifies the X-coordinate of the first point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Param2** | | Specifies the Y-coordinate of the first point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Param3** | | Specifies the X-coordinate of the second point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Param4** | | Specifies the Y-coordinate of the second point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_RESULT_MET | | | Specifies to use the resulting position and angle of a metrology reference frame. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **CalOrLocSourceId** | | Specifies the identifier of a Metrology result buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Param1** | | Specifies the index of the reference frame feature to use. An index value of 0 represents the global frame. Note that the only metrology features supported are local or global frame results. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_RESULT_MOD | | | Specifies to use the position and angle of a Model Finder model occurrence. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **CalOrLocSourceId** | | Specifies the identifier of the Model Finder result buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Param1** | | Specifies the index of the result occurrence. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_RESULT_PAT | | | Specifies to use the position and angle of a Pattern Matching model occurrence. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **CalOrLocSourceId** | | Specifies the identifier of the Pattern Matching result buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | | **Param1** | | Specifies the index of the result occurrence. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_MOVE_RELATIVE | Moves the relative coordinate system to a location that is at a specified offset from the specified reference location. If no fixturing offset object is specified, the relative coordinate system is placed at the specified reference location. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LASER_3DMAP | Specifies to use the laser line coordinate system, defined by the 3D reconstruction context, to define the location. Note that the context must have been allocated using M_CALIBRATED_CAMERA_LINEAR_MOTION and have been calibrated using M3dmapCalibrate() or M3dmapCalibrateMultiple(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the identifier of the 3D reconstruction context which defines the laser line coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_POINT_AND_ANGLE | Specifies to use the specified point and angle as the reference location. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the relative coordinate system in which the point and angle are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies to use the current relative coordinate system of the target calibrated image or target camera calibration context (DstCalibratedAilObjectId). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT_UNIFORM_CALIBRATION | Specifies to use the relative coordinate system of a uniform camera calibration where the absolute and relative coordinate systems are equal to the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibrated image ID* | Specifies the identifier of the calibrated image.<br>> If an image is passed but is not associated with a camera calibration context, it will be treated as if associated with a uniform camera calibration context where there is no scaling, rotation, or offset present. Therefore, each pixel in the image will be considered to represent the same area in real-world units and it will be assumed that no non-linear distortions are present within the image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibration context ID* | Specifies the identifier of the camera calibration context which defines the relative coordinate system in which the point and angle are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the X-coordinate of the point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Y-coordinate of the point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the angle, in degrees. The angle is measured from the positive X-axis towards the negative Y-axis of the specified relative coordinate system, as described in the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_POINT_AND_DIRECTION_POINT | Specifies to use two points to define the reference location. The first point defines the position, and the angle of the vector going from the first point to the second point defines the angle.<br><br>If there is no fixturing offset object, the origin of the new relative coordinate system is placed at the first point and the angle of the X-axis of the new relative coordinate system is set to the angle of the vector going from the first point to the second point. The Y-axis is 90° from the X-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the relative coordinate system in which the two points are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies to use the current relative coordinate system of the target calibrated image or target camera calibration context (DstCalibratedAilObjectId). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT_UNIFORM_CALIBRATION | Specifies to use the relative coordinate system of a uniform camera calibration where the absolute and relative coordinate systems are equal to the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibrated image ID* | Specifies the identifier of the calibrated image.<br><br>> If an image is passed but is not associated with a camera calibration context, it will be treated as if associated with a uniform camera calibration context where there is no scaling, rotation, or offset present. Therefore, each pixel in the image will be considered to represent the same area in real-world units and it will be assumed that no non-linear distortions are present within the image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibration context ID* | Specifies the identifier of the camera calibration context which defines the relative coordinate system in which the point and angle are expressed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the X-coordinate of the first point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Y-coordinate of the first point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the X-coordinate of the second point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param4** | Specifies the Y-coordinate of the second point, expressed in the specified relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RESULT_MET | Specifies to use the resulting position and angle of a metrology reference frame as the reference location. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the identifier of a Metrology result buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the index of the frame feature to use. An index value of 0 represents the global frame. Note that the only metrology features supported are local or global frame results. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_RESULT_MOD | Specifies to use the position and angle of a Model Finder model occurrence as the reference location. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the identifier of the Model Finder result buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the index of the result occurrence. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RESULT_PAT | Specifies to use the position and angle of a Pattern Matching model occurrence as the reference location. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the identifier of the Pattern Matching result buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the index of the result occurrence. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RESULT_POINT_CLOUD_3DMAP | Specifies to use the relative coordinate system of the 3D reconstruction result buffer to define the location. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the identifier of the 3D reconstruction result buffer, which defines the relative coordinate system of the result buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_SAME_AS_SOURCE | Specifies that the relative coordinate system of CalOrLocSourceId is copied into DstCalibratedAilObjectId and takes the fixturing offset into account (if any). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **CalOrLocSourceId** | Specifies the camera calibration context to be copied. This can be any calibrated object. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalFree

## Synopsis

Free a camera calibration context, a fixturing offset object, or a calibration result buffer.

## Syntax

```
void McalFree(
    AIL_ID CalibrationId //in
)
```

## Description

This function frees a camera calibration context, a fixturing offset object, or a calibration result buffer.

All camera calibration contexts allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

## Parameter

CalibrationId

Specifies the identifier of the camera calibration context, fixturing offset object or calibration result buffer.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalGetCoordinateSystem

## Synopsis

Return the position and orientation of a coordinate system.

## Syntax

```
void McalGetCoordinateSystem(
    AIL_ID CalibratedAilObjectId,        //in
    AIL_INT64 TargetCoordinateSystem,    //in
    AIL_INT64 ReferenceCoordinateSystem, //in
    AIL_INT64 InquireType,               //in
    AIL_ID ArrayBufOrMatrix3dgeoId,      //out
    AIL_DOUBLE *Param1Ptr,               //out
    AIL_DOUBLE *Param2Ptr,               //out
    AIL_DOUBLE *Param3Ptr,               //out
    AIL_DOUBLE *Param4Ptr                //out
)
```

## Description

This function returns the position and orientation of one coordinate system as a transformation of another coordinate system. For example, you can use this function to inquire about the position of your camera in the absolute coordinate system; essentially, you can inquire about the origin and orientation of the camera coordinate system with respect to the absolute coordinate system. Note that to move a specified (target) coordinate system relative to a reference coordinate system, you can use McalSetCoordinateSystem().

Note that **McalGetCoordinateSystem**() returns angles in degrees. However, unlike most other Aurora Imaging Library functions, this function returns angles following the right-hand grip rule around the axis of rotation; if you wrap your right hand around the axis of rotation, pointing your thumb in the positive direction of the axis, your fingers wrap in the direction of rotation. For example, a positive rotation around the Z-axis corresponds to a rotation that turns the positive X-axis toward the positive Y-axis.

## Parameters

CalibratedAilObjectId

Specifies the identifier of a camera calibration context, a 3D reconstruction context of type M_CALIBRATED_CAMERA_LINEAR_MOTION, a 3D reconstruction result buffer of type M_POINT_CLOUD_RESULT, or any object that has camera calibration information, such as an image or a result buffer.

> Note that if you specify a 3D reconstruction context, it must have already been calibrated using M3dmapCalibrate() or M3dmapCalibrateMultiple().

TargetCoordinateSystem

Specifies the coordinate system for which to inquire about its position and orientation. This parameter must be set to one of the following values.

> Note that if you specify a 3D reconstruction context, it must have already been calibrated using M3dmapCalibrate() or M3dmapCalibrateMultiple(). In this case, TargetCoordinateSystem can only be set to either M_ABSOLUTE_COORDINATE_SYSTEM or M_LASER_LINE_COORDINATE_SYSTEM.

> Note that when CalibratedAilObjectId is set to a 3D reconstruction result buffer, TargetCoordinateSystem can only be set to either M_ABSOLUTE_COORDINATE_SYSTEM or M_RELATIVE_COORDINATE_SYSTEM.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the target coordinate system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_ABSOLUTE_COORDINATE_SYSTEM | Specifies an implicit and unmovable coordinate system from which all other coordinate systems are defined. By default, it corresponds to the center of the top-left calibration point of the camera calibration grid if calibrated using McalGrid(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CAMERA_COORDINATE_SYSTEM | Specifies the coordinate system whose origin corresponds to the effective pinhole of the camera and whose Z-axis points in the direction that the camera is facing. This coordinate system is only defined after a successful camera calibration in M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED mode. Alternatively, you can define this coordinate system using McalSetCoordinateSystem() with M_ASSIGN. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LASER_LINE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is on the first laser line at the point closest to the origin of the absolute coordinate system.<br>This value is only supported when CalibratedAilObjectId is set to a valid calibrated 3D reconstruction context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_COORDINATE_SYSTEM | Specifies the coordinate system which determines the world plane used when working with world units which can be recentered and/or re-oriented using McalSetCoordinateSystem() or McalRelativeOrigin(). By default, it corresponds to the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROBOT_BASE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is positioned at the base of the robot holding the camera. This coordinate system is only supported for M_3D_ROBOTICS and M_3D_ROBOTICS_ZHANG_BASED camera calibration modes. It is defined using McalSetCoordinateSystem() with M_ASSIGN, or after a successful camera calibration with McalGrid() or McalList(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_COORDINATE_SYSTEM | Specifies the coordinate system used to change the position of the camera coordinate system. Although you can use this coordinate system to move the camera coordinate system, it need not be associated with the real camera position. By default, its axes are parallel to the absolute coordinate system, and its origin is the same as that of the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ReferenceCoordinateSystem

Specifies the coordinate system that will be used as a reference for calculations. All returned coordinates or angles will be calculated relative to this coordinate system. This parameter must be set to one of the following values.

> Note that if you specify a 3D reconstruction context, it must have already been calibrated using M3dmapCalibrate() or M3dmapCalibrateMultiple(). In this case, ReferenceCoordinateSystem can only be set to either M_ABSOLUTE_COORDINATE_SYSTEM or M_LASER_LINE_COORDINATE_SYSTEM.

> Note that when CalibratedAilObjectId is set to a 3D reconstruction result buffer, ReferenceCoordinateSystem can only be set to either M_ABSOLUTE_COORDINATE_SYSTEM or M_RELATIVE_COORDINATE_SYSTEM.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the reference coordinate system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_ABSOLUTE_COORDINATE_SYSTEM. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_ABSOLUTE_COORDINATE_SYSTEM | Specifies an implicit and unmovable coordinate system from which all other world coordinate systems are defined. By default, the origin lies at the center of the top-left calibration point when using McalGrid(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CAMERA_COORDINATE_SYSTEM | Specifies the coordinate system whose origin corresponds to the effective pinhole of the camera and whose Z-axis points in the direction that the camera is facing. This coordinate system can only be used as a reference coordinate system after it has been defined, either by a successful camera calibration in M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED mode. Alternatively, you can define this coordinate system using McalSetCoordinateSystem() with M_ASSIGN. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LASER_LINE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is on the first laser line at the point closest to the origin of the absolute coordinate system.<br>This value is only supported when CalibratedAilObjectId is set to a valid calibrated 3D reconstruction context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_COORDINATE_SYSTEM | Specifies the coordinate system used when working in world units, which can be recentered and/or re-oriented using McalSetCoordinateSystem() or McalRelativeOrigin(). By default, it is equivalent to the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROBOT_BASE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is positioned at the base of the robot holding the camera. This coordinate system is only supported for M_3D_ROBOTICS and M_3D_ROBOTICS_ZHANG_BASED camera calibration modes. It is defined using McalSetCoordinateSystem() with M_ASSIGN, or after a successful camera calibration with McalGrid() or McalList(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_COORDINATE_SYSTEM | Specifies the coordinate system used to change the position of the camera coordinate system. Although this is the coordinate system used to move the camera coordinate system, it need not be associated with the real camera position. By default, its axes are parallel to the absolute coordinate system, and its origin is the same as that of the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

InquireType

Specifies the type of transformation to use to return the location of the target coordinate system with respect to the reference coordinate system.

See the Parameter associations section for possible values that can be specified.

ArrayBufOrMatrix3dgeoId

Specifies the identifier of the destination Aurora Imaging Library array buffer or transformation matrix object in which to return the matrix representation of the transformation required to cause the reference coordinate system to align with the target coordinate system.

The Aurora Imaging Library array buffer must be a 32-bit float buffer, previously allocated using MbufAlloc2d() with M_ARRAY. The required buffer size will depend on the results being returned.

The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

If this parameter is not used, it must be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Param1Ptr

Specifies the address in which to return information about the transformation required to cause the reference coordinate system to overlap or align with the target coordinate system, depending on which is requested.

If this parameter is not used, it must be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Param2Ptr

Specifies the address in which to return information about the transformation required to cause the reference coordinate system to overlap or align with the target coordinate system, depending on which is requested.

If this parameter is not used, it must be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Param3Ptr

Specifies the address in which to return information about the transformation required to cause the reference coordinate system to overlap or align with the target coordinate system, depending on which is requested.

If this parameter is not used, it must be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Param4Ptr

Specifies the address in which to return information about the transformation required to cause the reference coordinate system to overlap or align with the target coordinate system, depending on which is requested.

If this parameter is not used, it must be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

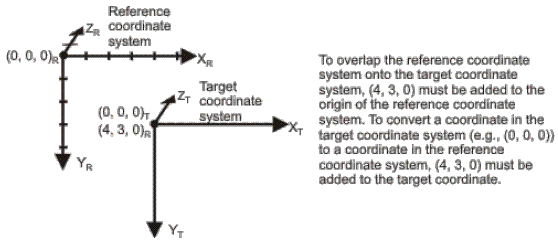
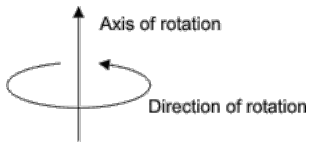
The table below lists possible values for the **InquireType** parameter and possible values returned to the **ArrayBufOrMatrix3dgeoId**, **Param1Ptr**, **Param2Ptr**, **Param3Ptr**, and **Param4Ptr** parameters.

- For inquiring about the transformation parameters

Set unused parameters to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For inquiring about the transformation parameters | | | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | | Description | | | | | | | | | | | | | | | |
| | ArrayBufOrMatrix3dgeoId<br>- Possible values returned | | | | | | | | | | | | | | | | |
| | Param1Ptr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| | Param2Ptr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| | Param3Ptr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| | Param4Ptr<br>- Possible values returned | | | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_HOMOGENEOUS_MATRIX | Retrieves the transformation which makes the axes of the reference coordinate system parallel to those of the target coordinate system, and makes the origin of the reference coordinate system coincide with that of the target coordinate system. The transformation is returned as a 4x4 homogenous matrix.<br><br>Note that this transformation is also the transformation required to express target coordinates with respect to the reference coordinate system. So you can use this result to convert a coordinate in the target coordinate system to the reference coordinate system.<br><br>  <br> | | | | | | | | | | | | | | |
| **ArrayBufOrMatrix3dgeoId** | Returns, into the specified Aurora Imaging Library array buffer or transformation matrix object, the matrix representation of the transformation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_AXIS_ANGLE | Retrieves the axis and angle of rotation which makes the axes of the reference coordinate system parallel to those of the target coordinate system. The axis of rotation is defined by a vector. The rotation angle is measured in the counter-clockwise direction around the axis of rotation, as per the right-hand rule.<br><br>  <br><br><br>The axis of rotation is always normalized.<br><br>When the axes of the coordinate systems are already parallel, Param1Ptr to Param4Ptr will be set to (0, 0, 0, 0). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the X-component of the vector which defines the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Y-component of the vector which defines the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Z-component of the vector which defines the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param4Ptr** | Returns the angle of the rotation around the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0 <= *Value* < 360 | Specifies the rotation angle, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_AXIS_X | Retrieves the normalized vector aligned with the X-axis of the reference coordinate system, expressed in the target coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the X-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Y-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Z-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_ROTATION_AXIS_Y | Retrieves the normalized vector aligned with the Y-axis of the reference coordinate system, expressed in the target coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the X-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Y-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Z-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_AXIS_Z | Retrieves the normalized vector aligned with the Z-axis of the reference coordinate system, expressed in the target coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the X-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Y-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Z-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_MATRIX | Retrieves the rotation matrix that makes the axes of the reference coordinate system parallel to those of the target coordinate system. The transformation is returned as a 3x3 matrix. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | Returns, into the specified Aurora Imaging Library array buffer or transformation matrix object, the matrix representation of the transformation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_QUATERNION | Retrieves the components of the quaternion that defines the rotation which makes the axes of the reference coordinate system parallel to those of the target coordinate system.<br><br>The quaternion is always normalized.<br><br>When the axes of the coordinate systems are already parallel, Param1Ptr to Param4Ptr will be set to (1, 0, 0, 0). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the scalar component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the X-component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Y-component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param4Ptr** | Returns the Z-component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_XYZ | Retrieves the angles of the three rotations that make the axes of the reference coordinate system parallel to those of the target coordinate system. The rotation is described by three distinct rotations about the axes of the reference coordinate system in the following order: the X-axis, Y-axis, and Z-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the X-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Y-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0.0 <= *Value* <= 90.0 | Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the positive X-axis. | c | e | f | g | a | i | j | k | l | m | n | b |
| 270.0 <= *Value* < 360.0 | Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the negative X-axis. | c | e | f | g | a | i | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Z-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the Z-axis rotation, in degrees. | c | e | f | g | a | i | j | k | l | m | n | b |
| M_ROTATION_XZY | Retrieves the angles of the three rotations that make the axes of the reference coordinate system parallel to those of the target coordinate system. The rotation is described by three distinct rotations about the axes of the reference coordinate system in the following order: the X-axis, Z-axis, and Y-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| **Param1Ptr** | Returns the X-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the X-axis rotation, in degrees. | c | e | f | g | a | i | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Z-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| 0.0 <= *Value* <= 90.0 | Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the positive Y-axis. | c | e | f | g | a | i | j | k | l | m | n | b |
| 270.0 <= *Value* < 360.0 | Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the negative Y-axis. | c | e | f | g | a | i | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Y-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the Y-axis rotation, in degrees. | c | e | f | g | a | i | j | k | l | m | n | b |
| M_ROTATION_YXZ | Retrieves the angles of the three rotations that make the axes of the reference coordinate system parallel to those of the target coordinate system. The rotation is described by three distinct rotations about the axes of the reference coordinate system in the following order: the Y-axis, X-axis, and Z-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| **Param1Ptr** | Returns the Y-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the Y-axis rotation, in degrees. | c | e | f | g | a | i | j | k | l | m | n | b |
| **Param2Ptr** | Returns the X-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |
| 0.0 <= *Value* <= 90.0 | Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the positive Z-axis. | c | e | f | g | a | i | j | k | l | m | n | b |
| 270.0 <= *Value* < 360.0 | Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the negative Z-axis. | c | e | f | g | a | i | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Z-axis rotation. | c | e | f | g | a | i | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0.0 <= *Value* < 360.0 | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_YZX | Retrieves the angles of the three rotations that make the axes of the reference coordinate system parallel to those of the target coordinate system. The rotation is described by three distinct rotations about the axes of the reference coordinate system in the following order: the Y-axis, Z-axis, and X-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the Y-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Z-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* <= 90.0 | Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the positive Y-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 270.0 <= *Value* < 360.0 | Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the negative Y-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the X-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_ZXY | Retrieves the angles of the three rotations that make the axes of the reference coordinate system parallel to those of the target coordinate system. The rotation is described by three distinct rotations about the axes of the reference coordinate system in the following order: the Z-axis, X-axis, and Y-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the Z-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the X-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* <= 90.0 | Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the positive Z-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 270.0 <= *Value* < 360.0 | Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the negative Z-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the Y-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_ZYX | Retrieves the angles of the three rotations that make the axes of the reference coordinate system parallel to those of the target coordinate system. The rotation is described by three distinct rotations about the axes of the reference coordinate system in the following order: Z-axis, Y-axis, and X-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the Z-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0.0 <= *Value* < 360.0 | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the Y-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* <= 90.0 | Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the positive X-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 270.0 <= *Value* < 360.0 | Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the negative X-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the X-axis rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 0.0 <= *Value* < 360.0 | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRANSLATION | Retrieves the components of the translation vector that makes the origin of the reference coordinate system coincide with the origin of the target coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1Ptr** | Returns the displacement along the X-axis of the reference coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2Ptr** | Returns the displacement along the Y-axis of the reference coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3Ptr** | Returns the displacement along the Z-axis of the reference coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalGrid

## Synopsis

Calibrate your camera setup using a grid.

## Syntax

```
void McalGrid(
    AIL_ID CalibrationId,     //out
    AIL_ID SrcImageBufId,     //in
    AIL_DOUBLE GridOffsetX,   //in
    AIL_DOUBLE GridOffsetY,   //in
    AIL_DOUBLE GridOffsetZ,   //in
    AIL_INT RowNumber,        //in
    AIL_INT ColumnNumber,     //in
    AIL_DOUBLE RowSpacing,    //in
    AIL_DOUBLE ColumnSpacing, //in
    AIL_INT64 Operation,      //in
    AIL_INT64 GridType        //in
)
```

## Description

This function uses an image of a grid of circles or squares and the world description of this grid to establish calibration points, calibrate your camera setup, and create an absolute world coordinate system. If using a circle grid, the center of each circle in the real-world grid is used as a calibration point; if using a chessboard grid, the intersection of four squares/rectangles is used as a calibration point. **McalGrid**() automatically determines the pixel coordinates of the calibration points in the image using feature extraction algorithms, and calculates the world coordinates of the calibration points using the world description of the grid (number of rows/columns and the space between rows/columns). From these calibration points and the specified calibration mode, **McalGrid**() establishes a pixel-to-world and world-to-pixel mapping function. The mapping is stored in the specified calibration context.

**McalGrid**() can also extract calibration points from a special type of chessboard grid which contains a fiducial; this is referred to as a fiducial grid. The fiducial supported in a fiducial grid is a Data Matrix code that encodes the world description of the grid itself. This information includes the grid spacing, the units in which the spacing is measured, the position of the grid's reference calibration point, and the grid's orientation.

Aurora Imaging Library is distributed with five different camera calibration grids that you can print and use: *ChessboardCalibrationGrid_15x16_Letter.pdf*, *ChessboardCalibrationGrid_18x22_Letter.pdf*, *CircleCalibrationGrid_15x16_Letter.pdf*, *CircleCalibrationGrid_18x22_Letter.pdf*, and *FiducialCalibrationGrid_Letter.pdf*. These files are located in the installed Images directory (for example, *C:\Program Files\Aurora Imaging Library\11\Images*). Each file includes the world description of the grid. For guidelines when printing the grid, see the Printing the grid subsection of the Generating a grid for calibration section of Chapter 28: Calibrating your camera setup.

After calling **McalGrid**(), you can inquire about the success of the camera calibration using McalInquire() with M_CALIBRATION_STATUS. You can inquire about the accuracy of the mapping function (camera calibration error) using McalInquire() with M_AVERAGE_PIXEL_ERROR, M_MAXIMUM_PIXEL_ERROR, M_AVERAGE_WORLD_ERROR, or M_MAXIMUM_WORLD_ERROR. In addition, you can visualize the calibration points that were used, using McalDraw() with M_DRAW_IMAGE_POINTS; this visualization can be especially effective when overlaid on the image of the original grid. It is important that the extracted calibration points are directly in the middle of each circle, for a circle grid, and at the intersection of the grid squares/rectangles, for a chessboard grid. Any extra points, missing points, or imprecisely positioned points will result in calibration inaccuracy. For more information, see the Inquiring about your camera calibration section of Chapter 28: Calibrating your camera setup.

In the default calibration mode, M_LINEAR_INTERPOLATION mode, a mapping function is created so that comparing transformed coordinates (either pixel-to-world or world-to-pixel) of the calibration points results in virtually no measured calibration error. This produces increased calibration accuracy for areas of the image inside the grid and reduced accuracy for areas outside the grid. For all other calibration modes, the mapping function applies the calibration error equally to all areas in the image, regardless of where the grid was in the image. If there are any problems with the mapping function, the calibration error is averaged out over the entire image.

For 3D-based camera calibration modes (M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED), it is possible to use a grid that is not in the XY-plane (Z=0) of the absolute coordinate system.

This allows you to account for two XY-planes in the working area of the camera or even for the thickness of the calibration grid itself. For both cases, you can use the GridOffsetZ parameter to specify the height or thickness of the grid being used.

When working in M_3D_ROBOTICS camera calibration mode, you must first call this function with M_ACCUMULATE at least three times to accumulate data with different images and orientations. Calling this function with M_ACCUMULATE more than 3 times greatly improves the accuracy of the camera calibration. Before each data accumulation call, you must change the position and orientation of the tool holding the camera with respect to the robot base. More specifically, you must rotate the tool along at least two non-parallel axes, retrieve the tool's new position with respect to the robot base from the robot encoder, and using this information, set the position of the tool coordinate system with respect to the robot base coordinate system using McalSetCoordinateSystem(). When you are done accumulating data, you must call this function with M_FULL_CALIBRATION and no image to perform the full camera calibration. Once you perform a full camera calibration, you can no longer accumulate camera calibration poses.

When working in M_ZHANG_BASED camera calibration mode, you must first call this function with M_ACCUMULATE at least three times to accumulate data with different images and orientations. Calling this function with M_ACCUMULATE more than three times greatly improves the accuracy of the camera calibration. Before each call, move the grid or camera such that the grid is not viewed at the same plane by the camera as a previous call of this function with M_ACCUMULATE. The pose provided in the last call will be used to automatically set the position of the absolute and camera coordinate systems. It is recommended to accumulate calibration data at various perceived depths, and at the edges of the working area of your application. When you are done accumulating data, you must call this function with M_FULL_CALIBRATION and no image to perform the full camera calibration. Once you perform a full camera calibration, you can no longer accumulate camera calibration poses.

When working in M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, you must first call this function with M_ACCUMULATE at least three times to accumulate data with different images and orientations. Calling this function with M_ACCUMULATE more than three times greatly improves the accuracy of the camera calibration. Before each call, you must follow the steps described in the previous two paragraphs. When you are done accumulating data, you must call this function with M_FULL_CALIBRATION and no image to perform the full camera calibration. Once you perform a full camera calibration, you can no longer accumulate camera calibration poses.

For 3D-based camera calibration modes, **McalGrid**() performs two types of calculations. It calculates the camera's internal attributes, and it calculates the orientation and distance between the camera and camera calibration plane. Initially, the latter is used to set the camera coordinate system. If you move the camera or the grid (not both), you can have **McalGrid**() recalculate only the orientation and distance between them. In this case, depending on what you moved, you can specify that **McalGrid**() displace either the camera coordinate system or the relative coordinate system, using M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD, respectively. Note that M_DISPLACE_CAMERA_COORD is not supported for an M_STATIONARY_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode. When using M_DISPLACE_RELATIVE_COORD, the camera calibration plane is considered to be the XY-plane of the relative coordinate system, regardless of how McalControl() with M_CALIBRATION_PLANE is set.

> Note that the internal attributes calculated for the camera are not those of the physical camera, but those of the ideal pinhole-camera used to model the physical camera.

Also, for most 3D-based camera calibration modes, a successful call to **McalGrid**() with M_FULL_CALIBRATION creates a rigid link between the tool coordinate system and the head (camera or reference/calibration object) coordinate system. This link ensures that moving either the tool or the head coordinate system will affect both. This link can be broken using McalControl() with M_LINK_TOOL_AND_HEAD.

Note that for an M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED calibration context, which requires multiple calls to **McalGrid**(), you can inquire about any particular call using McalInquireSingle().

Depending on the type of calibration grid and the orientation of the camera and grid in your setup, you might have to specify a hint pixel to indicate the grid's reference calibration point, used to determine the origin of the absolute coordinate system. You can do this using McalControl() set to M_GRID_HINT_PIXEL_X and M_GRID_HINT_PIXEL_Y. Note that the default position of the reference calibration point for complete grids (grids that are unobstructed and show all specified rows and columns) is the calibration point in the corner of the grid closest to the top-left corner of the image. For chessboard grids that are partially obscured and for all fiducial grids, the default reference calibration point is the calibration point closest to the center of the image.

You can limit results to a region of interest (ROI) of an image buffer using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

For more information on performing camera calibration using a chessboard or circle grid, see the Calibrating using calibration points from a grid section of Chapter 28: Calibrating your camera setup. For information on camera calibration using a fiducial grid, see the Calibrating using a fiducial grid section of Chapter 28: Calibrating your camera setup.

## Parameters

CalibrationId

Specifies the identifier of the camera calibration context.

SrcImageBufId

Specifies the identifier of the image buffer containing the grid. It must be an 8- or 16-bit unsigned buffer, with 1 or 3 bands.

This image buffer can have a region of interest (ROI) associated with it (set using MbufSetRegion()). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Note that the source image used for camera calibration is automatically calibrated after a successful call to **McalGrid**() with M_FULL_CALIBRATION.

> When performing an M_FULL_CALIBRATION operation in an M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

GridOffsetX INQ

Specifies the X-coordinate of the grid's reference calibration point, in the real world. Specify its X-coordinate in the world coordinate system specified using McalControl() with M_CALIBRATION_PLANE. Typically, set this value to 0.

> When performing an M_FULL_CALIBRATION operation in an M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

GridOffsetY INQ

Specifies the Y-coordinate of the grid's reference calibration point, in the real world. Specify its Y-coordinate in the world coordinate system specified using McalControl() with M_CALIBRATION_PLANE. Typically, set this value to 0.

> When performing an M_FULL_CALIBRATION operation in an M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

GridOffsetZ INQ

Specifies the Z-coordinate of the grid's reference calibration point, in the real world. Specify its Z-coordinate in the world coordinate system specified using McalControl() with M_CALIBRATION_PLANE. Typically, set this value to 0.

> When performing an M_FULL_CALIBRATION operation in an M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

RowNumber INQ

Specifies the number of rows in the camera calibration grid.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the row number | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NULL | Specifies that you are performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNKNOWN | Specifies that you are calibrating using a partial chessboard grid.<br><br>This can be set if and only if McalControl() with M_GRID_PARTIAL is set to M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Value* >= 2 | Specifies the total number of rows in the real-world grid. Generally, camera calibration accuracy increases with the number of rows.<br><br>The minimum number of rows is 2 when GridType is set to M_CIRCLE_GRID and 3 when set to M_CHESSBOARD_GRID.<br><br>For M_TSAI_BASED and M_3D_ROBOTICS camera calibration modes, you must provide a grid that contains at least 6 grid points, even though the minimum number of rows and the minimum number of columns are both 2.<br><br>For M_ZHANG_BASED and M_3D_ROBOTICS_ZHANG_BASED camera calibration modes, you must provide a grid that contains at least 8 grid points, even though the minimum number of rows and the minimum number of columns are both 2.<br><br>> Note that in the case of a rotated image of a rectangular camera calibration grid, you must specify the number of rows of the real-world grid and not the number of rows as there appears to be in the rotated image of the grid. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## ColumnNumber INQ

Specifies the number of columns in the camera calibration grid.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the column number | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NULL | Specifies that you are performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNKNOWN | Specifies that you are calibrating using a partial chessboard grid.<br><br>This can be set if and only if McalControl() with M_GRID_PARTIAL is set to M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 2 | Specifies the total number of columns in the real-world grid. Generally, camera calibration accuracy increases with the number of columns.<br><br>The minimum number of columns is 2 when GridType is set to M_CIRCLE_GRID and 3 when set to M_CHESSBOARD_GRID.<br><br>For M_TSAI_BASED and M_3D_ROBOTICS camera calibration modes, you must provide a grid that contains at least 6 grid points, even though the minimum number of rows and the minimum number of columns are both 2.<br><br>For M_ZHANG_BASED and M_3D_ROBOTICS_ZHANG_BASED camera calibration modes, you must provide a grid that contains at least 8 grid points, even though the minimum number of rows and the minimum number of columns are both 2.<br><br>> Note that in the case of a rotated image of a rectangular camera calibration grid, you must specify the number of columns of the real-world grid and not the number of rows as there appears to be in the rotated image of the grid. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## RowSpacing INQ

Specifies the number of world units between rows.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the row spacing | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NULL | Specifies to not use row spacing.<br><br>You must set this value when performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_FROM_FIDUCIAL | Specifies to set the row spacing according to the data in the chessboard grid fiducial. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the row spacing, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

### ColumnSpacing INQ

Specifies the number of world units between columns.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the column spacing | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NULL | Specifies to not use column spacing.<br><br>You must set this value when performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FROM_FIDUCIAL | Specifies to set the column spacing according to the data in the chessboard grid fiducial. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the column spacing, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

### Operation

Specifies the camera calibration operation to perform. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the camera calibration operation | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_FULL_CALIBRATION. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ACCUMULATE | Calculates only the position of the calibration points and stores them in the camera calibration context.<br><br>For M_3D_ROBOTICS, M_ZHANG_BASED, and M_3D_ROBOTICS_ZHANG_BASED, at least three calls to **McalGrid**() should be made with this operation before performing the full camera calibration with M_FULL_CALIBRATION. Providing more than three calls can greatly improve the accuracy of the camera calibration.<br><br>> This operation is only supported for M_3D_ROBOTICS, M_ZHANG_BASED, and M_3D_ROBOTICS_ZHANG_BASED camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

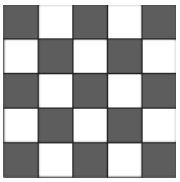
| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DISPLACE_CAMERA_COORD | Calculates only the position and orientation between the camera and the camera calibration plane, and displaces the camera coordinate system accordingly. Note that besides the camera coordinate system, this also displaces the tool coordinate system (if still linked with M_LINK_TOOL_AND_HEAD); no other coordinate system is affected. This camera calibration operation is only supported for 3D-based camera calibration contexts that are fully calibrated with M_FULL_CALIBRATION.<br><br>If the camera is moved to a new position but its internal attributes are already known by a previous full camera calibration, you can use this operation to allow for a faster camera calibration.<br><br>> Note that M_DISPLACE_CAMERA_COORD is not supported for an M_STATIONARY_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISPLACE_RELATIVE_COORD | Calculates only the position and orientation between the camera and the camera calibration plane, and displaces the relative coordinate system accordingly. This camera calibration operation is only supported for 3D-based camera calibration contexts that are fully calibrated with M_FULL_CALIBRATION.<br><br>You can use this operation to move the relative coordinate system only and remain calibrated. This operation keeps the camera fixed with respect to the absolute coordinate system, and its intrinsic attributes unmodified. You can then obtain the position and orientation of the relative coordinate system with respect to any other coordinate system using McalGetCoordinateSystem(), and calculate the unknown position of an object in space.<br><br>When using this operation, the camera calibration plane is considered to be the relative coordinate system regardless of the M_CALIBRATION_PLANE setting. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FULL_CALIBRATION | Performs a full camera calibration. For 3D-based camera calibration contexts, this operation calculates the camera's internal attributes and the difference in position and orientation between the camera and the camera calibration plane. It then sets the camera coordinate system accordingly.<br><br>> When working in M_TSAI_BASED camera calibration mode, the camera's optical axis should be at least 30 degrees away from the axis perpendicular to the camera calibration plane (also known as, angle of incidence). Otherwise, the camera calibration might fail. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

GridType INQ

Specifies the type of grid used. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of grid | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_CIRCLE_GRID. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_CHESSBOARD_GRID + | Specifies a chessboard grid. The number of rows and columns represents the number of corners horizontally and vertically. A corner is the intersection of 4 cells, meaning that the corners on the perimeter do not count. Note that, for a chessboard grid type, the number of rows and the number of columns must be at least 3. The following is an example of a valid 4x4 chessboard grid.<br>  <br>> Note that when calibrating using a chessboard grid, the entire grid does not have to be present in the camera calibration image. In this case, you must call McalControl() with M_GRID_PARTIAL set to M_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CIRCLE_GRID + | Specifies a grid of circles. Note that, for a grid of circles, the number of rows and the numbers of columns must be at least 2. To create an accurate (subpixel) mapping, the image of the grid should meet the following guidelines (at the working resolution):<br>• The radius of the grid's circles should range between 6 and 10 pixels.<br>• The center-to-center distance between the grid's circles should range from 18 to 32 pixels (22 pixels recommended).<br>• The minimum distance between the edges of the circles should be 6 pixels.<br>• The grid should be large enough to cover the area of the image from which you want real-world results.<br>• The grid must be on a single plane although it can be at a slant.<br>• The image of the grid should have high contrast.<br>> Note that circle grids must be complete grids in which all circles are present in a rectangular grid. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for M_CHESSBOARD_GRID; M_CIRCLE_GRID.

You can add one of the following values to the above-mentioned values to set the orientation of the Y-axis.

These combination values are not supported if using M_UNIFORM_TRANSFORMATION camera calibration mode.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the orientation of the Y-axis INQ | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_Y_AXIS_CLOCKWISE | Orients the positive Y-axis 90° clockwise from the X-axis.<br>Typically, for a complete grid that has its reference calibration point at the top-left of the grid, the orientation of the Y-axis is along the first column of circles in a grid of circles or along the leftmost vertical line of intersection points in a chessboard grid. The axis is oriented from the top to the bottom.<br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_Y_AXIS_COUNTER_CLOCKWISE | Orients the positive Y-axis 90° counter-clockwise from the X-axis.<br><br>Typically, for a complete grid that has its reference calibration point at the top-left of the grid, the orientation of the Y-axis is along the first column of circles in a grid of circles, or along the leftmost vertical line of intersection points in a chessboard grid. The axis is oriented from the bottom to the top. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_CIRCLE_GRID.

You can add the following value to the above-mentioned value to set the speed of camera calibration.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For increasing the speed of camera calibration | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_FAST | Speeds up the time it takes to perform the camera calibration by using a faster camera calibration algorithm. Note that using a faster camera calibration algorithm can decrease the robustness and accuracy of the camera calibration.<br><br>When using M_FAST, memory consumption is reduced. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalInquire

## Synopsis

Inquire about a camera calibration context, a calibrated image, a calibrated result buffer, a fixturing offset object, a 3D draw calibration context, or about the camera calibration state of an image, digitizer, or result buffer.

## Syntax

```
AIL_INT McalInquire(
    AIL_ID ContextCalOrCalibratedAilObjectId, //in
    AIL_INT64 InquireType,                    //in
    void *UserVarPtr                          //out
)
```

## Description

This function inquires about a setting of a camera calibration context, a calibrated image, a calibrated result buffer, a fixturing offset object, or a 3D draw calibration context. It can also be used to determine if a camera calibration context is associated with an image, digitizer, or result buffer, and whether or not an image has been corrected.

When working in M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration mode, the function returns information about the last camera calibration performed on that object. To inquire about previous camera calibration poses within the M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context, you can use McalInquireSingle() with the Index parameter set to the required camera calibration pose.

To inquire the default value of a setting, add M_DEFAULT to the InquireType parameter.

## Parameters

ContextCalOrCalibratedAilObjectId

Specifies the identifier of the camera calibration context, image buffer, result buffer, digitizer, fixturing offset object, or 3D draw calibration context.

InquireType

Specifies the setting about which to inquire. The setting for InquireType depends on whether you are inquiring about a camera calibration context, image, result buffer, or digitizer.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT> ]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID> ]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT> ]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16> ]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32> ]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64> ]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address in which to return the value of the inquired setting. Since the **McalInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For a 3D robotics camera calibration context
- For a camera calibration context
- For a camera calibration context calibrated using McalGrid()
- For a camera calibration context calibrated using McalGrid() or McalList()
- For a camera calibration context calibrated using McalGrid(), McalList(), or McalUniform()
- For a 3D-based camera calibration context
- For any image, digitizer, or result buffer
- For a camera calibration context, image, or result buffer
- For a calibrated image or result buffer
- For a calibrated image with distortions
- For a calibrated image or result buffer, with a constant pixel size
- For a fixturing offset object
- For a 3D draw calibration context

For a 3D robotics camera calibration context, InquireType can be set to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a 3D robotics camera calibration context | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_3D_ROBOTICS_CAMERA_MODEL | Inquires the camera calibration model used by the 3D robotics camera calibration context. To use the Zhang-based camera model in 3D robotics mode use M_3D_ROBOTICS_ZHANG_BASED. To use the Tsai-based camera model in 3D robotics mode, use M_3D_ROBOTICS. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TSAI_BASED | Specifies that Tsai-based camera model is used for the 3D robotics camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ZHANG_BASED | Specifies that Zhang-based camera model is used for the 3D robotics camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_3D_ROBOTICS_MODE | Inquires the camera calibration mode used by the 3D robotics camera calibration context. To use a moving camera setup in 3D robotics mode, use M_MOVING_CAMERA. To use a stationary camera setup in 3D robotics mode, use M_STATIONARY_CAMERA. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MOVING_CAMERA | Specifies that a moving camera setup is used for the 3D robotics camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_STATIONARY_CAMERA | Specifies that a stationary camera setup is used for the 3D robotics camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a camera calibration context, InquireType can be set to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | |
| M_CALIBRATION_INPUT_DATA + | Specifies the type of data that was used to perform the camera calibration. | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NULL | Specifies that no camera calibration was performed on the camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID | Specifies that the camera calibration was performed using a camera calibration grid (McalGrid()). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LIST | Specifies that the camera calibration was performed by explicitly specifying the correspondence between some pixels and their real-world coordinates (McalList()). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PARAMETRIC | Specifies that the camera calibration was performed using an explicitly specified translation, scale, and offset from the absolute world coordinate system (McalUniform()). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_PLANE + | Inquires the plane in which the calibration points are defined. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ABSOLUTE_COORDINATE_SYSTEM | Specifies that the calibration points are defined in the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_COORDINATE_SYSTEM | Specifies that the calibration points are defined in the relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_STATUS + | Inquires the status of a camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATED | Specifies that the camera calibration was successful. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATING | Specifies that the last call to McalGrid() / McalList() was made with M_ACCUMULATE and the camera calibration was successful. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_NOT_FOUND | Specifies that McalGrid() was unable to find an appropriate camera calibration grid in the provided image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INVALID_CALIBRATION_POINTS | Specifies that the provided calibration points do not contain sufficient spatial information to perform a camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MATHEMATICAL_EXCEPTION | Specifies that the calculation of the camera's parameters has failed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_NOT_INITIALIZED | Specifies that no camera calibration has been performed yet. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_PLANE_ANGLE_TOO_SMALL | Specifies that the camera's optical axis is not sufficiently inclined. For full M_TSAI_BASED camera calibration, the camera's optical axis should be placed at least at a 30-degrees angle away from the axis perpendicular to the camera calibration plane. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_TOO_MANY_OUTLIERS | Specifies that the calculation performed by McalGrid() or McalList() with M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD has failed. There were too many possible outliers to correctly compute the new location of the coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CALIBRATION_ERROR_SCALE_FACTOR + | | Inquires the scale factor used when drawing arrows, representing the camera calibration error, using McalDraw() with M_DRAW_CALIBRATION_ERROR. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* > 0 | Specifies the scale factor. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CALIBRATION_ERROR_SCALE_MODE + | | Inquires the scale mode used when drawing arrows, representing the camera calibration error, using McalDraw() with M_DRAW_CALIBRATION_ERROR. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ABSOLUTE | Specifies to use M_DRAW_CALIBRATION_ERROR_SCALE_FACTOR as a scaling factor that directly multiplies the arrows' horizontal and vertical displacements. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_AUTO | Specifies that Aurora Imaging Library determines the scale mode. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LINK_TOOL_AND_HEAD + | | Inquires whether a rigid link exists between the head (camera or reference/calibration object) coordinate system and the tool coordinate system.<br><br>Note that for M_LINK_TOOL_AND_HEAD, the head represents a camera or a reference/calibration object depending on the type of robotic setup. When the robotic setup is M_MOVING_CAMERA, the head refers to a camera attached to the last joint of the robotic arm. When the robotic setup is M_STATIONARY_CAMERA, the head refers to a reference/calibration object attached to the last joint of the robotic arm. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies to remove the link between the two coordinate systems, allowing both to be moved independently. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to link the two coordinate systems, allowing both to be moved together. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_LOCALIZATION_NB_ITERATIONS_MAX + | Inquires the maximum number of iterations to attempt to fit the provided points when calculating the new position of the camera or relative coordinate system, when using McalList() and McalGrid() with M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD respectively. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LOCALIZATION_NB_OUTLIERS_MAX + | Inquires the maximum number of possible outliers that can occur in the dataset used by McalList() or McalGrid() with M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_OWNER_SYSTEM + | Inquires the identifier of the system on which the camera calibration context has been allocated. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT_HOST | Specifies the default Host system of the current Aurora Imaging Library application. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *System identifier* | Specifies a valid system identifier, previously allocated using MsysAlloc(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_ORIGIN_ANGLE + | Inquires the angle of rotation of the relative world coordinate system.<br><br>For 2D-based camera calibration contexts, the angle of rotation is of the relative coordinate system about the Z-axis of the absolute coordinate system. A positive angle corresponds to the rotation of the X-axis in the direction of the negative Y-axis.<br><br>For 3D-based camera calibration contexts (M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED), the axis of rotation is not necessarily the Z-axis. The returned value is equivalent to Param4Ptr of M_ROTATION_AXIS_ANGLE in McalGetCoordinateSystem().<br><br>The angle of rotation of the relative coordinate system is affected by transformations performed using McalSetCoordinateSystem() or McalRelativeOrigin() with AngularOffset. If you are using a 3D-based camera calibration context, it is recommended to use McalGetCoordinateSystem(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the angle of rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_RELATIVE_ORIGIN_X + | Inquires the X-coordinate of the origin of the relative world coordinate system.<br><br>The X-coordinate of the origin of the relative coordinate system is affected by transformations performed using McalSetCoordinateSystem() or McalRelativeOrigin() with XOffset. If you are using a 3D-based camera calibration context, it is recommended to use McalSetCoordinateSystem(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_ORIGIN_Y + | Inquires the Y-coordinate of the origin of the relative world coordinate system.<br><br>The Y-coordinate of the origin of the relative coordinate system is affected by transformations performed using McalSetCoordinateSystem() or McalRelativeOrigin() with YOffset. If you are using a 3D-based camera calibration context, it is recommended to use McalSetCoordinateSystem(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_ORIGIN_Z + | Inquires the Z-coordinate of the origin of the relative world coordinate system.<br><br>The Z-coordinate of the origin of the relative coordinate system is affected by transformations performed using McalSetCoordinateSystem() or McalRelativeOrigin() with ZOffset. If you are using a 3D-based camera calibration context, it is recommended to use McalSetCoordinateSystem(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Z-coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_POSITION_X + | Inquires the X-position of the origin of the tool coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_POSITION_Y + | Inquires the Y-position of the origin of the tool coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_POSITION_Z + | Inquires the Z-position of the origin of the tool coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Value* | Specifies the Z-coordinate, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_ROTATION_Z + | Inquires the rotation angle of the tool coordinate system about its Z-axis.<br>A positive angle corresponds to the rotation of the tool coordinate system in the counter-clockwise direction (from the positive X-axis to the negative Y-axis of the tool coordinate system). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the angle, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRANSFORM_CACHE + | Inquires whether a cache is used to accelerate McalTransformImage(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies not to use a cache. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies to use a cache. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a camera calibration context that has been calibrated using McalGrid(), the InquireType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context calibrated using McalGrid() | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType<br>UserVarPtr<br>- Possible values returned | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_COLUMN_NUMBER + | Inquires the number of columns in the camera calibration grid. SET | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 2 | Specifies the number of columns. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_COLUMN_SPACING + | Inquires the number of world units between columns. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the spacing between columns. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FOREGROUND_VALUE + | Inquires whether the grid's circles are lighter or darker than the background. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Determines the appropriate setting automatically. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FOREGROUND_BLACK | Specifies that the grid's circles are darker than the background. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FOREGROUND_WHITE | Specifies that the grid's circles are lighter than the background. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_GRID_FIDUCIAL + | Inquires whether the partial chessboard grid has a fiducial. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DATAMATRIX | Specifies that a Data Matrix code is used as a fiducial in a chessboard grid. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies that there is no fiducial in the grid. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_HINT_ANGLE_X + | Inquires the hint angle used to help determine the orientation of the X-axis when calibrating your camera with a partial chessboard grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies that no hint angle is used. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the hint angle, measured counter-clockwise. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_HINT_PIXEL_X + | Inquires the X-coordinate of the hint pixel used to help determine the grid's reference calibration point when calibrating your camera with a grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies not to use a hint pixel. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate of the hint pixel, in the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_HINT_PIXEL_Y + | Inquires the Y-coordinate of the hint pixel used to help determine the grid's reference calibration point when calibrating your camera with a grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NONE | Specifies not to use a hint pixel. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate of the hint pixel, in the pixel coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_ORIGIN_X + | Inquires the X-coordinate of the grid's reference calibration point, in the world coordinate system. The world coordinate system could be either the absolute coordinate system or the relative coordinate system, depending on which coordinate system was specified when you called McalGrid(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_ORIGIN_Y + | Inquires the Y-coordinate of the grid's reference calibration point, in the world coordinate system. The world coordinate system could be either the absolute coordinate system or the relative coordinate system, depending on which coordinate system was specified when you called McalGrid(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Value* | Specifies the Y-coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_ORIGIN_Z + | Inquires the Z-coordinate of the grid's reference calibration point, in the world coordinate system. The world coordinate system could be either the absolute coordinate system or the relative coordinate system, depending on which coordinate system was specified when you called McalGrid(). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Z-coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_PARTIAL + | Inquires whether the chessboard grid in the camera calibration image is allowed to be a partial grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies that McalGrid() will only calibrate the camera setup when a complete grid is found in the image. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies that McalGrid() can calibrate the camera setup when a partial grid is found in the image. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_SHAPE + | Inquires whether the partial chessboard grid in the camera calibration image is assumed to be a rectangle. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ANY | Specifies to include all potential calibration points in the image; McalGrid() will not look for the boundary of the real-world grid. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RECTANGLE | Specifies to exclude potential calibration points in the image that McalGrid() determines are outside the boundaries of the partial grid. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_TYPE + | Inquires the type of grid used to perform the camera calibration. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CHESSBOARD_GRID | Specifies a chessboard grid. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CIRCLE_GRID | Specifies a grid of circles. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_UNIT_SHORT_NAME + | Inquires the abbreviated name of the world units encoded in the Data Matrix code of the fiducial grid used to calibrate the camera calibration context.<br><br>Note that this information is not used by the Calibration module. No unit consistency is enforced and no unit conversion is performed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*cm*") 1 | Specifies the grid units are centimeters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*ft*") 1 | Specifies the grid units are feet. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| AIL_TEXT("*in*") 1 | Specifies the grid units are inches. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*km*") 1 | Specifies the grid units are kilometers. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*m*") 1 | Specifies the grid units are meters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*miles*") 1 | Specifies the grid units are miles. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*mils*") 1 | Specifies the grid units are mils. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*mm*") 1 | Specifies the grid units are millimeters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*um*") 1 | Specifies the grid units are micrometers.<br>> Note that micrometers are abbreviated as "um" to ensure easy printing in a non-unicode environment. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*units*") 1 | Specifies the grid units are unknown. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_UNITS + | Inquires the world units that were encoded in the Data Matrix code of the fiducial grid used to calibrate the camera calibration context.<br>Note that this information is not used by the Calibration module. No unit consistency is enforced and no unit conversion is performed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CENTIMETERS | Specifies that the grid units are measured in centimeters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FEET | Specifies that the grid units are measured in feet. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INCHES | Specifies that the grid units are measured in inches. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_KILOMETERS | Specifies that the grid units are measured in kilometers. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_METERS | Specifies that the grid units are measured in meters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MICROMETERS | Specifies that the grid units are measured in micrometers. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MILES | Specifies that the grid units are measured in miles. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MILLIMETERS | Specifies that the grid units are measured in millimeters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MILS | Specifies that the grid units are measured in mils. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNKNOWN | Specifies that grid units are measured in an unknown unit.<br>If your calibration grid does not have a fiducial, the grid units are unknown. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROW_NUMBER + | Inquires the number of rows in the camera calibration grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Value* >= 2 | Specifies the number of rows. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROW_SPACING + | Inquires the spacing between rows in the camera calibration grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the spacing between rows, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

For a camera calibration context that has been calibrated using McalGrid() or McalList(), the InquireType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context calibrated using McalGrid() or McalList() | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType<br>UserVarPtr<br>- Possible values returned | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_AVERAGE_PIXEL_ERROR + | Inquires the average camera calibration error in the pixel coordinate system. You can only inquire this value for a successfully calibrated camera calibration context.<br>This is the average distance in the pixel coordinate system between the initial calibration points and their projected points in an image.<br>When working in M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration mode, this only returns the average camera calibration error of the last camera calibration call. To return an average camera calibration error for all camera calibration grids used, it is recommended to use M_GLOBAL_AVERAGE_PIXEL_ERROR instead. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the average camera calibration error, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_AVERAGE_WORLD_ERROR + | Inquires the average camera calibration error in the absolute coordinate system. You can only inquire this value for a successfully calibrated camera calibration context.<br>This is the average distance in the absolute coordinate system between the initial calibration points and their projected points in an image.<br>When working in M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration mode, this only returns the average camera calibration error of the last camera calibration call. To return an average camera calibration error for all camera calibration grids used, it is recommended to use M_GLOBAL_AVERAGE_WORLD_ERROR instead. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the average camera calibration error, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_CALIBRATION_IMAGE_POINTS_X + | Inquires the X-pixel coordinate of the calibration points.<br><br>If you used McalGrid() to calibrate your camera setup, the calibration points' pixel coordinates are determined by the pixel positions of the centers of the circles in a circle grid or the intersections of four squares/rectangles in a chessboard grid.<br><br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' pixel coordinates with the PixCoordXArrayPtr parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_IMAGE_POINTS_Y + | Inquires the Y-pixel coordinate of the calibration points.<br><br>If you used McalGrid() to calibrate your camera setup, the calibration points' pixel coordinates are determined by the pixel positions of the centers of the circles in a circle grid or the intersections of four squares/rectangles in a chessboard grid.<br><br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' pixel coordinates with the PixCoordYArrayPtr parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_WORLD_POINTS_X + | Inquires the X-world coordinate of the calibration points. The coordinate is expressed in world units of the camera calibration plane (M_CALIBRATION_PLANE).<br><br>If you used McalGrid() to calibrate your camera setup, the calibration points are computed from the parameters GridOffsetX, ColumnNumber, and ColumnSpacing.<br><br>If you used McalList() to calibrate your camera setup, the world coordinates of the calibration points are those you set with the WorldCoordXArrayPtr parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, in real-world units of the camera calibration plane (M_CALIBRATION_PLANE). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_WORLD_POINTS_Y + | Inquires the Y-world coordinate of the calibration points. The coordinate is expressed in world units of the camera calibration plane (M_CALIBRATION_PLANE).<br><br>If you used McalGrid() to calibrate your camera setup, the calibration points are computed from the parameters GridOffsetY, RowNumber, and RowSpacing.<br><br>If you used McalList() to calibrate your camera setup, the world coordinates of the calibration points are those you set with the WorldCoordYArrayPtr parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Value* | Specifies the Y-coordinate, in real-world units of the camera calibration plane (M_CALIBRATION_PLANE). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_WORLD_POINTS_Z + | Inquires the Z-world coordinate of the calibration points that are based on explicitly specified values. The computed calibration points are expressed in world units of the camera calibration plane (M_CALIBRATION_PLANE). If this value is inquired for 2D-based camera calibration contexts, the specified array will be filled with 0.0 values.<br><br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' pixel coordinates with the WorldCoordZArrayPtr parameter.<br><br>If you used McalGrid() to calibrate your camera setup, you explicitly set the Z-coordinate of all the calibration points with the GridOffsetZ parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Z-coordinate, in real-world units of the camera calibration plane (M_CALIBRATION_PLANE). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GLOBAL_AVERAGE_PIXEL_ERROR + | Inquires the average camera calibration error, in pixels, for all the points used in all successive calls to McalGrid() and McalList(). You can only inquire this value for a successfully calibrated M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the average camera calibration error, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GLOBAL_AVERAGE_WORLD_ERROR + | Inquires the average camera calibration error, in world units, for all the points used in all successive calls to McalGrid() and McalList(). You can only inquire this value for a successfully calibrated M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the average camera calibration error, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GLOBAL_MAXIMUM_PIXEL_ERROR + | Inquires the maximum camera calibration error, in pixels, for all the points used in all successive calls to McalGrid() and McalList(). You can only inquire this value for a successfully calibrated M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the maximum camera calibration error, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GLOBAL_MAXIMUM_WORLD_ERROR + | Inquires the maximum camera calibration error, in world units, for all the points used in all successive calls to McalGrid() and McalList(). You can only inquire this value for a successfully calibrated M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the maximum camera calibration error, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MAXIMUM_PIXEL_ERROR + | Inquires the maximum camera calibration error, in pixels. You can only inquire this value for a successfully calibrated camera calibration context.<br><br>For an M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context, this only returns the average camera calibration error of the last camera calibration call. To return an average camera calibration error for all camera calibration grids used, it is recommended to use M_GLOBAL_MAXIMUM_PIXEL_ERROR instead. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the maximum camera calibration error, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MAXIMUM_WORLD_ERROR + | Inquires the maximum camera calibration error, in world units. You can only inquire this value for a successfully calibrated camera calibration context.<br><br>For an M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context, this only returns the average camera calibration error of the last camera calibration call. To return an average camera calibration error for all camera calibration grids used, it is recommended to use M_GLOBAL_MAXIMUM_WORLD_ERROR instead. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the maximum camera calibration error, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NUMBER_OF_CALIBRATION_POINTS + | Inquires the number of calibration points found by McalGrid() or passed to McalList().<br><br>If you used McalList() to calibrate your camera setup, you can explicitly set the number of calibration points with the NumPoint parameter.<br><br>If you used McalGrid() to calibrate your camera setup, the number of calibration points is determined by the number of columns and rows in your grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0 | Specifies the number of calibration points. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NUMBER_OF_CALIBRATION_POSES + | Inquires the number of calls made to McalGrid() or McalList() with the same ContextCalOrCalibratedAilObjectId parameter passed. You can only inquire this value for a successfully calibrated M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0 | Specifies the number of camera calibration poses. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_CALIBRATION_IMAGE_POINTS_X; M_CALIBRATION_IMAGE_POINTS_Y; M_CALIBRATION_WORLD_POINTS_X; M_CALIBRATION_WORLD_POINTS_Y; M_CALIBRATION_WORLD_POINTS_Z.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For determining the required array size (number of elements) to store the returned values | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_NB_ELEMENTS | Retrieves the required array size (number of elements) to store the returned values. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a camera calibration context that has been calibrated using McalGrid(), McalList(), or McalUniform(), the InquireType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context calibrated using McalGrid(), McalList(), or McalUniform() | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_ASPECT_RATIO + | Inquires the average aspect ratio. The ratio is the average pixel width, divided by average pixel height, calculated with world units. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INVALID_SCALE | Specifies that camera calibration was not successful. This value can also be returned if the camera is positioned and oriented in such a way that all points in the image plane are invalid. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the average aspect ratio. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a 3D-based camera calibration context ( M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED), the InquireType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a 3D-based camera calibration context | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_CCD_ASPECT_RATIO + | Inquires the width to height ratio of the individual elements of the CCD. SET | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| 1.0 | Specifies that the width and height of the CCD element are equal. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the value of the width of a CCD element divided by its height. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DISTORTION_RADIAL_1 + | Inquires the value of the second order radial distortion coefficient used in the camera calibration algorithm. Radial distortion refers to image distortions caused by the camera's lens. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the value of the second order radial distortion coefficient. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISTORTION_RADIAL_2 + | Inquires the value of the fourth order radial distortion coefficient used in the camera calibration algorithm. Radial distortion refers to image distortions caused by the camera's lens. This radial distortion coefficient is only supported for M_ZHANG_BASED and M_3D_ROBOTICS_ZHANG_BASED camera calibration modes. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the value of the fourth order radial distortion coefficient. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FOCAL_LENGTH + | Inquires the effective focal length of the pinhole camera model used in the camera calibration. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the effective focal length of the pinhole camera model, expressed in horizontal pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PRINCIPAL_POINT_X + | Inquires the X-coordinate of the intersection of the camera's optical axis and the image plane. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies half of the image's width, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, in pixels. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PRINCIPAL_POINT_Y + | Inquires the Y-coordinate of the intersection of the camera's optical axis and the image plane. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies half of the image's height, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate, in pixels. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For any image, result buffer, or digitizer, the InquireType parameter can be set to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For any image, digitizer, or result buffer | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_ASSOCIATED_CALIBRATION + | Inquires the identifier of the associated camera calibration context. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_NULL | Specifies that there is no camera calibration context associated with the image or digitizer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT_UNIFORM_CALIBRATION | Specifies that the image was calibrated using McalUniform(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Calibration object identifier* | Specifies the camera calibration context that is associated with the image or digitizer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a camera calibration context, image, or result buffer, the InquireType parameter can be set to one of the following values. Note that a result buffer has exactly the same calibration information as the calibration information of the image used to obtain the results, if the result buffer's Aurora Imaging Library module supports returning results in real-world units; so when you inquire about a result buffer, the information returned is about the image on which the results were obtained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context, image, or result buffer | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| | UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_CALIBRATION_CATEGORY + | | Inquires whether the specified Aurora Imaging Library object is or is associated with a camera calibration context, and whether the associated context was allocated with a 2D-based or 3D-based calibration mode. SET | | | | | | | | | | | | | | | |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NULL | Specifies the Aurora Imaging Library object is not associated with a camera calibration context.<br><br>Inquiring the camera calibration category of a camera calibration context never returns M_NULL. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_2D_CALIBRATION | Specifies that the camera calibration context was allocated with a 2D-based camera calibration mode, such as M_LINEAR_INTERPOLATION.<br><br>When the specified Aurora Imaging Library object is an image with a constant pixel size, such as corrected images and depth maps, M_2D_CALIBRATION is returned, regardless of the calibration mode of the camera calibration context with which the object is associated. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_3D_CALIBRATION | Specifies that the camera calibration context was allocated with a 3D-based camera calibration mode, such as M_3D_ROBOTICS. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_MODE + | | Inquires the camera calibration mode of the associated camera calibration of the specified Aurora Imaging Library object. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NULL | Specifies the Aurora Imaging Library object is not associated with a camera calibration.<br><br>Inquiring the camera calibration mode of a camera calibration context never returns M_NULL. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_3D_ROBOTICS | Specifies a camera calibration context for a 3D-based camera calibration that uses a Tsai-based camera model in robotics mode. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_3D_ROBOTICS_ZHANG_BASED | Specifies a camera calibration context for a 3D-based camera calibration that uses a Zhang-based camera model in robotics mode. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LINEAR_INTERPOLATION | Specifies a camera calibration context for a 2D-based camera calibration that uses piecewise linear interpolation mode. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PERSPECTIVE_TRANSFORMATION | Specifies a camera calibration context for a 2D-based camera calibration that uses perspective transformation mode. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TSAI_BASED | Specifies a camera calibration context for a 3D-based camera calibration that uses Tsai-based mode. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNIFORM_TRANSFORMATION | Specifies uniform transformation mode.<br><br>Images with a constant pixel size (including corrected images and depth maps) are always uniform, even if the camera calibration mode of their associated camera calibration is not. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ZHANG_BASED | Specifies a camera calibration context for a 3D-based camera calibration that uses Zhang-based mode. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CONSTANT_PIXEL_SIZE + | Inquires whether the image has a constant pixel size.<br><br>When inquiring a camera calibration context, M_CONSTANT_PIXEL_SIZE will always return M_FALSE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FALSE | Specifies that the image does not have a constant pixel size, or that you are inquiring a camera calibration context. The size of each pixel depends on the camera calibration context associated with the image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRUE | Specifies that the image has a constant pixel size. Note, however, that pixels are not necessarily square; they can be rectangular and might be rotated. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CORRECTION_STATE + | Inquires whether the image has been physically corrected. Physically corrected images have square pixels with a constant size and no rotation between the pixel and absolute coordinate systems. Images can be corrected using McalTransformImage() with M_FULL_CORRECTION.<br><br>When inquiring a camera calibration context, M_CORRECTION_STATE will always return M_FALSE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FALSE | Specifies that the image has been neither corrected nor calibrated, or that you are inquiring a camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRUE | Specifies that the image has been corrected. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DEPTH_MAP + | Inquires whether the Aurora Imaging Library object is a fully corrected depth map. A fully corrected buffer is a calibrated buffer with a constant pixel size and with a valid Z-scale. Note that the size of the pixels is not necessarily square. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FALSE | Specifies that the Aurora Imaging Library object is not a fully corrected depth map.<br><br>Inquiring M_DEPTH_MAP on a camera calibration context returns M_FALSE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRUE | Specifies that the image is a fully corrected depth map.<br><br>> Note that when M_DEPTH_MAP returns M_TRUE, M_CONSTANT_PIXEL_SIZE also returns M_TRUE and M_GRAY_LEVEL_SIZE_Z will not return M_INVALID_SCALE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_Y_AXIS_DIRECTION + | Inquires the direction the Y-axis of the absolute coordinate system is oriented with respect to its positive X-axis.<br><br>If you used McalList() to calibrate your camera setup, the Y-axis orientation was determined by the calibration points that you specified. For more information, see the Calibrating using calibration points from a list section of Chapter 28: Calibrating your camera setup.<br><br>If you used McalGrid() to calibrate your camera setup, the explicitly specified Y-axis orientation is used.<br><br>Note that this constant does not support images without an associated camera calibration context. An uncalibrated image will return an error. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_Y_AXIS_CLOCKWISE | Specifies that the positive Y-axis is oriented 90° clockwise with respect to the positive X-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_Y_AXIS_COUNTER_CLOCKWISE | Specifies that the positive Y-axis is oriented 90° counter-clockwise with respect to the positive X-axis. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For an image or result buffer that is calibrated, the InquireType parameter can be set to one of the following values. Note that a result buffer has exactly the same calibration information as the calibration information of the image used to obtain the results, if the result buffer's Aurora Imaging Library module supports returning results in real-world units; so when you inquire about a result buffer, the information returned is about the image on which the results were obtained.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a calibrated image or result buffer | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_CALIBRATION_CHILD_OFFSET_X + | Inquires the X-offset of a child buffer relative to the highest calibrated parent image that was originally associated with the camera calibration context. SET | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-offset, relative to the child buffer's highest order calibrated parent buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_CALIBRATION_CHILD_OFFSET_Y + | Inquires the Y-offset of a child buffer relative to the highest calibrated parent image that was originally associated with the camera calibration context. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-offset, relative to the child buffer's highest order calibrated parent buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For an image that is calibrated and is distorted, the InquireType parameter can be set to one of the following values, before calling McalTransformImage().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a calibrated image with distortions | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_TRANSFORM_CLIP_SIZE_X_PRESERVE_PIXEL_SIZE + | Inquires the buffer width (SizeX) that the destination image should have to preserve the average pixel size of the source image, when calling McalTransformImage() with M_CLIP. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INVALID | Specifies that the width cannot be determined. This occurs when M_ASPECT_RATIO returns M_INVALID_SCALE, or when most of the image border pixels are outside of the calibration region. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the ideal width of the destination image buffer to preserve the average pixel size of the source image, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRANSFORM_CLIP_SIZE_Y_PRESERVE_PIXEL_SIZE + | Inquires the buffer height (SizeY) that the destination image should have to preserve the average pixel size of the source image, when calling McalTransformImage() with M_CLIP. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INVALID | Specifies that the height cannot be determined. This occurs when M_ASPECT_RATIO returns M_INVALID_SCALE, or when most of the image border pixels are outside of the calibration region. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the ideal height of the destination image buffer to preserve the average pixel size of the source image, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRANSFORM_FIT_SIZE_X_PRESERVE_PIXEL_SIZE + | Inquires the buffer width (SizeX) that the destination image should have to preserve the average pixel size of the source image, when calling McalTransformImage() with M_FIT. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_INVALID | Specifies that the width cannot be determined. This occurs when M_ASPECT_RATIO returns M_INVALID_SCALE, or when most of the image border pixels are outside of the calibration region. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the ideal width of the destination image buffer to preserve the average pixel size of the source image, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRANSFORM_FIT_SIZE_Y_PRESERVE_PIXEL_SIZE + | | Inquires the buffer height (SizeY) that the destination image should have to preserve the average pixel size of the source image, when calling McalTransformImage() with M_FIT. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_INVALID | Specifies that the height cannot be determined. This occurs when M_ASPECT_RATIO returns M_INVALID_SCALE, or when most of the image border pixels are outside of the calibration region. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* | Specifies the ideal height of the destination image buffer to preserve the average pixel size of the source image, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

The following formulas give the relationship between coordinates in pixel units and world units when an image or result buffer is calibrated and has a constant pixel size:

$$\begin{bmatrix} W_x \\ W_y \end{bmatrix} = \begin{bmatrix} \cos R & \sin R \\ -\sin R & \cos R \end{bmatrix} \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} P_x \\ P_y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix}$$

$$\begin{bmatrix} P_x \\ P_y \end{bmatrix} = \begin{bmatrix} 1/S_x & 0 \\ 0 & 1/S_y \end{bmatrix} \begin{bmatrix} \cos R & -\sin R \\ \sin R & \cos R \end{bmatrix} \begin{bmatrix} W_x - T_x \\ W_y - T_y \end{bmatrix}$$

If an image or result buffer is calibrated and has a constant pixel-size, the InquireType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a calibrated image or result buffer, with a constant pixel size | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| | UserVarPtr - Possible values returned | | | | | | | | | | | | | | | | |
| M_GRAY_LEVEL_SIZE_Z + | | Inquires the step, in world units, along the Z-axis of the relative coordinate system, represented by one gray level. Since the Z-axis is pointing downwards, positive M_GRAY_LEVEL_SIZE_Z values mean that lower (darker) pixel values represent higher world points (top-black); negative M_GRAY_LEVEL_SIZE_Z values mean that higher (brighter) pixel values represent higher world points (top-white).<br><br>This inquire type is only useful when inquiring about an image buffer that contains a depth map. SET | | | | | | | | | | | | | | | |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_INVALID_SCALE | Specifies that the image is not a depth map. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *Value* != 0.0 | Specifies the height, in world units, corresponding to a difference of one gray level. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_PIXEL_ROTATION + | Inquires the angle of the X-axis of the pixel coordinate system measured in the relative world coordinate system. A positive value indicates a counter-clockwise rotation (from the positive X-axis of the relative world coordinate system toward its negative Y-axis).<br><br>This value corresponds to *R* in the above formulas. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the angle, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PIXEL_SIZE_X + | Inquires the width of the pixels in the corrected image.<br><br>You can multiply a measure in pixels, along the X-axis, by M_PIXEL_SIZE_X to get the measure in world units. If there is rotation between the relative world coordinate system and the pixel coordinate system, this distance won't be along the X-axis of the relative world coordinate system.<br><br>This value corresponds to $S_x$ in the above formulas. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the width, in world units/pixel. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PIXEL_SIZE_Y + | Inquires the height of the pixels in the corrected image.<br><br>You can multiply a measure in pixels, along the Y-axis, by M_PIXEL_SIZE_Y to get the measure in world units. If there is rotation between the relative world coordinate system and the pixel coordinate system, this distance won't be along the Y-axis of the relative world coordinate system.<br><br>This value corresponds to $S_y$ in the above formulas. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the height, in world units/pixel. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_POS_X + | Inquires the X-coordinate of the center of the top-left pixel in the highest calibrated parent image that was originally associated with the camera calibration context. This value corresponds to $T_x$ in the above formulas.<br><br>Note that this also applies to a clone of a child buffer (which is not a child itself). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, expressed in the relative world coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_POS_Y + | Inquires the Y-coordinate of the center of the top-left pixel in the highest calibrated parent image that was originally associated with the camera calibration context. This value corresponds to $T_y$ in the above formulas.<br><br>Note that this also applies to a clone of a child buffer (which is not a child itself). SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate, expressed in the relative world coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_POS_Z + | Inquires the Z-coordinate of gray level 0 in the corrected image. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Value* | Specifies the base height of a gray level of 0, expressed in the relative world coordinate system . | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for the values listed in For a calibrated image or result buffer, For a calibrated image with distortions; and for the following values: M_CALIBRATION_PLANE; M_LINK_TOOL_AND_HEAD; M_RELATIVE_ORIGIN_ANGLE; M_RELATIVE_ORIGIN_X; M_RELATIVE_ORIGIN_Y; M_RELATIVE_ORIGIN_Z; M_TOOL_POSITION_X; M_TOOL_POSITION_Y; M_TOOL_POSITION_Z; M_TOOL_ROTATION_Z; M_TRANSFORM_CACHE; M_FOREGROUND_VALUE; M_GRID_FIDUCIAL; M_GRID_HINT_ANGLE_X; M_GRID_HINT_PIXEL_X; M_GRID_HINT_PIXEL_Y; M_CCD_ASPECT_RATIO; M_GRAY_LEVEL_SIZE_Z; M_WORLD_POS_Z.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For inquiring the default value | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_DEFAULT | Inquires the default value of the specified inquire type. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a fixturing offset object, the InquireType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a fixturing offset object | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_ANGLE + | Inquires the angular offset to apply to the reference location when setting the relative world coordinate system. SET | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the angular offset. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_POSITION_X + | Inquires the X-offset to apply to the reference location when setting the relative world coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-offset. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_POSITION_Y + | Inquires the Y-offset to apply to the reference location when setting the relative world coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-offset. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

For a 3D draw calibration context, InquireType can be set to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a 3D draw calibration context |
|---|

| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_DRAW_ABSOLUTE_COORDINATE_SYSTEM + | Inquires whether to draw the absolute coordinate system's axes. SET | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies not to draw the absolute coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies to draw the absolute coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CAMERA_COORDINATE_SYSTEM + | Inquires whether to draw the camera coordinate system's axes. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies not to draw the camera coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies to draw the camera coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_CAMERA_COORDINATE_SYSTEM_NAME + | Inquires the name to draw for the camera coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_COORDINATE_SYSTEM_LENGTH + | Inquires the length at which to draw the specified coordinate system's axes. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the length (in world units) at which to draw the axes of the specified coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_FRUSTUM + | Inquires whether to draw the frustum. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISABLE | Specifies not to draw the frustum. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ENABLE | Specifies to draw the frustum. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_FRUSTUM_COLOR + | Inquires the frustum's color. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_COLOR_BLACK | Specifies the color black. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_COLOR_BLUE | Specifies the color blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BRIGHT_GRAY | Specifies the color bright gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_CYAN | Specifies the color cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_BLUE | Specifies the color dark blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_CYAN | Specifies the color dark cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_GREEN | Specifies the color dark green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_MAGENTA | Specifies the color dark magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_RED | Specifies the color dark red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_YELLOW | Specifies the color dark yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GRAY | Specifies the color gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GREEN | Specifies the color green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_BLUE | Specifies the color light blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GRAY | Specifies the color light gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GREEN | Specifies the color light green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_WHITE | Specifies the color light white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_MAGENTA | Specifies the color magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_RED | Specifies the color red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_WHITE | Specifies the color white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_YELLOW | Specifies the color yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NO_COLOR | Specifies no color. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_COORDINATE_SYSTEM + | | Inquires whether to draw the relative coordinate system's axes. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the relative coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_ENABLE | Specifies to draw the relative coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_COORDINATE_SYSTEM_NAME + | | Inquires the name to draw for the relative coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE + | | Inquires whether to draw the relative XY plane. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the relative XY plane. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the relative XY plane. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE_COLOR_FILL + | | Inquires the relative XY plane's fill color. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_AUTO_COLOR | Specifies either the color white or the texture image.<br><br>If a texture image is specified (using McalDraw3d() with RelXYPlaneTextureImageBufId), the texture image is drawn on the laser plane. Otherwise, the plane is drawn with M_COLOR_WHITE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLACK | Specifies the color black. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLUE | Specifies the color blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BRIGHT_GRAY | Specifies the color bright gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_CYAN | Specifies the color cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_BLUE | Specifies the color dark blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_CYAN | Specifies the color dark cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_GREEN | Specifies the color dark green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_MAGENTA | Specifies the color dark magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_RED | Specifies the color dark red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_YELLOW | Specifies the color dark yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GRAY | Specifies the color gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_COLOR_GREEN | Specifies the color green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_BLUE | Specifies the color light blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GRAY | Specifies the color light gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GREEN | Specifies the color light green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_WHITE | Specifies the color light white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_MAGENTA | Specifies the color magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_RED | Specifies the color red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_WHITE | Specifies the color white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_YELLOW | Specifies the color yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NO_COLOR | Specifies no color. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_TEXTURE_IMAGE | Specifies to use the image passed to McalDraw3d() with RelXYPlaneTextureImageBufId, when drawing the relative XY plane. The texture image is typically a 2D image of the 3D scene. For example, you can specify the image used for calibration. If no texture image is specified using McalDraw3d(), setting M_DRAW_RELATIVE_XY_PLANE_COLOR_FILL to M_TEXTURE_IMAGE will cause an error. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE_COLOR_OUTLINE + | | Inquires the relative XY plane's outline color. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLACK | Specifies the color black. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BLUE | Specifies the color blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_BRIGHT_GRAY | Specifies the color bright gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_CYAN | Specifies the color cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_BLUE | Specifies the color dark blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_CYAN | Specifies the color dark cyan. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_GREEN | Specifies the color dark green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_MAGENTA | Specifies the color dark magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_COLOR_DARK_RED | Specifies the color dark red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_DARK_YELLOW | Specifies the color dark yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GRAY | Specifies the color gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_GREEN | Specifies the color green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_BLUE | Specifies the color light blue. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GRAY | Specifies the color light gray. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_GREEN | Specifies the color light green. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_LIGHT_WHITE | Specifies the color light white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_MAGENTA | Specifies the color magenta. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_RED | Specifies the color red. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_WHITE | Specifies the color white. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_COLOR_YELLOW | Specifies the color yellow. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_NO_COLOR | Specifies no color. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_RELATIVE_XY_PLANE_OPACITY + | | Inquires the relative XY plane's opacity. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | 0.0 <= *Value* <= 100.0 | Specifies the relative XY plane's opacity. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_ROBOT_BASE_COORDINATE_SYSTEM + | | Inquires whether to draw the robot base coordinate system's axes. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the robot base coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_ENABLE | Specifies to draw the robot base coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DRAW_TOOL_COORDINATE_SYSTEM + | | Inquires whether to draw the tool coordinate system's axes. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DISABLE | Specifies not to draw the tool coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_ENABLE | Specifies to draw the tool coordinate system's axes. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_GRID_UNIT_SHORT_NAME; M_DRAW_CAMERA_COORDINATE_SYSTEM_NAME; M_DRAW_RELATIVE_COORDINATE_SYSTEM_NAME.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For getting the string size | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_STRING_SIZE + | Retrieves the length of the string, including the terminating null character ("\0"). | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for the values listed in For a camera calibration context, For a camera calibration context calibrated using McalGrid() or McalList(), For a camera calibration context calibrated using McalGrid(), McalList(), or McalUniform(), For a 3D-based camera calibration context, For any image, digitizer, or result buffer, For a camera calibration context, image, or result buffer, For a calibrated image or result buffer, For a calibrated image with distortions, For a calibrated image or result buffer, with a constant pixel size, For a fixturing offset object; and for the following values: M_COLUMN_NUMBER; M_COLUMN_SPACING; M_FOREGROUND_VALUE; M_GRID_FIDUCIAL; M_GRID_HINT_ANGLE_X; M_GRID_HINT_PIXEL_X; M_GRID_HINT_PIXEL_Y; M_GRID_ORIGIN_X; M_GRID_ORIGIN_Y; M_GRID_ORIGIN_Z; M_GRID_PARTIAL; M_GRID_SHAPE; M_GRID_TYPE; M_GRID_UNITS; M_ROW_NUMBER; M_ROW_SPACING; M_DRAW_ABSOLUTE_COORDINATE_SYSTEM; M_DRAW_CAMERA_COORDINATE_SYSTEM; M_DRAW_COORDINATE_SYSTEM_LENGTH; M_DRAW_FRUSTUM; M_DRAW_FRUSTUM_COLOR; M_DRAW_RELATIVE_COORDINATE_SYSTEM; M_DRAW_RELATIVE_XY_PLANE; M_DRAW_RELATIVE_XY_PLANE_COLOR_FILL; M_DRAW_RELATIVE_XY_PLANE_COLOR_OUTLINE; M_DRAW_RELATIVE_XY_PLANE_OPACITY; M_DRAW_ROBOT_BASE_COORDINATE_SYSTEM; M_DRAW_TOOL_COORDINATE_SYSTEM; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the data type | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_TYPE_AIL_DOUBLE | Casts the requested information to an *AIL_DOUBLE*. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* **\|** *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_FLOAT | Casts the requested information to an *AIL_FLOAT*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT> ]* **\|** *address of an AIL_FLOAT* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_ID | Casts the requested information to an *AIL_ID*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID> ]* **\|** *address of an AIL_ID* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT | Casts the requested information to an *AIL_INT*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT> ]* **\|** *address of an AIL_INT* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT16 | Casts the requested results to an *AIL_INT16*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16> ]* **\|** *address of an AIL_INT16* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT32 | Casts the requested information to an *AIL_INT32*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32> ]* **\|** *address of an AIL_INT32* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT64 | Casts the requested information to an *AIL_INT64*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64> ]* **\|** *address of an AIL_INT64* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalInquireSingle

## Synopsis

Inquire about a single pose in a sequence of poses taken during camera calibration.

## Syntax

```
AIL_INT McalInquireSingle(
    AIL_ID ContextCalOrCalibratedAilObjectId, //in
    AIL_INT Index,                            //in
    AIL_INT64 InquireType,                    //in
    void *UserVarPtr                          //out
)
```

## Description

This function allows you to inquire about the settings of a single pose in the sequence of poses performed to calibrate M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration contexts. Every call to McalGrid() or McalList() with M_ACCUMULATE corresponds to a pose used to obtain information about the camera setup during the camera calibration process.

## Parameters

ContextCalOrCalibratedAilObjectId

Specifies the identifier of the camera calibration context.

Index

Specifies the index of the pose of an M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration context from which settings should be inquired. The first call corresponds to the index value 0 and the last one is the return value of McalInquire() with M_NUMBER_OF_CALIBRATION_POSES minus one.

InquireType

Specifies the setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT> ]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID> ]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT> ]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16> ]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32> ]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64> ]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_UINT64

Specifies the address in which to return the value of the inquired setting. Since the **McalInquireSingle**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

## Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For a camera calibration context being calibrated using McalGrid()
- For a camera calibration context being calibrated using McalGrid() or McalList()

The InquireType parameter can be set to one of the following values only after calibrating with McalGrid().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context being calibrated using McalGrid() | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_COLUMN_NUMBER + | Inquires the number of columns in the camera calibration grid. SET | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 2 | Specifies the number of columns. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_COLUMN_SPACING + | Inquires the number of world units between columns. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the spacing between columns. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_ORIGIN_X + | Inquires the X-position of the top-left circle of a circle grid, or the top-left point connecting four squares in a chessboard grid, in the camera calibration plane coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-position, in the camera calibration plane coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_ORIGIN_Y + | Inquires the Y-position of the top-left circle of a circle grid, or the top-left point connecting four squares in a chessboard grid, in the camera calibration plane coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-position, in the camera calibration plane coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_ORIGIN_Z + | Inquires the Z-position of the top-left circle of a circle grid, or the top-left point connecting four squares in a chessboard grid, in the camera calibration plane coordinate system. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Z-position, in the camera calibration plane coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_TYPE + | Inquires the type of grid used to perform the camera calibration. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CHESSBOARD_GRID | Specifies a chessboard grid. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CIRCLE_GRID | Specifies a grid of circles. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_GRID_UNIT_SHORT_NAME + | Inquires the units returned by M_GRID_UNITS. You can find the particular abbreviation for each unit in the description of M_GRID_UNITS. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID_UNITS + | Inquires the units of measurement encoded in a chessboard grid with a fiducial.<br><br>Note that this information is not used by the calibration module. No unit consistency is enforced and no unit conversion is performed. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CENTIMETERS | Specifies that the grid units are measured in centimeters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FEET | Specifies that the grid units are measured in feet. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INCHES | Specifies that the grid units are measured in inches. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_KILOMETERS | Specifies that the grid units are measured in kilometers. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_METERS | Specifies that the grid units are measured in meters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MICROMETERS | Specifies that the grid units are measured in micrometers. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MILES | Specifies that the grid units are measured in miles. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MILLIMETERS | Specifies that the grid units are measured in millimeters. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MILS | Specifies that the grid units are measured in mils. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNKNOWN | Specifies that grid units are measured in an unknown unit. INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROW_NUMBER + | Inquires the number of rows in the camera calibration grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 2 | Specifies the number of rows. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROW_SPACING + | Inquires the spacing between rows in the camera calibration grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* > 0.0 | Specifies the spacing between rows, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_GRID_UNIT_SHORT_NAME.

You can add the following value to the above-mentioned value to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For getting the string size | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | |
| M_STRING_SIZE + | Retrieves the length of the string, including the terminating null character ("\0"). | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

The InquireType parameter can be set to one of the following values after a call to either McalGrid() or McalList().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For a camera calibration context being calibrated using McalGrid() or McalList() | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |
| M_AVERAGE_PIXEL_ERROR + | Inquires the average camera calibration error in the pixel coordinate system.<br>This is the average distance in the pixel coordinate system between the initial calibration points and their projected points in an image. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the average camera calibration error, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_AVERAGE_WORLD_ERROR + | Inquires the average camera calibration error in the absolute coordinate system.<br>This is the average distance in the absolute coordinate system between the initial calibration points and their projected points in an image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the average camera calibration error, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_IMAGE_POINTS_X + | Inquires the X-pixel coordinate of the calibration points.<br>If you used McalGrid() to calibrate your camera setup, the calibration points' pixel coordinates are determined by the pixel positions of the centers of the circles in a circle grid or the intersections of four squares/rectangles in a chessboard grid.<br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' pixel coordinates with the PixCoordXArrayPtr parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_CALIBRATION_IMAGE_POINTS_Y + | Inquires the Y-pixel coordinate of the calibration points.<br><br>If you used McalGrid() to calibrate your camera setup, the calibration points' pixel coordinates are determined by the pixel positions of the centers of the circles in a circle grid or the intersections of four squares/rectangles in a chessboard grid.<br><br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' pixel coordinates with the PixCoordYArrayPtr parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_INPUT_DATA + | Specifies the type of data that was used to perform the camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_GRID | Specifies that the camera calibration was performed using a camera calibration grid (McalGrid()). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LIST | Specifies that the camera calibration was performed by explicitly specifying the correspondence between some pixels and their real-world coordinates (McalList()). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_WORLD_POINTS_X + | Inquires the X-world coordinate of the calibration points that are computed based on explicitly specified values. The coordinate is expressed in world units of the camera calibration plane (M_CALIBRATION_PLANE).<br><br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' world coordinates with the WorldCoordXArrayPtr parameter.<br><br>If you used McalGrid() to calibrate your camera setup, the calibration points are computed from the parameters GridOffsetX, ColumnNumber, and ColumnSpacing. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the X-coordinate, in real-world units of the camera calibration plane (M_CALIBRATION_PLANE). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CALIBRATION_WORLD_POINTS_Y + | Inquires the Y-world coordinate of the calibration points that are computed based on explicitly specified values. The coordinate is expressed in world units of the camera calibration plane (M_CALIBRATION_PLANE).<br><br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' world coordinates with the WorldCoordYArrayPtr parameter.<br><br>If you used McalGrid() to calibrate your camera setup, the calibration points are computed from the parameters GridOffsetY, RowNumber, and RowSpacing. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Y-coordinate, in real-world units of the camera calibration plane (M_CALIBRATION_PLANE). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_CALIBRATION_WORLD_POINTS_Z + | Inquires the Z-world coordinate of the calibration points that are based on explicitly specified values. The computed calibration points are expressed in world units of the camera calibration plane (M_CALIBRATION_PLANE). If this value is inquired for 2D-based camera calibration contexts, the specified array will be filled with 0.0 values.<br><br>If you used McalList() to calibrate your camera setup, you explicitly set the calibration points' pixel coordinates with the WorldCoordZArrayPtr parameter.<br><br>If you used McalGrid() to calibrate your camera setup, you explicitly set the Z-coordinate of all the calibration points with the GridOffsetZ parameter. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the Z-coordinate, in real-world units of the camera calibration plane (M_CALIBRATION_PLANE). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MAXIMUM_PIXEL_ERROR + | Inquires the maximum camera calibration error, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the maximum camera calibration error, in pixels. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MAXIMUM_WORLD_ERROR + | Inquires the maximum camera calibration error, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0.0 | Specifies the maximum camera calibration error, in world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NUMBER_OF_CALIBRATION_POINTS + | Inquires the number of calibration points found by McalGrid() or passed to McalList().<br><br>If you used McalList() to calibrate your camera setup, you can explicitly set the number of calibration points with the NumPoint parameter.<br><br>If you used McalGrid() to calibrate your camera setup, the number of calibration points is determined by the number of columns and rows in your grid. SET | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 0 | Specifies the number of calibration points. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_CALIBRATION_IMAGE_POINTS_X; M_CALIBRATION_IMAGE_POINTS_Y; M_CALIBRATION_WORLD_POINTS_X; M_CALIBRATION_WORLD_POINTS_Y; M_CALIBRATION_WORLD_POINTS_Z.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For determining the required array size (number of elements) to store the returned values | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr<br>- Possible values returned | | | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_NB_ELEMENTS | Retrieves the required array size (number of elements) to store the returned values. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for the values listed in For a camera calibration context being calibrated using McalGrid() or McalList(); and for the following values: M_COLUMN_NUMBER; M_COLUMN_SPACING; M_GRID_ORIGIN_X; M_GRID_ORIGIN_Y; M_GRID_ORIGIN_Z; M_GRID_TYPE; M_GRID_UNITS; M_ROW_NUMBER; M_ROW_SPACING; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the data type | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| InquireType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| UserVarPtr - Possible values returned | | | | | | | | | | | | | | | | |
| M_TYPE_AIL_DOUBLE | Casts the requested information to an *AIL_DOUBLE*. | | | | | | | | | | | | | | | |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE> ]* **\|** *address of an AIL_DOUBLE* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_FLOAT | Casts the requested information to an *AIL_FLOAT*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT> ]* **\|** *address of an AIL_FLOAT* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_ID | Casts the requested information to an *AIL_ID*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID> ]* **\|** *address of an AIL_ID* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT | Casts the requested information to an *AIL_INT*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT> ]* **\|** *address of an AIL_INT* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT16 | Casts the requested results to an *AIL_INT16*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16> ]* **\|** *address of an AIL_INT16* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT32 | Casts the requested information to an *AIL_INT32*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32> ]* **\|** *address of an AIL_INT32* | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TYPE_AIL_INT64 | Casts the requested information to an *AIL_INT64*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *UserVarPtr extra info* | Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64> ]* **\|** *address of an AIL_UINT64* | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

### Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalList

## Synopsis

Calibrate your camera setup using a list of coordinates.

## Syntax

```
void McalList(
    AIL_ID CalibrationId,                  //out
    const AIL_DOUBLE *PixCoordXArrayPtr,   //in
    const AIL_DOUBLE *PixCoordYArrayPtr,   //in
    const AIL_DOUBLE *WorldCoordXArrayPtr, //in
    const AIL_DOUBLE *WorldCoordYArrayPtr, //in
    const AIL_DOUBLE *WorldCoordZArrayPtr, //in
    AIL_INT NumPoint,                      //in
    AIL_INT64 Operation,                   //in
    AIL_INT64 ControlFlag                  //in
)
```

## Description

This function uses a specified list of calibration points to calibrate your camera setup. Calibration points are pixel coordinates and their associated real-world coordinates. The mapping is stored with the specified camera calibration context. The specified camera calibration context determines the camera calibration mode used.

Unlike when using McalGrid(), **McalList**() does not use a default orientation for the Y-axis of the absolute coordinate system. It establishes the orientation based on the specified calibration points. It arbitrarily selects three of the specified points (for example, point A, B, and C). It then establishes the direction that a line between point A and B would rotate to meet a line between point A and C while using the smallest angle of rotation. It finally selects the Y-axis orientation for the absolute coordinate system so that the rotation direction in the pixel coordinate system and in the absolute coordinate system are the same. You can use M_Y_AXIS_DIRECTION to ensure that the orientation of the Y-axis was set correctly. For more information, see the Calibrating using calibration points from a list section of Chapter 28: Calibrating your camera setup.

Typically, you specify the list of calibration points in the absolute coordinate system. However, for 3D-based camera calibration modes (M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, and M_3D_ROBOTICS_ZHANG_BASED), it is possible to use a list of calibration points that are not in the Z=0 plane of the absolute coordinate system. In this case, it is possible to describe the points in the relative coordinate system instead, using McalControl() with M_CALIBRATION_PLANE set to M_RELATIVE_COORDINATE_SYSTEM. You can then use McalSetCoordinateSystem() to move the relative coordinate system. Both of these calls must be made before calling **McalList**().

When working in M_3D_ROBOTICS camera calibration mode, you must perform at least three calls to **McalList**() with M_ACCUMULATE before performing a full camera calibration. Before each call, you must change the position and orientation of the tool holding the camera with respect to the robot base. More specifically, you must rotate the tool along at least two non-parallel axes, retrieve the tool's new position with respect to the robot base from the robot encoder, and using this information, set the position of the tool coordinate system with respect to the robot base coordinate system using McalSetCoordinateSystem(). When you are done accumulating data, you must call this function with M_FULL_CALIBRATION and no image to perform the full camera calibration. Once you perform a full camera calibration, you can no longer accumulate camera calibration poses. Using **McalList**() with M_ACCUMULATE more than three times before performing a full camera calibration greatly improves the accuracy of your camera calibration.

When working in M_ZHANG_BASED camera calibration mode, you must perform at least three calls to **McalList**() with M_ACCUMULATE before performing a full camera calibration. For each call, you must provide a list of calibration points taken from a different point of view than a previous call to this function with M_ACCUMULATE. The pose provided in the last call will be used to automatically set the position of the absolute and camera coordinate systems. When you are done accumulating data, you must call this function with M_FULL_CALIBRATION and no image to perform the full camera calibration. Once you perform a full camera calibration, you can no longer accumulate camera calibration poses. Calling this function with M_ACCUMULATE more than three times greatly improves the accuracy of the camera calibration.

When working in M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, you must perform at least three calls to **McalList**() with M_ACCUMULATE before performing a full camera calibration. For each call, you must follow the steps described in the previous two paragraphs. When you are done accumulating data, you must call this function with M_FULL_CALIBRATION and no image to perform the full camera calibration. Once you perform a full camera calibration, you can no longer accumulate camera calibration poses. Calling this function with M_ACCUMULATE more than three times greatly improves the accuracy of the camera

calibration.

> Note that for an M_3D_ROBOTICS, M_ZHANG_BASED, and M_3D_ROBOTICS_ZHANG_BASED calibration context, which require multiple calls to **McalList**(), you can inquire about any particular call using McalInquireSingle(). For example, you can inquire about the pixel coordinates and associated real-world coordinates of each calibration point of one particular call using McalInquireSingle().

For 3D-based camera calibration modes, **McalList**() performs two types of calculations. It calculates the camera's internal attributes, and it calculates the orientation and distance between the camera and camera calibration plane. Initially, the latter is used to set the camera coordinate system. If you move the camera or the grid (not both), you can have **McalList**() recalculate only the orientation and distance between them. In this case, depending on what you moved, you can specify that **McalList**() displace either the camera coordinate system or the relative coordinate system, using M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD, respectively. Note that M_DISPLACE_CAMERA_COORD is not supported for an M_STATIONARY_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode. When using M_DISPLACE_RELATIVE_COORD, the camera calibration plane is considered to be the XY-plane of the relative coordinate system, regardless of how McalControl() with M_CALIBRATION_PLANE is set.

When working with M_3D_ROBOTICS and M_TSAI_BASED, it is mandatory to set the image coordinates of the principal point prior to calibrating using McalControl() with M_PRINCIPAL_POINT_X, and M_PRINCIPAL_POINT_Y. For M_ZHANG_BASED and M_3D_ROBOTICS_ZHANG_BASED camera calibrations, M_PRINCIPAL_POINT_X and M_PRINCIPAL_POINT_Y are determined during calibration. If the aspect ratio of the CCD element is different than 1, specify it using McalControl() with M_CCD_ASPECT_RATIO before calling **McalList**().

Also, for most 3D-based camera calibration modes, a successful call to McalGrid() with M_FULL_CALIBRATION creates a rigid link between the tool coordinate system and the head (camera or reference/calibration object) coordinate system. This link ensures that moving either the tool or the head coordinate system will affect both. This link can be broken using McalControl() with M_LINK_TOOL_AND_HEAD.

> Note that the internal attributes calculated for the camera are not those of the physical camera, but those of the ideal pinhole-camera used to model the physical camera.

After calling **McalList**(), you can inquire about the success of the camera calibration using McalInquire() with M_CALIBRATION_STATUS. Also, you can inquire about the pixel coordinates and associated real-world coordinates of each calibration point using McalInquire() with M_CALIBRATION_IMAGE_POINTS_X, M_CALIBRATION_IMAGE_POINTS_Y, M_CALIBRATION_WORLD_POINTS_X and M_CALIBRATION_WORLD_POINTS_Y.

For more information on performing camera calibration, see the Steps to performing a camera calibration section of Chapter 28: Calibrating your camera setup.

### Parameters

CalibrationId

Specifies the identifier of the camera calibration context.

PixCoordXArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoint)

Specifies the address of the array containing the X-pixel coordinates.

> When performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

PixCoordYArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoint)

Specifies the address of the array containing the Y-pixel coordinates.

> When performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

WorldCoordXArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoint)

Specifies the address of the array containing the X-world coordinates in the camera calibration-plane coordinate system.

> When performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

WorldCoordYArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoint)

Specifies the address of the array containing the Y-world coordinates in the camera calibration-plane coordinate system.

> When performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_ZHANG_BASED, or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

WorldCoordZArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoint)

Specifies the address of the array containing the Z-world coordinates.

For 2D-based camera calibration modes (M_UNIFORM_TRANSFORMATION, M_LINEAR_INTERPOLATION or M_PERSPECTIVE_TRANSFORMATION), this parameter must be set to **M_NULL**.

For an M_TSAI_BASED camera calibration mode, if the object you are imaging is planar (has all its points on a single plane), this parameter must be set to **M_NULL**. However, if the object is non-planar, it is possible to provide the Z-coordinates and the minimum camera angle is not required.

For an M_ZHANG_BASED camera calibration mode, if the object you are imaging is planar (has all its points on a single plane), this parameter must be set to **M_NULL**. However, if the object is non-planar, it is possible to provide the Z-coordinates.

For an M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED camera calibration mode, this parameter must be set to **M_NULL** if the context is not yet calibrated. However, you can provide Z-world coordinates when using **McalList**() with M_DISPLACE_RELATIVE_COORD or M_DISPLACE_CAMERA_COORD.

NumPoint INQ

Specifies the number of coordinates in the supplied arrays.

For M_UNIFORM_TRANSFORMATION and M_LINEAR_INTERPOLATION camera calibration modes, the minimum number of coordinates is 3. For M_PERSPECTIVE_TRANSFORMATION camera calibration mode, the minimum number of coordinates is 4.

For M_TSAI_BASED and M_3D_ROBOTICS camera calibrations, using no Z-coordinates, the minimum number of coordinates is 5.

For M_TSAI_BASED camera calibrations, using Z-coordinates, the minimum number of coordinates is 7.

For M_ZHANG_BASED and M_3D_ROBOTICS_ZHANG_BASED camera calibrations the minimum number of coordinates is 7.

For 3D-based camera calibration contexts (M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, and M_3D_ROBOTICS_ZHANG_BASED camera calibrations), using Operation with M_DISPLACE_CAMERA_COORD or M_DISPLACE_RELATIVE_COORD, the minimum number of calibration points required is 4.

> Note, the specified pixel coordinates should cover the area of the image from which you want real-world coordinates (the working area). When performing an M_FULL_CALIBRATION operation in M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, or M_ZHANG_BASED camera calibration mode, set this parameter to **M_NULL**.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the PixCoordXArrayPtr, PixCoordYArrayPtr, WorldCoordXArrayPtr, WorldCoordYArrayPtr, or WorldCoordZArrayPtr parameter.

Operation

Specifies the camera calibration operation to perform. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the camera calibration operation | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_FULL_CALIBRATION. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ACCUMULATE | Specifies to receive only the position of the calibration points and stores them in the camera calibration context.<br><br>For M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, and M_ZHANG_BASED, at least three calls to **McalList**() should be made with this operation before performing the full camera calibration with M_FULL_CALIBRATION. Providing more than three calls can greatly improve the accuracy of the camera calibration.<br><br>This operation is only supported for M_3D_ROBOTICS, M_3D_ROBOTICS_ZHANG_BASED, and M_ZHANG_BASED camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISPLACE_CAMERA_COORD | Calculates only the position and orientation between the camera and the camera calibration plane, and displaces the camera coordinate system accordingly. Note that besides the camera coordinate system, this also displaces the tool coordinate system (if still linked with M_LINK_TOOL_AND_HEAD); no other coordinate system is affected. This camera calibration operation is only supported for 3D-based camera calibration contexts that are fully calibrated with M_FULL_CALIBRATION.<br><br>If the camera is moved to a new position but its internal attributes are already known by a previous full camera calibration, you can use this operation to allow for a faster camera calibration.<br><br>Note that M_DISPLACE_CAMERA_COORD is not supported for an M_STATIONARY_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DISPLACE_RELATIVE_COORD | Calculates only the relative coordinate system position in space. This camera calibration operation is only supported for 3D-based camera calibration contexts that are fully calibrated with M_FULL_CALIBRATION.<br><br>You can use this operation to move the relative coordinate system only and remain calibrated. This operation keeps the camera fixed with respect to the absolute coordinate system, and its intrinsic attributes unmodified. You can then obtain the position and orientation of the relative coordinate system with respect to any other coordinate system using McalGetCoordinateSystem(), and calculate the unknown position of an object in space.<br><br>When using this operation, the camera calibration plane is considered to be the relative coordinate system regardless of the M_CALIBRATION_PLANE setting. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_FULL_CALIBRATION | Performs a full camera calibration. For 3D-based camera calibration contexts, this operation calculates the camera's internal attributes and the difference in position and orientation between the camera and the camera calibration plane. It then sets the camera coordinate system accordingly.<br><br>> When working in M_TSAI_BASED camera calibration mode, the camera's optical axis should be at least 30 degrees away from the axis perpendicular to the camera calibration plane (also known as, angle of incidence). Otherwise, the camera calibration might fail. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalRelativeOrigin

## Synopsis

Change the origin and/or orientation of the relative coordinate system.

## Syntax

```
void McalRelativeOrigin(
    AIL_ID CalibrationOrImageId, //out
    AIL_DOUBLE XOffset,          //in
    AIL_DOUBLE YOffset,          //in
    AIL_DOUBLE ZOffset,          //in
    AIL_DOUBLE AngularOffset,    //in
    AIL_INT64 ControlFlag        //in
)
```

## Description

This function changes the origin and/or orientation of the relative coordinate system of a camera calibration context or a calibrated image. If you specify a camera calibration context, the new location of the relative coordinate system will be taken into account in images associated with the camera calibration context after the function call, but images previously associated with the camera calibration context will not be affected. After the function call, you can reference all images with the new relative coordinate system in world units with respect to the new origin.

Moving the relative coordinate system of an image is useful for analyzing an object with respect to a temporary local coordinate system. The relative coordinate system is moved with respect to the specified reference coordinate system. The reference coordinate system can be the absolute coordinate system (M_ASSIGN) or the current location of relative coordinate system (M_COMPOSE_WITH_CURRENT); when it is the absolute coordinate system, the relative coordinate system is re-initialized to the location and orientation of the absolute coordinate system before applying the specified transformation. You can also use McalFixture() to move the relative coordinate system to the location of a specified occurrence of an Aurora Imaging Library Pattern Matching or Model Finder model.

If you specify both a rotation and a translation of the relative coordinate system, the rotation is done first.

> Note that when the reference coordinate system is the absolute coordinate system, setting the XOffset, YOffset, ZOffset, and AngularOffset parameters to 0 has the effect of resetting the relative coordinate system to that of the absolute coordinate system.

To manipulate the relative coordinate system of a 3D-based camera calibration context (M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED), it is recommended to use McalSetCoordinateSystem() instead of **McalRelativeOrigin**() since the former supports more transformation types.

If you adjust the relative coordinate system of a calibrated image associated with an M_VECTOR_AND_RASTER ROI, the raster information will be discarded, causing the ROI to become an M_VECTOR ROI. See MbufSetRegion() for more information.

## Parameters

CalibrationOrImageId

Specifies the identifier of the camera calibration context or calibrated image.

The image buffer must be a 1- or 3-band buffer.

XOffset INQ

Specifies the X-offset of the relative coordinate system origin from the reference coordinate system, in world-units.

YOffset INQ

Specifies the Y-offset of the relative coordinate system origin from the reference coordinate system, in world-units.

ZOffset INQ

Specifies the Z-offset of the relative coordinate system origin from the reference coordinate system, in world-units.

> For 2D-based camera calibration contexts, this parameter must be set to **M_NULL**.

AngularOffset INQ

Specifies the angle, in degrees, at which to rotate the relative coordinate system around the Z-axis of the reference coordinate system. A positive angle will rotate the relative coordinate system in the direction from the positive X-axis to the negative Y-axis of the reference coordinate system.

ControlFlag

Specifies how to apply the transformation. This establishes the reference coordinate system. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the reference coordinate system | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_ASSIGN. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ASSIGN | Specifies to assign the specified transformation to the relative coordinate system from the origin and orientation of the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_COMPOSE_WITH_CURRENT | Specifies to compose the specified transformation with the current position and orientation of the relative coordinate system. This effectively applies the transformation to the current position and orientation of the relative coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalRestore

## Synopsis

Restore a camera or 3D draw calibration context from disk.

## Syntax

```
AIL_ID McalRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *CalibrationIdPtr     //out
)
```

## Description

This function restores a camera or 3D draw calibration context that was previously saved to a file, using McalSave() or McalStream(), and assigns it an Aurora Imaging Library identifier.

This function can also restore the camera calibration context associated with an Aurora Imaging Library image buffer (MIM) that was previously saved to a file, using MbufExport() with M_AIL_TIFF + M_WITH_CALIBRATION, and assign the context an Aurora Imaging Library identifier. If you then associate this restored camera calibration context to the restored image buffer, the calibrated image will be in the same state as the previously saved image (at the moment it was saved). For more information, see the Saving and reloading a calibrated image subsection of the Camera calibration propagation section of Chapter 28: Calibrating your camera setup.

When the restored camera or 3D draw calibration context, is no longer required, release it using McalFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the camera or 3D draw calibration context's lifetime and you must not manually free it.

## Parameters

FileName

Specifies the name and path of the file from which to restore the camera or 3D draw calibration context. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the file name and path | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_INTERACTIVE | [This is only applicable to Windows]<br><br>Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*FileName*") 1 | Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, camera and 3D draw calibration context files have an MCA extension.<br><br>To specify a file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*"). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the camera or 3D draw calibration context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT_HOST | Specifies the default Host system of the current Aurora Imaging Library application. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *System identifier* | Specifies a valid system identifier, previously allocated using MsysAlloc(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

### ControlFlag

Specifies the function's control flag. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the function's control flag | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Sets the function's control flag to the default. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

### CalibrationIdPtr

Specifies the address of the variable in which to write a camera or 3D draw calibration context identifier or specifies the data type that the function should use to return the camera or 3D draw calibration context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For retrieving the identifier or specifying how to return it | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NULL | Specifies that you will use this function's return value to obtain the identifier of the allocated camera calibration context, or 3D draw calibration context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNIQUE_ID | Specifies that you will use this function's return value to obtain the identifier of the allocated camera calibration context, or 3D draw calibration context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_CAL_ID* is returned instead of a standard Aurora Imaging Library identifier.<br>> This setting is only available when using C++11 (or later).<br>An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the camera calibration context, or 3D draw calibration context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Address in which to write the 3D draw calibration context identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the restored 3D draw calibration context.<br>If allocation fails, **M_NULL** is written as the identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Address in which to write the camera calibration context identifier* | Specifies the address of an *AIL_ID* in which to write the identifier of the restored camera calibration context.<br>If allocation fails, **M_NULL** is written as the identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

### Return value

The returned value is the camera calibration context, or 3D draw calibration context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_CAL_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

### Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalSave

## Synopsis

Save a camera or 3D draw calibration context to a file.

## Syntax

```
void McalSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID CalibrationId,        //in
    AIL_INT64 ControlFlag        //in
)
```

## Description

This function saves a camera or 3D draw calibration context to a file. This information can be reloaded, using McalRestore() or McalStream().

## Parameters

FileName

Specifies the name and path of the file in which to save the camera or 3D draw calibration context. It is recommended that you use the MCA file extension for easier use with other Aurora Imaging software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the file name and path | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_INTERACTIVE | [This is only applicable to Windows]<br><br>Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*FileName*") 1 | Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, camera and 3D draw calibration context files have an MCA extension.<br><br>To save the file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*"). | c | | e | f | g | a | | i | | j | k | l | m | n | b |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

CalibrationId

Specifies the camera or 3D draw calibration context to save.

ControlFlag

Specifies the function's control flag. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the function's control flag | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Sets the function's control flag to the default. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalSetCoordinateSystem

## Synopsis

Change the position and orientation of a coordinate system.

## Syntax

```
void McalSetCoordinateSystem(
    AIL_ID CalibratedAilObjectId,        //out
    AIL_INT64 TargetCoordinateSystem,    //in
    AIL_INT64 ReferenceCoordinateSystem, //in
    AIL_INT64 TransformType,             //in
    AIL_ID ArrayBufOrMatrix3dgeoId,      //in
    AIL_DOUBLE Param1,                   //in
    AIL_DOUBLE Param2,                   //in
    AIL_DOUBLE Param3,                   //in
    AIL_DOUBLE Param4                    //in
)
```

## Description

This function moves a specified (target) coordinate system in relation to a specified (reference) coordinate system. For example, you can move the relative coordinate system by (0, 6, 0) in the absolute coordinate system. This will displace the origin of the relative coordinate system by 6 units along the Y-axis of the absolute coordinate system. You can call this function to move the relative coordinate system of a 3D-based camera calibration context, calibrated image, or 3D reconstruction result buffer of type M_POINT_CLOUD_RESULT. Note that to return the position and orientation of one coordinate system as a transformation of another coordinate system, you can use McalGetCoordinateSystem().

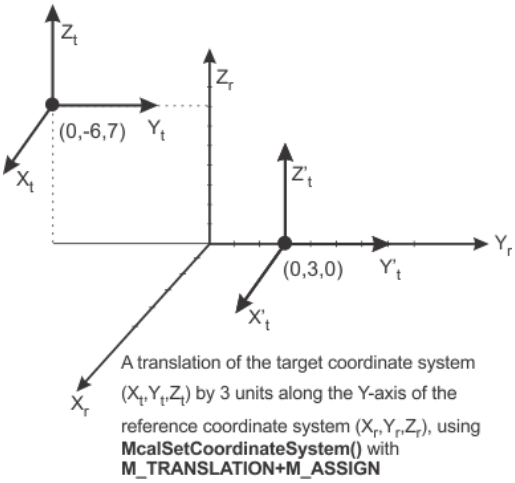
If you are moving the relative coordinate system of an image that has been corrected, the XY-plane of the relative coordinate system should not be moved in the Z-direction and should not be rotated around its X or Y-axis. In other words, the relative coordinate system of a corrected image can only be translated along its X or Y-axis or rotated around its Z-axis. Moving the relative coordinate system of an image is useful for analyzing an object using a temporary local coordinate system. You can also use McalFixture() to move the relative coordinate system with respect to a result. If you move the relative coordinate system, results returned in world units from other modules are returned with respect to the relative coordinate system's new position, and settings which accept input in world units will accept that input with respect to the relative coordinate system's new position.

If you transform the camera or tool coordinate system, Aurora Imaging Library assumes that the camera has been moved in your camera setup, so it adjusts the camera calibration mapping between the absolute coordinate system and the pixel coordinate system.

You can also use this function to transform the relative coordinate system of a 3D reconstruction result buffer of type M_POINT_CLOUD_RESULT, which M3dmapGetResult() uses to express world coordinates. To transform the relative coordinate system of a result buffer, TargetCoordinateSystem must be set to M_RELATIVE_COORDINATE_SYSTEM and ReferenceCoordinateSystem must be set to M_RELATIVE_COORDINATE_SYSTEM or M_ABSOLUTE_COORDINATE_SYSTEM.

Note that all angles should be given in degrees. However, unlike most other Aurora Imaging Library functions (including McalRelativeOrigin()), angles are interpreted using the right-hand grip rule around the axis of rotation; if you wrap your right hand around the axis of rotation, pointing your thumb in the positive direction of the axis, your fingers wrap in the direction of rotation. For example, a positive rotation around the Z-axis corresponds to a rotation that turns the positive X-axis toward the positive Y-axis.

You can specify whether the transformation of the target coordinate system is applied from the current position (M_COMPOSE_WITH_CURRENT) of the target coordinate system or from the origin of the reference coordinate system (M_ASSIGN).




> Note that you cannot move the pixel coordinate system nor the absolute coordinate system.

If you adjust the coordinate system of a calibrated image associated with an M_VECTOR_AND_RASTER ROI, the raster information will be discarded, causing the ROI to become an M_VECTOR ROI. See MbufSetRegion() for more information.

## Parameters

CalibratedAilObjectId

Specifies the identifier of a 3D-based camera calibration context, 3D reconstruction result buffer of type M_POINT_CLOUD_RESULT, or a calibrated image.

The image buffer must be a 1- or 3-band buffer.

> Note that the 3D-based camera calibration context must have been allocated using McalAlloc() with M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED. You cannot specify a 2D-based camera calibration context.

TargetCoordinateSystem

Specifies the coordinate system on which to apply the transformation. This parameter can be set to one of the following.

> If CalibratedAilObjectId is set to a 3D reconstruction result buffer (must be of type M_POINT_CLOUD_RESULT) or a calibrated image, TargetCoordinateSystem must be set to M_RELATIVE_COORDINATE_SYSTEM.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the target coordinate system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_CAMERA_COORDINATE_SYSTEM | Specifies to apply the transformation to the camera coordinate system. The origin of the camera coordinate system corresponds to the effective pinhole of the modeled camera and the Z-axis of the camera coordinate system points in the direction that the camera is facing.<br><br>By default, for an M_TSAI_BASED, M_ZHANG_BASED, or M_MOVING_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED, a rigid link exists between the tool and camera coordinate systems such that moving one automatically moves the other accordingly. This can be temporarily broken using McalControl() with M_LINK_TOOL_AND_HEAD.<br><br>This coordinate system can only be moved when CalibratedAilObjectId is set to a 3D-based camera calibration context (McalAlloc() with M_TSAI_BASED, M_ZHANG_BASED, or M_MOVING_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED) that you have successfully calibrated, or after using **McalSetCoordinateSystem**() with M_ASSIGN. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_COORDINATE_SYSTEM | Specifies to apply the transformation to the relative coordinate system. The relative coordinate system defines the world plane in which results are measured. By default, it corresponds to the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROBOT_BASE_COORDINATE_SYSTEM | Specifies to apply the transformation to the robot base coordinate system. The origin of this coordinate system is positioned at the base of the robot holding the camera. This coordinate system is defined as the origin of the robot encoders. If the encoders of the robot all indicate 0, it means that the tool coordinate system is at the same position and orientation as the robot base coordinate system.<br><br>This coordinate system can only be moved when CalibratedAilObjectId is set to a camera calibration context of type M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED that you have successfully calibrated, or after using **McalSetCoordinateSystem**() with M_ASSIGN.<br><br>The only way to change the position and orientation of this coordinate system is when you reset the encoders on a mobile robot. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_COORDINATE_SYSTEM | Specifies to apply the transformation to the tool coordinate system. The tool coordinate system is used to position the camera. Though this coordinate system can be used to move the camera, it need not be associated with the real camera position. By default, its axes are parallel to the absolute coordinate system, and its origin is the same as that of the absolute coordinate system.<br><br>By default, for an M_TSAI_BASED, M_ZHANG_BASED, or M_MOVING_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED, a rigid link exists between the tool and camera coordinate systems such that moving one automatically moves the other accordingly. This can be temporarily broken using McalControl() with M_LINK_TOOL_AND_HEAD.<br><br>This coordinate system can only be moved when CalibratedAilObjectId is set to a 3D-based camera calibration context (McalAlloc() with M_TSAI_BASED, M_ZHANG_BASED, or M_MOVING_CAMERA type of M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED) that you have successfully calibrated, or after using **McalSetCoordinateSystem**() with M_ASSIGN. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ReferenceCoordinateSystem

Specifies the reference coordinate system. The reference coordinate system must be defined before calling this function. This

parameter can be set to one of the following values.

> If your CalibratedAilObjectId is a 3D reconstruction result buffer (must be of type M_POINT_CLOUD_RESULT), ReferenceCoordinateSystem must be set to M_RELATIVE_COORDINATE_SYSTEM or M_ABSOLUTE_COORDINATE_SYSTEM.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the reference coordinate system | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_ABSOLUTE_COORDINATE_SYSTEM. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ABSOLUTE_COORDINATE_SYSTEM | Specifies to use the absolute coordinate system as a reference coordinate system for the transformation. The absolute coordinate system is unmovable and is the coordinate system from which all other world coordinate systems are defined. By default, the origin of the absolute coordinate system corresponds to the center of the top-left calibration point when using McalGrid(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CAMERA_COORDINATE_SYSTEM | Specifies to use the camera coordinate system as a reference coordinate system for the transformation. The camera coordinate system's origin corresponds to the effective pinhole of the modeled camera and its Z-axis points in the direction that the camera is facing. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_COORDINATE_SYSTEM | Specifies to use the relative coordinate system as a reference coordinate system for the transformation. The relative coordinate system defines the world plane in which results are returned. By default, it corresponds to the absolute coordinate system. The relative coordinate system can be recentered and/or re-oriented using either **McalSetCoordinateSystem**() or McalRelativeOrigin(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROBOT_BASE_COORDINATE_SYSTEM | Specifies to use the robot base coordinate system as a reference coordinate system for the transformation. This coordinate system is defined as the origin of the robot encoders. If the encoders of the robot all indicate 0, it means that the tool coordinate system is at the same position and orientation as the robot base coordinate system.<br><br>This coordinate system can only used as a reference when CalibratedAilObjectId is set to an M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED that you have successfully calibrated, or after using **McalSetCoordinateSystem**() with M_ASSIGN.<br><br>The position and orientation of this coordinate system are only changed when resetting the encoders on a mobile robot. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_COORDINATE_SYSTEM | Specifies to use the tool coordinate system as a reference coordinate system for the transformation. The tool coordinate system is used to position the camera. By default, its axes are parallel to the absolute coordinate system, and its origin is the same as that of the absolute coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

TransformType

Specifies the type of transformation to apply to the target coordinate system.

See the Parameter associations section for possible values that can be specified.

ArrayBufOrMatrix3dgeoId

Specifies the identifier of an Aurora Imaging Library array buffer that contains the transformation matrix to use, or specifies the identifier of a transformation matrix object. The expected transformation matrix is dependent on the type of transformation selected.

The Aurora Imaging Library array buffer must be a 32-bit floating-point buffer, allocated using MbufAlloc2d() with M_ARRAY. The required buffer size is dependent on the type of transformation selected.

The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

> If using a transformation matrix, it must represent a rotation and/or translation, nothing else (for example, it cannot represent a scale); otherwise, an error is reported.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies an attribute of the transformation. Its definition is dependent on the type of transformation selected.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the transformation. Its definition is dependent on the type of transformation selected.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies an attribute of the transformation. Its definition is dependent on the type of transformation selected.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies an attribute of the transformation. Its definition is dependent on the type of transformation selected.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

## Parameter associations

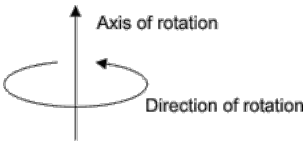
The table below lists possible values for the **TransformType**, **ArrayBufOrMatrix3dgeoId**, **Param1**, **Param2**, **Param3**, and **Param4** parameters.

- For specifying the transformation type

Note that if Param1, Param2, Param3, or Param4 are not required by the transformation, they should be set to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the transformation type | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| TransformType | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| ArrayBufOrMatrix3dgeoId | | | | | | | | | | | | | | | | |
| Param1 | | | | | | | | | | | | | | | | |
| Param2 | | | | | | | | | | | | | | | | |
| Param3 | | | | | | | | | | | | | | | | |
| Param4 | | | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_HOMOGENEOUS_MATRIX + | Specifies to apply a translation, a rotation, or both to the target coordinate system. Specify the transformation using a 4x4 homogenous matrix.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | | | | | | | | | | | | | | |
| **ArrayBufOrMatrix3dgeoId** | Specifies the identifier of the Aurora Imaging Library array buffer or the transformation matrix object containing the matrix representation of the rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_IDENTITY + | Specifies to apply the identity transformation to the target coordinate system. When used with the M_ASSIGN combination constant, this operation transforms the target coordinate system so that it has the same position and orientation as the reference coordinate system.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_AXIS_ANGLE + | Specifies to apply a rotation operation described by an axis and angle of rotation. The axis of rotation is defined by a vector. The angle of rotation is measured in the counter-clockwise direction around the axis of rotation.<br>  <br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the X-component of the vector that defines the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Y-component of the vector that defines the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the Z-component of the vector that defines the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param4** | Specifies the angle of the rotation, in degrees, around the axis of rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_AXIS_X + | Specifies to apply a rotation operation on the target coordinate system which corresponds to the rotation that aligns the X-axis of the reference coordinate system with the specified vector.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the X-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Param2** | Specifies the Y-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the Z-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_AXIS_Y + | Specifies to apply a rotation operation on the target coordinate system which corresponds to the rotation that aligns the Y-axis of the reference coordinate system with the specified vector.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the X-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Y-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the Z-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_AXIS_Z + | Specifies to apply a rotation operation on the target coordinate system which corresponds to the rotation that aligns the Z-axis of the reference coordinate system with the specified vector.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the X-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Y-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the Z-component of the vector. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_MATRIX + | Specifies to apply a rotation operation that is described by a 3x3 rotation matrix.<br><br>Note that, if using a transformation matrix, the translation part of the matrix is ignored (no error is reported if it contains a translation).<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | Specifies the identifier of the Aurora Imaging Library array buffer or the transformation matrix object containing the matrix representation of the rotation. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_QUATERNION + | Specifies to apply a rotation operation that is described by a rotation quaternion.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the scalar component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | **Param2** | Specifies the X-component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param3** | Specifies the Y-component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param4** | Specifies the Z-component of the quaternion. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_X + | | Specifies to apply a rotation operation that is described by a rotation around the X-axis.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param1** | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_XYZ + | | Specifies to apply a rotation operation that is described by three distinct rotations about the axes of the reference coordinate system in the following order: a rotation about the X-axis, a rotation about the Y-axis, and a rotation about Z-axis rotation.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param1** | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param2** | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param3** | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_XZY + | | Specifies to apply a rotation operation that is described by three distinct rotations about the axes of the reference coordinate system in the following order: a rotation about the X-axis, a rotation about the Z-axis, and a rotation about the Y-axis rotation.<br><br>You must specify a combination value from the following table:<br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param1** | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param2** | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | **Param3** | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_ROTATION_Y + | Specifies to apply a rotation operation that is described by a rotation about the Y-axis.<br><br>You must specify a combination value from the following table:<br><br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_YXZ + | Specifies to apply a rotation operation that is described by three distinct rotations about the axes of the reference coordinate system in the following order: a rotation about the Y-axis, a rotation about the X-axis, and a rotation about Z-axis rotation. This is also known as roll-pitch-yaw rotation.<br><br>You must specify a combination value from the following table:<br><br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_YZX + | Specifies to apply a rotation operation that is described by three distinct rotations about the axes of the reference coordinate system in the following order: a rotation about the Y-axis, a rotation about the Z-axis, and a rotation about the X-axis rotation.<br><br>You must specify a combination value from the following table:<br><br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_Z + | Specifies to apply a rotation operation that is described by a rotation about the Z-axis.<br><br>You must specify a combination value from the following table:<br><br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Param1** | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_ZXY + | Specifies to apply a rotation operation that is described by three distinct rotations about the axes of the reference coordinate system in the following order: a rotation about the Z-axis, a rotation about the X-axis, and a rotation about the Y-axis rotation.<br><br>You must specify a combination value from the following table:<br><br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROTATION_ZYX + | Specifies to apply a rotation operation that is described by three distinct rotations about the axes of the reference coordinate system in the following order: a rotation about the Z-axis, a rotation about the Y-axis, and a rotation about the X-axis rotation.<br><br>You must specify a combination value from the following table:<br><br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the Z-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the Y-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the X-axis rotation, in degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TRANSLATION + | Specifies to apply a translation operation along each of the reference coordinate system's axes.<br><br>You must specify a combination value from the following table:<br><br>• To specify how to apply the transformation | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ArrayBufOrMatrix3dgeoId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param1** | Specifies the displacement along the X-axis of the reference coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param2** | Specifies the displacement along the Y-axis of the reference coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Param3** | Specifies the displacement along the Z-axis of the reference coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for the values listed in For specifying the transformation type.

You must add one of the following values to the above-mentioned values to specify how to apply the transformation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying how to apply the transformation | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_ASSIGN | Specifies to assign the specified transformation to the target coordinate system from the origin and orientation of the reference coordinate system.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_COMPOSE_WITH_CURRENT | Specifies to compose the specified transformation with the current position and orientation in the reference coordinate system. This effectively applies the transformation to the current position and orientation of the target coordinate system. Note that this constant cannot be used with an undefined coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalStream

## Synopsis

Load, restore, or save a camera or 3D draw calibration context from/to a file or a memory stream.

## Syntax

```
void McalStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextCalIdPtr,       //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

## Description

This function can load, restore, or save a camera or 3D draw calibration context from/to a file or memory stream.

To inquire the number of bytes necessary to save a camera or 3D draw calibration context to memory stream, you should first call this function (**McalStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with McalSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

Using **McalStream**(), you can choose to save a backwards-compatible version of the camera calibration context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore camera calibration contexts saved using Aurora Imaging Library version 7.0 or above. Settings that do not exist in the lower version will be filled with default values when the camera calibration context is loaded or restored.

## Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the camera or 3D draw calibration context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the camera or 3D draw calibration context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the camera or 3D draw calibration context. This parameter must

be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the stream type | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_FILE | Specifies a file stream. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_MEMORY | Specifies a memory stream. You are responsible for allocating a block of memory for the stream. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Version

Specifies the Aurora Imaging Library version of the camera or 3D draw calibration context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ContextCalIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the camera or 3D draw calibration context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the camera or 3D draw calibration context, in bytes. If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of a camera or 3D draw calibration context will vary depending on the Aurora Imaging Library version specified.

## Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextCalIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For performing the stream operation. | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Operation | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| MemPtrOrFileName<br>- Possible values passed or returned | | | | | | | | | | | | | | | | |
| SysId | | | | | | | | | | | | | | | | |
| Version | | | | | | | | | | | | | | | | |
| ContextCalIdPtr<br>- Possible values passed or returned | | | | | | | | | | | | | | | | |
| M_INQUIRE_SIZE_BYTE | Inquires the number of bytes required to save a camera or 3D draw calibration context to a memory stream. This operation is not supported when the StreamType parameter is set to M_FILE. | | | | | | | | | | | | | | | |
| **MemPtrOrFileName** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **SysId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Version** | Specifies the Aurora Imaging Library version with which the camera or 3D draw calibration context must be compatible when saved. The size of a camera or 3D draw calibration context might differ from one Aurora Imaging Library version to another.<br><br>The earliest version in which the 3D draw calibration context can be used is Aurora Imaging Library 10 Service Pack 4. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT | Specifies the current version of Aurora Imaging Library. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PROC_VERSION_100_SP4 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PROC_VERSION_100_SP5 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PROC_VERSION_100_SP6 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PROC_VERSION_100_SP7 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PROC_VERSION_110 | Specifies the version as being Aurora Imaging Library 11.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ContextCalIdPtr** | Specifies the address of the variable containing the identifier of the camera or 3D draw calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LOAD | Loads the content of a specified file or memory stream into a previously allocated camera or 3D draw calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **MemPtrOrFileName** | Specifies the file or memory stream from which to load the context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INTERACTIVE | [This is only applicable to Windows]<br><br>Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*FileName*") 1 | Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Camera and 3D draw calibration contexts typically have an MCA file extension. The function handles (internally) the opening and closing of the file.<br><br>To specify a file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*"). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *MemPtr* | Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object.<br><br>Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **SysId** | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Version** | This parameter must be set to **M_DEFAULT**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ContextCalIdPtr** | Specifies the address of the variable containing the identifier of the camera or 3D draw calibration context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RESTORE | Restores a camera or 3D draw calibration context from a file or memory stream and assigns it an Aurora Imaging Library identifier. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **MemPtrOrFileName** | Specifies the file or memory stream from which to restore the context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_INTERACTIVE | [This is only applicable to Windows]<br><br>Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| AIL_TEXT("*FileName*") 1 | Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Camera and 3D draw calibration contexts typically have an MCA file extension. The function handles (internally) the opening and closing of the file.<br><br>To specify a file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*"). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *MemPtr* | Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object.<br><br>Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **SysId** | Specifies the system on which to allocate the camera or 3D draw calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_DEFAULT_HOST | Specifies the default Host system of the current Aurora Imaging Library application. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *System identifier* | Specifies a valid system identifier, previously allocated using MsysAlloc(). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Version** | This parameter must be set to **M_DEFAULT**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ContextCalIdPtr** | Specifies the address of the variable in which to write the identifier of the camera or 3D draw calibration context. If the operation is not successful, **M_NULL** is returned. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_SAVE | Saves a camera or 3D draw calibration context to a specified file or memory stream. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **MemPtrOrFileName** | Specifies the file or memory stream. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | M_INTERACTIVE | [This is only applicable to Windows]<br><br>Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | AIL_TEXT("*FileName*") 1 | Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Zebra Aurora software products, when saving a calibration context to a file, use the MCA file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten.<br><br>To specify a file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*"). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | *MemPtr* | Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE.<br><br>Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **SysId** | | This parameter must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **Version** | | Specifies the Aurora Imaging Library version with which the camera or 3D draw calibration context must be compatible when saved. Note that the camera calibration context can only be up to one major release older than the current version.<br><br>The earliest version in which the 3D draw calibration context can be used is Aurora Imaging Library 10 Service Pack 4. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_DEFAULT | Specifies the current version of Aurora Imaging Library. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_PROC_VERSION_100_SP4 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_PROC_VERSION_100_SP5 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_PROC_VERSION_100_SP6 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_PROC_VERSION_100_SP7 | Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| | M_PROC_VERSION_110 | Specifies the version as being Aurora Imaging Library 11.0. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| **ContextCalIdPtr** | | Specifies the address of the variable containing the identifier of the camera or 3D draw calibration context to save. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalTransformCoordinate

## Synopsis

Convert coordinates between their world and pixel values.

## Syntax

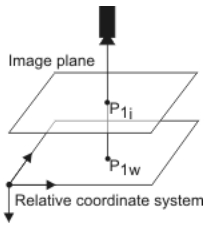
```
void McalTransformCoordinate(
    AIL_ID CalibrationOrImageId, //in
    AIL_INT64 TransformType,     //in
    AIL_DOUBLE X,                //in
    AIL_DOUBLE Y,                //in
    AIL_DOUBLE *ResXPtr,         //out
    AIL_DOUBLE *ResYPtr          //out
)
```

## Description

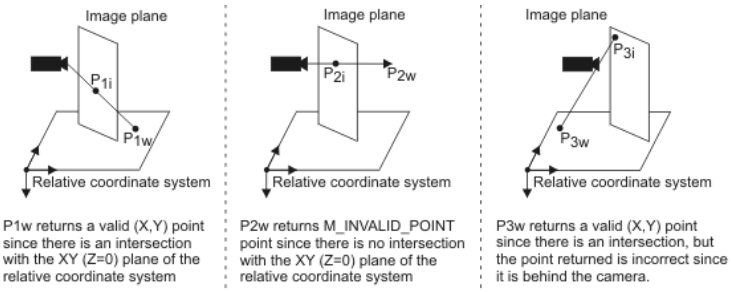
This function converts a pair of coordinates from their pixel value to their world value (or vice versa). The conversion can be performed according to a camera calibration context, calibrated image, or corrected image.

> Note that, if you changed the origin and/or orientation of the relative coordinate system (using McalRelativeOrigin() or McalSetCoordinateSystem()), world coordinates will be returned, or assumed to be given, with respect to this relative coordinate system.

This function converts the coordinates of a point by making a line connecting the center of the camera's lens with the point provided, and then finding the intersection of that line with the required plane. To convert an image pixel to a world point, this function defines a line connecting the center of the camera's lens with the image plane, and then returns the intersection of this line with the world plane.




However, if the image plane is not parallel to the relative coordinate system, due to the camera setup or a displacement of the relative coordinate system, not every point in the image plane will have a valid real-world equivalent. Three types of intersections can occur when transforming from the image plane to the world plane. If the specified point in the image plane corresponds to a point in front of the camera and in the relative coordinate system, that point in the relative coordinate system is returned. However, if the line traced through the specified point in the image plane does not intersect the XY (Z=0) plane of the relative coordinate system, **M_INVALID_POINT** is returned since there is no intersection. If the line's intersection with the XY (Z=0) plane of the relative coordinate system is behind the camera, then the mathematically computed value is returned, even though it is not the correct world location of the point specified in the image.




However, using M_NO_POINTS_BEHIND_CAMERA returns **M_INVALID_POINT** if the returned point is behind the camera.

## Parameters

CalibrationOrImageId

Specifies the identifier of the camera calibration context, calibrated image, or corrected image. When an image is specified, the

transformation uses the camera calibration information associated with this image. The image buffer must be a 1- or 3-band buffer.

> If you are transforming coordinates obtained from an image, pass the identifier of that image. For example, for coordinates originating from a child buffer, pass the child buffer's identifier.

TransformType

Specifies whether to perform a pixel-to-world or world-to-pixel conversion. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying pixel-to-world or world-to-pixel | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_PIXEL_TO_WORLD + | Converts from pixel to world. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_TO_PIXEL + | Converts from world to pixel. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for any of the possible values of the TransformType parameter.

You can add one of the following values to the above-mentioned values to specify the return of invalid points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying to return invalid points | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NO_EXTRAPOLATED_POINTS | Specifies that if a pixel involved in the transformation is not inside the calibrated region, **M_INVALID_POINT** will be returned, instead of a coordinate resulting from the extrapolation. The calibrated region is defined as the image region covered by the camera calibration grid. This region can be displayed by calling McalDraw() with M_DRAW_VALID_REGION.<br>> This combination value only applies to piecewise linear camera calibrations (M_LINEAR_INTERPOLATION); if this combination value is specified and the object passed to CalibrationOrImageId has any other type of camera calibration, this combination value is ignored. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NO_POINTS_BEHIND_CAMERA | Specifies that **M_INVALID_POINT** is returned when a computed point is mathematically valid but physically impossible (behind the camera).<br>> If this value does not apply to the specified transformation type, then it is ignored. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

X

Specifies the X-coordinate of the input.

Y

Specifies the Y-coordinate of the input.

ResXPtr

Specifies the address of the variable in which to place the returned value of the X-coordinate. One of the following can be written to the address.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the potential return values of the X-coordinate |
|---|

| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_INVALID_POINT | Specifies a non-valid coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the transformed X-coordinate as a value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ResYPtr

Specifies the addres of the variable in which to place the returned value of the Y-coordinate. One of the following can be written to the address.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the potential return values of the Y-coordinate | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_INVALID_POINT | Specifies a non-valid coordinate. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* | Specifies the transformed Y-coordinate as a value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalTransformCoordinate3dList

## Synopsis

Convert a list of coordinates, including 3D coordinates, between two coordinate systems.

## Syntax

```
void McalTransformCoordinate3dList(
    AIL_ID CalibrationOrImageId,         //in
    AIL_INT64 SrcCoordinateSystem,       //in
    AIL_INT64 DstCoordinateSystem,       //in
    AIL_INT NumPoints,                   //in
    const AIL_DOUBLE *SrcCoordXArrayPtr, //in
    const AIL_DOUBLE *SrcCoordYArrayPtr, //in
    const AIL_DOUBLE *SrcCoordZArrayPtr, //in
    AIL_DOUBLE *DstCoordXArrayPtr,       //out
    AIL_DOUBLE *DstCoordYArrayPtr,       //out
    AIL_DOUBLE *DstCoordZArrayPtr,       //out
    AIL_INT64 ModeFlag                   //in
)
```

## Description

This function converts a list of coordinates from a source coordinate system to a destination coordinate system, where coordinates of unit direction vectors and of depth maps are also accepted as inputs or outputs.

Using this function, you can convert between the following:

- Pixel coordinate system and world coordinate system.
- World coordinate system and any other world coordinate system.
- Fully corrected depth maps and world coordinate system.
- Pixel coordinate system and unit direction vectors in any world coordinate system.

> Note that conversion between unit direction vectors and the pixel coordinate system is supported, but conversion of direction vectors to depth maps or world coordinate systems is not. Also, if you pass a corrected image to **McalTransformCoordinate3dList**(), and you specify that the source points are expressed in the pixel coordinate system (M_PIXEL_COORDINATE_SYSTEM), the function assumes that the source points are expressed in the corrected pixel coordinate system.

To convert a point from the pixel coordinate system to its corresponding point in the specified world coordinate system, this function creates a line (simulating a light ray) from the effective pinhole of the camera through the specified point in the image plane, and then to the specified relative coordinate system (DstCoordinateSystem). Since all points along a given light ray project onto the same point in the image plane, every point along the ray is theoretically a solution. Therefore, to produce a unique solution, the point of intersection between the line and the XY-plane (Z=0) of the relative coordinate system is transformed to the specified world coordinate system. Then, the corresponding world point is returned. Alternatively, **McalTransformCoordinate3dList**() can return the X, Y, and Z-components of the unit direction vector corresponding to the direction of the calculated light ray as it intersects the source point (using M_UNIT_DIRECTION_VECTOR).

If the image plane is not parallel to the XY-plane (Z=0) of the relative coordinate system, due to the camera setup or a displacement of the relative coordinate system, not every point in the image plane will have a valid real-world equivalent. Three types of intersections can occur when transforming from the image plane to the world plane. If the specified point in the image plane corresponds to a point in front of the camera in the relative coordinate system, its corresponding position in the destination world coordinate system is returned. However, if the line traced through the specified point in the image plane does not intersect the XY-plane of the relative coordinate system, **M_INVALID_POINT** is returned. If the projected line's intersection with the XY-plane of the relative coordinate system is behind the camera, the mathematically computed value of this point of intersection is returned. Because this point of intersection is located behind the camera, its location in the relative coordinate system is considered as incorrect.

However, using M_NO_POINTS_BEHIND_CAMERA with **McalTransformCoordinate3dList**() returns **M_INVALID_POINT** if the returned point is behind the camera.

To convert a point specified in a world coordinate system to its corresponding point in the pixel coordinate system, the function transforms the specified point to the relative coordinate system and a line is formed (simulating a light ray) from the point to the effective pinhole of the camera. The function then returns the point of intersection of this line with the image plane. If the point given is behind the camera, the function will return the mathematically computed value. However, using M_NO_POINTS_BEHIND_CAMERA with **McalTransformCoordinate3dList**() will return **M_INVALID_POINT** instead of the mathematically computed value.

This function also supports the conversion of coordinates from a depth map to a specified world coordinate system and vice versa. To convert from a depth map to a world coordinate system, you must provide the depth map's X and Y coordinates and, optionally, the intensity value of the pixel. You can choose to omit the intensity value, in which case the function will obtain the value from the specified location in the depth map. If you choose to omit the intensity values, set SrcCoordZArrayPtr to **M_NULL**. The intensity value of the depth map point is linearly mapped to a corresponding Z-coordinate for the world coordinate system. Note, source pixel coordinates that are outside the depth map image will generate **M_INVALID_POINT**.

If the intensity of a depth map pixel is equal to the buffer's maximum intensity value, it is interpreted as missing data and cannot be converted. **M_INVALID_POINT** will be returned. Note, that this applies whether you choose to omit the intensity values or not.

To convert a point from a specified world coordinate system to a depth map, you must provide the X-, Y-, and Z-coordinates of the point. The function converts the X-, Y-, and Z-coordinates of the world coordinate system to X, Y and intensity values, respectively. The Z-coordinate of the world coordinate system is linearly mapped to a corresponding intensity value where the maximum Z-coordinate in the list is given the highest possible depth value and the minimum Z-coordinate is give the lowest possible depth value.

## Parameters

CalibrationOrImageId

Specifies the identifier of a camera calibration context, 3D reconstruction context, calibrated image, corrected image, or 3d reconstruction result buffer of type M_POINT_CLOUD_RESULT. When an identifier of an image buffer is specified, the transformation uses the camera calibration information associated with this image. The image buffer must be a 1- or 3-band buffer.

> If you are transforming coordinates obtained from an image, pass the identifier of that image. For example, for coordinates originating from a child buffer, pass the child buffer's identifier.

When the identifier is set to a 3D reconstruction context or an M_POINT_CLOUD_RESULT 3D reconstruction result buffer, only world-to-world coordinate system conversions are supported. Specifically, for a 3D reconstruction context, you can only convert absolute coordinates to laser coordinates, or vice versa. For an M_POINT_CLOUD_RESULT 3D reconstruction result buffer, you can only convert absolute coordinates to relative coordinates, or vice versa.

SrcCoordinateSystem

Specifies the coordinate system of the source coordinates. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of the source coordinate system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_ABSOLUTE_COORDINATE_SYSTEM | Specifies an implicit and fixed coordinate system from which all other world coordinate systems are defined.<br>> Note that for a 3D reconstruction context, you can only convert absolute coordinates to laser coordinates, and vice versa. For an M_POINT_CLOUD_RESULT 3D reconstruction result buffer identifier, you can only convert absolute coordinates to relative coordinates, and vice versa. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_CAMERA_COORDINATE_SYSTEM | Specifies the coordinate system whose origin corresponds to the effective pinhole of the camera and whose Z-axis points in the direction that the camera is facing. This coordinate system is only defined after a successful camera calibration in M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED mode. Alternatively, you can define this coordinate system using McalSetCoordinateSystem() with M_ASSIGN. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LASER_LINE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is on the first laser line at the point closest to the origin of the absolute coordinate system.<br>> Note that for a 3D reconstruction context, you can only convert laser coordinates to absolute coordinates, and vice versa. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PIXEL_COORDINATE_SYSTEM | Specifies the pixel coordinate system of the image passed to CalibrationOrImageId (corrected or uncorrected); if a camera calibration context is passed instead, the pixel coordinate system is of any image that has the same pixel-to-world mapping as the specified camera calibration context. The origin of this coordinate system, (0, 0), is at the center of the image's top-left pixel. This is a 2D coordinate system. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_COORDINATE_SYSTEM | Specifies the relative coordinate system. The XY-plane (Z=0) of the relative coordinate system defines the world plane in which results are measured. By default, it is positioned at the absolute coordinate system.<br>> Note that for an M_POINT_CLOUD_RESULT 3D reconstruction result buffer, you can only convert relative coordinates to absolute coordinates, and vice versa. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROBOT_BASE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is positioned at the base of the robot holding the camera. This coordinate system is only available for an M_3D_ROBOTICS or M_3D_ROBOTICS_ZHANG_BASED camera calibration context, calibrated image, or corrected image. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_COORDINATE_SYSTEM | Specifies the coordinate system used to position the camera. This coordinate system is not necessarily associated with the real camera position. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

DstCoordinateSystem

Specifies the coordinate system in which to return the coordinates. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of the destination coordinate system | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_ABSOLUTE_COORDINATE_SYSTEM | Specifies an implicit and fixed coordinate system from which all other world coordinate systems are defined. INFO INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CAMERA_COORDINATE_SYSTEM | Specifies the coordinate system whose origin corresponds to the effective pinhole of the camera and whose Z-axis points in the direction that the camera is facing. INFO INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_LASER_LINE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is on the first laser line at the point closest to the origin of the absolute coordinate system. INFO INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_PIXEL_COORDINATE_SYSTEM | Specifies the pixel coordinate system of the image passed to CalibrationOrImageId (corrected or uncorrected); if a camera calibration context is passed instead, the pixel coordinate system is of any image that has the same pixel-to-world mapping as the specified camera calibration context. INFO INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_RELATIVE_COORDINATE_SYSTEM | Specifies the relative coordinate system. INFO INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_ROBOT_BASE_COORDINATE_SYSTEM | Specifies the coordinate system whose origin is positioned at the base of the robot holding the camera. INFO INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_TOOL_COORDINATE_SYSTEM | Specifies the coordinate system used to position the camera. INFO INFO | c | | e | f | g | a | | i | | j | k | l | m | n | b |

NumPoints

Specifies the number of points to convert.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcCoordXArrayPtr, SrcCoordYArrayPtr, or SrcCoordZArrayPtr parameter.

SrcCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array containing the X-coordinates of the source points. These coordinates must be expressed in the source coordinate system.

SrcCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array containing the Y-coordinates of the source points. These coordinates must be expressed in the source coordinate system.

SrcCoordZArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array containing the Z-coordinates of the source points. These coordinates must be expressed in the source coordinate system.

This parameter must be set to **M_NULL** if the source coordinate system is set to M_PIXEL_COORDINATE_SYSTEM. If the source coordinate system is a world coordinate system and the camera calibration information associated to CalibrationOrImageId is using a 2D camera calibration mode, this parameter must either be set to **M_NULL** or all the Z-coordinates must be set to 0.

DstCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to return the X-coordinates of the resulting points or the X-components of the calculated unit direction vectors, depending on the transformation mode (ModeFlag) . These coordinates are expressed in the destination coordinate system.

If one of the source points cannot be converted, the entry in the destination array corresponding to such point is set to **M_INVALID_POINT**.

DstCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to return the Y-coordinates of the resulting points or the Y-components of the calculated unit direction vectors, depending on the transformation mode (ModeFlag) . These coordinates are expressed in the destination coordinate system.

If one of the source points cannot be converted, the entry in the destination array corresponding to such point is set to **M_INVALID_POINT**.

DstCoordZArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to return the Z-coordinates of the resulting points or the Z-components of the calculated unit direction vectors, depending on the transformation mode (ModeFlag) . These coordinates are expressed in the destination coordinate system.

If one of the source points cannot be converted, the entry in the destination array corresponding to such point is set to **M_INVALID_POINT**.

This parameter must be set to **M_NULL** if the destination coordinate system is set to M_PIXEL_COORDINATE_SYSTEM.

ModeFlag

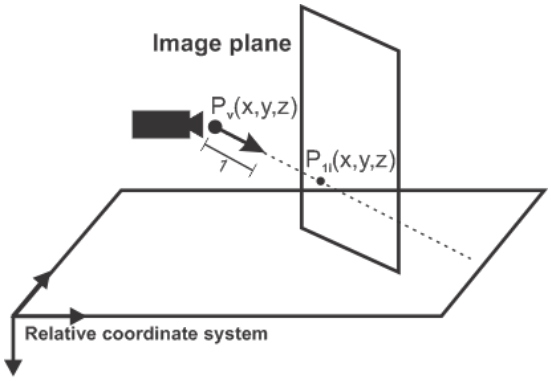
Specifies the mode of transformation and how to deal with invalid points.

You can set the ModeFlag parameter to one of the following values to specify the mode of transformation.

> Note that if a 3D reconstruction context or an M_POINT_CLOUD_RESULT 3D reconstruction result buffer is specified with CalibrationOrImageId, the ModeFlag parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the mode of transformation | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Specifies the default mode of transformation.<br><br>When converting world points to points in the pixel coordinate system, this function makes a line (simulating a light ray) from the effective pinhole of the camera to the point in the world coordinate system, and then returns the intersection of this line with the image plane; there is no ambiguity.<br><br>When converting world points from one world coordinate system to another, a matrix multiplication is used. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DEPTH_MAP + | Specifies that coordinates expressed in the pixel coordinate system, whether source or destination coordinates, are interpreted as depth map coordinates. The X- and Y-coordinates describe the pixels in the image, while the Z-coordinate parameter describes the intensity (gray level) of the pixels.<br><br>To convert from a depth map using M_DEPTH_MAP, CalibrationOrImageId must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer and must contain a fully corrected depth map. The 3D Image Processing module is capable of generating these fully corrected depth maps.<br><br>> This value cannot be used if both SrcCoordinateSystem and DstCoordinateSystem are world coordinate systems. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_UNIT_DIRECTION_VECTOR + | Specifies that points expressed in a world coordinate system, whether they are source or destination coordinates, are interpreted as unit direction vectors. Each vector originates at the camera's effective pinhole and points towards the specified point in the image plane.<br><br>This transformation mode is only available:<br><br>• After a successful camera calibration in M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED mode.<br><br>• When converting points in the pixel coordinate system to unit direction vectors in any world coordinate system, and vice versa.<br><br>When SrcCoordinateSystem is set to M_PIXEL_COORDINATE_SYSTEM, unit direction vectors are generated for each point specified in the image plane. When DstCoordinateSystem is set to M_PIXEL_COORDINATE_SYSTEM, points in the image plane corresponding to the specified unit direction vectors are returned.<br><br>  <br><br>**Using McalTransformCoordinate3dList with M_UNIT_DIRECTION_VECTOR, points in the pixel coordinate system are mapped to** | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values that can be used alone or as combination values for M_DEPTH_MAP; M_UNIT_DIRECTION_VECTOR.

You can use one or more of the following values in combination with each other or with the above-mentioned values, to specify when to return **M_INVALID_POINT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying when to return M_INVALID_POINT | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_NO_EXTRAPOLATED_POINTS | Specifies that if a pixel involved in the transformation is not inside the calibrated region, **M_INVALID_POINT** will be returned, instead of a coordinate resulting from the extrapolation. The calibrated region is defined as the image region covered by the camera calibration grid. This can be displayed by calling McalDraw() with M_DRAW_VALID_REGION.<br><br>> This value only applies to piecewise linear camera calibrations (M_LINEAR_INTERPOLATION); if it is set and the image has any other type of camera calibration, it is ignored. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NO_POINTS_BEHIND_CAMERA | Specifies that **M_INVALID_POINT** is returned when a computed point is mathematically valid but physically impossible (behind the camera).<br><br>> If this value does not apply to the specified transformation type, then it is ignored. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalTransformCoordinateList

## Synopsis

Convert a list of coordinates between their world and pixel values.

## Syntax

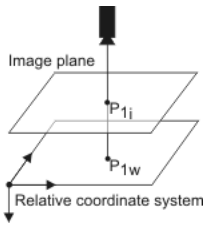
```
void McalTransformCoordinateList(
    AIL_ID CalibrationOrImageId,         //in
    AIL_INT64 TransformType,             //in
    AIL_INT NumPoints,                   //in
    const AIL_DOUBLE *SrcCoordXArrayPtr, //in
    const AIL_DOUBLE *SrcCoordYArrayPtr, //in
    AIL_DOUBLE *DstCoordXArrayPtr,       //out
    AIL_DOUBLE *DstCoordYArrayPtr        //out
)
```

## Description

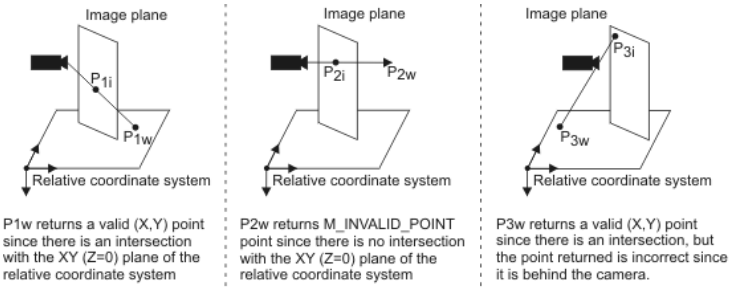
This function converts a list of coordinates from their pixel value to their world value (or vice versa). The conversion can be performed according to a camera calibration context, calibrated image, or corrected image.

> Note that, if you changed the origin and/or orientation of the relative coordinate system (using McalRelativeOrigin() or McalSetCoordinateSystem()), world coordinates will be returned, or assumed to be given, with respect to this relative coordinate system.

This function converts the coordinates of a point by making a line connecting the center of the camera's lens with the point provided, and then finding the intersection of that line with the required plane. To convert an image pixel to a world point, this function defines a line connecting the center of the camera's lens with the image plane, and then returns the intersection of this line with the world plane.




However, if the image plane is not parallel to the relative coordinate system, due to the camera setup or a displacement of the relative coordinate system, not every point in the image plane will have a valid real-world equivalent. Three types of intersections can occur when transforming from the image plane to the world plane. If the specified point in the image plane corresponds to a point in front of the camera and in the relative coordinate system, that point in the relative coordinate system is returned. However, if the line traced through the specified point in the image plane does not intersect the XY (Z=0) plane of the relative coordinate system, **M_INVALID_POINT** is returned since there is no intersection. If the line's intersection with the XY (Z=0) plane of the relative coordinate system is behind the camera, then the mathematically computed value is returned, even though it is not the correct world location of the point specified in the image.




However, using M_NO_POINTS_BEHIND_CAMERA returns **M_INVALID_POINT** if the returned point is behind the camera.

## Parameters

CalibrationOrImageId

Specifies the identifier of the camera calibration context, calibrated image, or corrected image. When an identifier of an image buffer is specified, the transformation uses the camera calibration information associated with this image. The image buffer must be a 1- or 3-band buffer.

> If you are transforming coordinates obtained from an image, pass the identifier of that image. For example, for coordinates originating from a child buffer, pass the child buffer's identifier.

TransformType

Specifies whether to perform a pixel-to-world or world-to-pixel conversion. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying pixel-to-world or world-to-pixel | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_PIXEL_TO_WORLD + | Converts from pixel to world. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_TO_PIXEL + | Converts from world to pixel. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for the values listed in For specifying pixel-to-world or world-to-pixel.

You can add the following value to the above-mentioned values to specify coordinates are packed.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying coordinates are packed | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_PACKED | Specifies that the source coordinates are passed in a packed format and/or that the result coordinates should be returned in a packed format depending on what you pass to the SrcCoordYArrayPtr and DstCoordYArrayPtr parameters.<br><br>If SrcCoordYArrayPtr is passed **M_NULL**, SrcCoordXArrayPtr is assumed to be a packed set of X- and Y-coordinates.<br><br>If DstCoordYArrayPtr is passed **M_NULL**, DstCoordXArrayPtr is filled with a packed set of X- and Y-coordinates.<br><br>If both SrcCoordYArrayPtr and DstCoordYArrayPtr are passed **M_NULL**, SrcCoordXArrayPtr is assumed to be a packed set of X- and Y-coordinates, and DstCoordXArrayPtr is filled with a packed set of X- and Y-coordinates. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for any of the possible values of the TransformType parameter.

You can add one of the following values to the above-mentioned values to set the return values of invalid points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying to return invalid points | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NO_EXTRAPOLATED_POINTS | Specifies that if a pixel involved in the transformation is not inside the calibrated region, **M_INVALID_POINT** will be returned, instead of a coordinate resulting from the extrapolation. The calibrated region is defined as the image region covered by the camera calibration grid. This can be displayed by calling McalDraw() with M_DRAW_VALID_REGION.<br><br>This value only applies to piecewise linear camera calibrations (M_LINEAR_INTERPOLATION); if it is set and the image has any other type of camera calibration, it is ignored. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_NO_POINTS_BEHIND_CAMERA | Specifies that **M_INVALID_POINT** is returned when a computed point is mathematically valid but physically impossible (behind the camera).<br><br>> If this value does not apply to the specified transformation type, then it is ignored. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

NumPoints

Specifies the number of points in the coordinate list.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcCoordXArrayPtr or SrcCoordYArrayPtr parameter.

SrcCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array of the input X-coordinates.

SrcCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array of the input Y-coordinates.

This parameter can be set to **M_NULL** when TransformType is set to M_PACKED. See the description of M_PACKED for more details.

DstCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array in which to place the output X-coordinates.

If M_NO_POINTS_BEHIND_CAMERA is used and one of the source points cannot be converted, the entry in the destination array corresponding to such point is set to **M_INVALID_POINT**.

DstCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE> ] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array in which to place the output Y-coordinates.

This parameter can be set to **M_NULL** when TransformType is set to M_PACKED. See the description of M_PACKED for more details.

If M_NO_POINTS_BEHIND_CAMERA is used and one of the source points cannot be converted, the entry in the destination array corresponding to such point is set to **M_INVALID_POINT**.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalTransformImage

## Synopsis

Physically transform an image by removing distortions, or create LUTs that can be used to do the same.

## Syntax

```
void McalTransformImage(
    AIL_ID SrcImageBufId,        //in
    AIL_ID DstImageOrLutId,      //out
    AIL_ID CalibrationId,        //in
    AIL_INT64 InterpolationMode, //in
    AIL_INT64 OperationType,     //in
    AIL_INT64 ControlFlag        //in
)
```

## Description

This function removes distortions in an image by physically transforming the image according to a specified camera calibration context. This function can also just extract the warping lookup tables (LUTs) that would be used to transform the image; you can then use these LUTs to transform images with the same distortions, using MimWarp().

Typically, the image is transformed such that:

- Its pixel coordinate system is aligned with its relative coordinate system.
- All the pixels in the destination image are square and represent the same size in world units.
- It is scaled and positioned in the destination image according to the specified fill mode.

After transformation, the image will be considered physically corrected if an M_FULL_CORRECTION operation was performed, and the fill mode scaled the image such that all its pixels are square and represent the same size in world units.

If you want to preserve the average pixel size of the source image, you must allocate your destination image buffer with an appropriate width and height. If you are performing an M_FIT type operation, you can use McalInquire() with M_TRANSFORM_FIT_SIZE_X_PRESERVE_PIXEL_SIZE and M_TRANSFORM_FIT_SIZE_Y_PRESERVE_PIXEL_SIZE to determine the width and height of your destination image buffer, in pixels, that will best preserve the source image's average pixel size. Likewise, if you are performing an M_CLIP type operation, you can use McalInquire() with M_TRANSFORM_CLIP_SIZE_X_PRESERVE_PIXEL_SIZE and M_TRANSFORM_CLIP_SIZE_Y_PRESERVE_PIXEL_SIZE to determine the width and height of your destination image buffer, in pixels, that will best preserve the source image's average pixel size.

## Parameters

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must be a 1- or 3-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageOrLutId

Specifies the identifier of the destination image buffer or LUT buffer.

This image buffer must be a 1- or 3-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

When transforming the image (M_WARP_IMAGE), set this parameter to the image buffer that has an appropriate size for the selected fill mode.

When extracting the X or Y warping LUT required for the image transformation (M_EXTRACT_LUT_X or M_EXTRACT_LUT_Y), set this parameter to a 32-bit signed, M_LUT buffer that has the same size as the destination image being passed to MimWarp().

While the transformed image retains the Aurora Imaging Library identifier of the camera calibration context associated with the source image, the transformed image itself has a uniform world-to-pixel mapping. Propagating the camera calibration information of the transformed image will pass both the identifier of the camera calibration context associated with the source image along with the uniform world-to-pixel mapping of the transformed image established during **McalTransformImage**().

CalibrationId

Specifies the camera calibration context with which to transform the image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the camera calibration context | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Specifies that the camera calibration context associated with the source image will be used.<br><br>If the source image is not associated with a camera calibration context, an Aurora Imaging Library error is returned. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Calibration context identifier* | Specifies the identifier of a valid camera calibration context, which you have calibrated using McalGrid(), McalList(), McalUniform(), or McalWarp().<br><br>If the source image is also associated with a camera calibration context, it will have precedence over the calibration context passed here, unless the calibration context identifiers differ and the source image does not have a constant pixel size. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

InterpolationMode

Specifies the interpolation mode to use when associating destination pixels with source points. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of interpolation to perform | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_NEAREST_NEIGHBOR + M_OVERSCAN_ENABLE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_BICUBIC + | Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_BILINEAR + | Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NEAREST_NEIGHBOR + | Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for any of the possible values of the InterpolationMode parameter.

You can add one of the following values to the above-mentioned values to specify how to determine the value of a destination pixel when its associated point falls outside the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying overscan | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_OVERSCAN_CLEAR | Sets the destination pixel to 0, if the associated point falls outside the source buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_OVERSCAN_DISABLE | Leaves the destination pixel as is, if the associated point falls outside the source buffer. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_OVERSCAN_ENABLE | Uses pixels from the source buffer's ancestor buffer, if the associated point falls outside the source buffer. If the source buffer is not a child buffer or if the associated point falls outside the ancestor buffer, it leaves the destination pixel as is.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_OVERSCAN_FAST | Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes.<br><br>Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

OperationType

Specifies the function's operation type. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the function's operation type | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_FULL_CORRECTION. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_CORRECT_LENS_DISTORTION_ONLY | Specifies a partial correction of the source image by only removing lens distortion, without modifying the perspective effect. In this case, the destination image is not calibrated. This operation is only supported for 3D-based camera calibration modes (M_TSAI_BASED, M_ZHANG_BASED, M_3D_ROBOTICS, or M_3D_ROBOTICS_ZHANG_BASED). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FULL_CORRECTION | Specifies a full correction of the source image. This corrects all distortions of the source image based on the provided camera calibration context. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ControlFlag

Specifies the function's control flag. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of result to output | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Same as M_WARP_IMAGE. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_EXTRACT_LUT_X + | Extracts the X warping LUT for the image transformation. You can use the extracted LUT to transform an image using MimWarp() with the OperationMode parameter to M_WARP_LUT + M_FIXED_POINT + 10. Since the LUTs are generated with 10 fractional bits, you must specify this number of fractional bits when you call MimWarp().<br><br>> Note, InterpolationMode must be set to M_DEFAULT. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_EXTRACT_LUT_Y + | Extracts the Y warping LUT for the image transformation. You can use the extracted LUT to transform an image using MimWarp() with the OperationMode parameter to M_WARP_LUT + M_FIXED_POINT + 10. Since the LUTs are generated with 10 fractional bits, you must specify this number of fractional bits when you call MimWarp().<br><br>Note, InterpolationMode must be set to M_DEFAULT. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WARP_IMAGE + | Transforms the source image and fits it into the destination image according to the specified fill mode. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for the values listed in For specifying the type of result to output.

You can add one of the following values to the above-mentioned values to specify the fill mode.

Essentially, these values allow you to specify how to position and scale the source image in the destination image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying how to position and scale the source image in the destination image | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_CLIP | Specifies that the source image is positioned and scaled such that every destination pixel maps to a valid source pixel. This fill mode does not produce any invalid pixels in the destination image, but some information might be lost. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_FIT | Specifies that the source image is positioned and scaled such that every source pixel maps to a valid destination pixel. This fill mode might produce invalid (undefined) pixels in the destination image, but no information is lost.<br><br>This is the default value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_USE_DESTINATION_CALIBRATION | Specifies that the source image is positioned and scaled using the camera calibration information of the destination image. The source image is positioned such that its relative coordinate system is placed at the same location as the current relative coordinate system of the destination image. In addition, the source image is scaled to have the same pixel-to-world mapping as the destination image.<br><br>To use this option, the destination image must be a calibrated image with a uniform pixel size. To apply a camera calibration to the destination image that just scales and offsets the world coordinate system from the pixel coordinate system, use McalUniform() on the destination buffer prior to calling **McalTransformImage**().<br><br>Note that this option only affects how the image is positioned and scaled in the destination image; the source image is always corrected according to the camera calibration setting of the CalibrationId parameter.<br><br>This option cannot be used with M_EXTRACT_LUT_X or M_EXTRACT_LUT_Y or when OperationType is set to M_CORRECT_LENS_DISTORTION_ONLY. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalTransformResult

## Synopsis

Convert a result between its world and pixel value.

## Syntax

```
void McalTransformResult(
    AIL_ID CalibrationOrImageId,     //in
    AIL_INT64 TransformType,         //in
    AIL_INT64 ResultType,            //in
    AIL_DOUBLE Result,               //in
    AIL_DOUBLE *TransformedResultPtr //out
)
```

## Description

This function converts a specific result (a length, area, or angle) from its pixel value to its world value or vice versa. The conversion can be performed according to a camera calibration context, calibrated image, or corrected image. However, since this function uses the average pixel size to perform the conversion, results will be more accurate if you use a corrected image.

## Parameters

CalibrationOrImageId

Specifies the identifier of the camera calibration context, calibrated image, or corrected image. When an image is specified, the transformation uses the camera calibration information associated with this image.

> If you are transforming results obtained from an image, pass the identifier of that image. For example, for results originating from a child buffer, pass the child buffer's identifier.

When specifying a calibrated image or corrected image, the image buffer must be a 1- or 3-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

TransformType

Specifies whether to perform a pixel-to-world or world-to-pixel conversion. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying pixel-to-world or world-to-pixel | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_PIXEL_TO_WORLD | Converts from pixel to world. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_TO_PIXEL | Converts from world to pixel. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ResultType

Specifies the type of result the given input value represents. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of result | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_ANGLE | Represents an angle.<br><br>An angle interpreted with respect to the pixel coordinate system (M_PIXEL_TO_WORLD) is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system (M_WORLD_TO_PIXEL), see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_AREA | Represents an area. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LENGTH | Represents a length (for example, the perimeter of an object). | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LENGTH_X | Represents a length in the X-direction only. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_LENGTH_Y | Represents a length in the Y-direction only. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Result

Specifies the input value.

TransformedResultPtr

Specifies the address in which to place the output value.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalTransformResultAtPosition

## Synopsis

Convert a result, taken at a specific position , between its world and pixel value.

## Syntax

```
void McalTransformResultAtPosition(
    AIL_ID CalibrationOrImageId,   //in
    AIL_INT64 TransformType,       //in
    AIL_INT64 ResultType,          //in
    AIL_DOUBLE PositionX,          //in
    AIL_DOUBLE PositionY,          //in
    AIL_DOUBLE Result,             //in
    AIL_DOUBLE *ConvertedResultPtr //out
)
```

## Description

This function converts an angle from its pixel value to its world value or vice versa. This conversion is done locally at the specified position, and can be performed according to a camera calibration context, calibrated image, or corrected image. This function is useful when in the presence of distortion and the result is meaningless when converted from real-world to pixel units unless considered at a specific position in the image . For example, if a Model Finder model appears warped in a target image, but the camera calibration context of the image compensates for this during the model search, the resulting angle is meaningful in the real-world coordinate system, and meaningless in the pixel coordinate system unless converted at a specific position in the target image.

The position should be given in the units of the source coordinate system. For example, if converting from pixel units to world units, specify the position in pixel units.

If the position is from a corrected image (McalTransformImage()), you must specify the identifier of the image, rather than its camera calibration context, to get the result in the corrected image .

> Note that, if the relative plane used to return results is set behind the camera, the return is undefined unless M_NO_POINTS_BEHIND_CAMERA is used.

## Parameters

CalibrationOrImageId

Specifies the identifier of the camera calibration context, calibrated image, or corrected image. When an image is specified, the transformation uses the camera calibration information associated with this image.

> If you are transforming results obtained from an image, pass the identifier of that image. For example, for results originating from a child buffer, pass the child buffer's identifier.

When specifying a calibrated image or corrected image, the image buffer must be a 1- or 3-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

TransformType

Specifies whether to perform a pixel-to-world or world-to-pixel conversion. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying pixel-to-world or world-to-pixel | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_PIXEL_TO_WORLD + | Converts from pixel to world units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WORLD_TO_PIXEL + | Converts from world to pixel units. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination values for any of the possible values of the TransformType parameter.

You can add one of the following values to the above-mentioned values to set the return values of invalid points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying to return invalid points | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_NO_EXTRAPOLATED_POINTS | Specifies that if a pixel involved in the transformation is not inside the calibrated region, **M_INVALID_POINT** will be returned, instead of a coordinate resulting from the extrapolation. The calibrated region is defined as the image region covered by the camera calibration grid. To display this region, call McalDraw() with M_DRAW_VALID_REGION.<br>> This value only applies to piecewise linear camera calibrations (M_LINEAR_INTERPOLATION); if it is set and the image has any other type of camera calibration, it is ignored. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_NO_POINTS_BEHIND_CAMERA | Returns **M_INVALID_POINT** instead of an undefined value. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ResultType

Specifies the type of result to convert. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of result | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_ANGLE | Specified that result to convert is an angle. Note that angles are always returned in the range of 0 to 360 degrees. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

PositionX

Specifies the X-coordinate of the position at which to convert the result, in the units of the source coordinate system.

PositionY

Specifies the Y-coordinate of the position at which to convert the result, in the units of the source coordinate system.

Result

Specifies the result to convert at the specified position. This result must be of the type specified using ResultType.

ConvertedResultPtr

Specifies the address in which to place the converted result.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalUniform

## Synopsis

Calibrate an image or a camera calibration context by specifying the translation, scale and offset of the absolute world coordinate system.

## Syntax

```
void McalUniform(
    AIL_ID CalibrationOrImageId, //out
    AIL_DOUBLE WorldPosX,        //in
    AIL_DOUBLE WorldPosY,        //in
    AIL_DOUBLE PixelSizeX,       //in
    AIL_DOUBLE PixelSizeY,       //in
    AIL_DOUBLE PixelRotation,    //in
    AIL_INT64 ControlFlag        //in
)
```

## Description

This function allows you to calibrate an image or a camera calibration context with a uniform camera calibration, by specifying the translation, rotation, and scale of the transformation between pixel and world units. You do not need to allocate a camera calibration context or supply an image of a grid or a list as with the other camera calibration modes. If you choose to apply **McalUniform**() to a camera calibration context, however, you must allocate the camera calibration context using McalAlloc() with M_UNIFORM_TRANSFORMATION. If you apply **McalUniform**() to an image, the associated camera calibration context will be M_DEFAULT_UNIFORM_CALIBRATION.

The origin of the relative coordinate system and of the tool coordinate system will be set at (0,0) in the absolute coordinate system. For more information on coordinate systems, see the Coordinate systems section of Chapter 28: Calibrating your camera setup.

The following linear transformations are used to relate the pixel coordinate system, in pixels, to the absolute coordinate system, in world units:

$$\begin{bmatrix} W_x \\ W_y \end{bmatrix} = \begin{bmatrix} \cos R & \sin R \\ -\sin R & \cos R \end{bmatrix} \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} P_x \\ P_y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix}$$

$$\begin{bmatrix} P_x \\ P_y \end{bmatrix} = \begin{bmatrix} 1/S_x & 0 \\ 0 & 1/S_y \end{bmatrix} \begin{bmatrix} \cos R & -\sin R \\ \sin R & \cos R \end{bmatrix} \begin{bmatrix} W_x - T_x \\ W_y - T_y \end{bmatrix}$$

where:

- $R$ = PixelRotation.
- $S_x$ = PixelSizeX.
- $S_y$ = PixelSizeY.
- $T_x$ = WorldPosX.
- $T_y$ = WorldPosY.

If the PixelRotation parameter is set to 0 and the same pixel size is specified in X and Y, the image will be considered corrected.

For more information on uniform camera calibration, see the Camera calibration modes section of Chapter 28: Calibrating your camera setup.

If you adjust the coordinate system of a calibrated image associated with an M_VECTOR_AND_RASTER ROI, the raster information will be discarded, causing the ROI to become an M_VECTOR ROI. See MbufSetRegion() for more information.

## Parameters

CalibrationOrImageId

Specifies the identifier of the camera calibration context or of the image to calibrate.

The image buffer must be a 1- or 3-band buffer.

WorldPosX INQ

Specifies the X-position in the absolute coordinate system of the center of the top-left pixel of the image.

WorldPosY INQ

Specifies the Y-position in the absolute coordinate system of the center of the top-left pixel of the image.

PixelSizeX INQ

Specifies the scale between the world and pixel units in the X-direction. Specify the scale in world units per pixel. The scale must be a positive value.

PixelSizeY INQ

Specifies the scale between the world and pixel units in the Y-direction. Specify the scale in world units per pixel. The scale must be a positive value.

PixelRotation INQ

Specifies the angle of the X-axis of the pixel coordinate system measured in the absolute coordinate system, in degrees. A positive value indicates a counter-clockwise rotation (from the positive X-axis toward the negative Y-axis).

ControlFlag

Specifies the function's control flag. Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |

# McalWarp

## Synopsis

Define the mapping characteristics of a camera calibration context according to an existing camera calibration context.

## Syntax

```
void McalWarp(
    AIL_ID SrcImageOrContextCalId, //in
    AIL_ID DstImageOrContextCalId, //out
    AIL_ID WarpParam1Id,           //in
    AIL_ID WarpParam2Id,           //in
    AIL_DOUBLE OffsetX,            //in
    AIL_DOUBLE OffsetY,            //in
    AIL_DOUBLE SizeX,              //in
    AIL_DOUBLE SizeY,              //in
    AIL_INT RowNumber,             //in
    AIL_INT ColumnNumber,          //in
    AIL_INT64 TransformationType,  //in
    AIL_INT64 ControlFlag          //in
)
```

## Description

This function defines the mapping of a camera calibration context according to a previously configured camera calibration context. Note that the camera calibration mode of the destination camera calibration context need not necessarily be the same as the camera calibration mode of the source context. The resulting mapping will be an accurate approximation of the source mapping for the specified image or context. This can be used, for example, to approximate a complex camera calibration using a constant pixel size camera calibration, simplifying further calculations.

This function can also be used to generate a new context by warping a previously calibrated context. The resulting context can then be associated with the uncalibrated warped image. This can be used, for example, to calibrate an image that has been warped using MimWarp(). MimWarp() returns an uncalibrated image which can then be associated (using McalAssociate()) with the newly warped camera calibration context.

To define new mapping that will only apply to a specific region of an image, use the OffsetX, OffsetY, SizeX and SizeY parameters. The mapping of the new camera calibration context will apply only to this defined area. See the description of these parameters in the Parameters section located below for more information.

## Parameters

SrcImageOrContextCalId

Specifies the identifier of the camera calibration context, calibrated image, or corrected image From which the source mapping is taken.

The image buffer must be a 1- or 3-band buffer.

Note that the underlying source camera calibration context must have been previously configured using a successful call to McalGrid(), McalList(), McalUniform(), or **McalWarp**().

DstImageOrContextCalId

Specifies the identifier of the destination camera calibration context or image buffer.

When the identifier of a destination camera calibration context is specified, the camera calibration context must have been allocated using McalAlloc() with M_UNIFORM_TRANSFORMATION, M_LINEAR_INTERPOLATION, or M_PERSPECTIVE_TRANSFORMATION, that is, the camera calibration mode of the context must be 2D.

When an identifier of an image buffer is specified, the default uniform camera calibration context (M_DEFAULT_UNIFORM_CALIBRATION) will be associated to the image, and the M_UNIFORM_TRANSFORMATION camera calibration mode will be assumed. Any current camera calibration context associated to the image is ignored. The image buffer must be a 1- or 3-band buffer.

If the operation fails, the image will be uncalibrated. In-place processing is supported, that is, DstImageOrContextCalId can be the same identifier as SrcImageOrContextCalId.

WarpParam1Id

Specifies the buffer containing the matrix of coefficients or the LUT buffer from which source X-coordinates are determined.

If specifying a matrix of coefficients, the buffer must be a single-band, 32-bit floating-point buffer that has an M_ARRAY attribute and that has dimensions 3x2 or 3x3.

If specifying a LUT buffer, then the buffer must be a signed 16- or 32-bit integer buffer and have an M_LUT attribute.

If no warping of the source camera calibration context is required, set WarpParam1Id to **M_NULL**.

Please refer to MimWarp() for more information on this parameter.

WarpParam2Id

Specifies the LUT buffer from which source Y-coordinates are determined.

This buffer must be a single-band, signed 16- or 32-bit integer buffer and have an M_LUT attribute. The buffer must also have the same type and dimensions as WarpParam1Id.

If no warping is applied to the source camera calibration context, or if you are not using LUTs to perform the warping, set WarpParam2Id to **M_NULL**.

Please refer to MimWarp() for more information on this parameter.

OffsetX

Specifies the X pixel coordinate of the top-left calibration point.

If the camera calibration context has been warped, then it refers to the coordinate in the newly warped object.

OffsetY

Specifies the Y pixel coordinate of the top-left calibration point.

If the camera calibration context has been warped, then it refers to the coordinate in the newly warped object.

SizeX

Specifies the width of the grid of calibration points, in pixels.

SizeX must be set to a value > 1.0.

However, if TransformationType is set to M_WARP_LUT, SizeX can be set to **M_NULL** to indicate that the width specified by the LUTs should be used. In this case, OffsetX, OffsetY, SizeX and SizeY must all be set to **M_NULL**.

SizeY

Specifies the height of the grid of calibration points, in pixels.

SizeY must be set to a value > 1.0.

However, if TransformationType is set to M_WARP_LUT, SizeY can be set to **M_NULL** to indicate that the width specified be the LUTs should be used. In this case, OffsetX, OffsetY, SizeX and SizeY must all be set to **M_NULL**.

RowNumber

Specifies the number of rows of calibration points to be used for camera calibration.

Depending on the camera calibration mode of the destination camera calibration context or image, the operation requires that enough calibration points be present. For camera calibration contexts allocated using McalAlloc() with M_UNIFORM_TRANSFORMATION and M_LINEAR_INTERPOLATION camera calibration modes, the minimum number of calibration points required is 3. For the M_PERSPECTIVE_TRANSFORMATION camera calibration mode, the minimum number of calibration points required is 4. Accuracy increases with the number of points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the number of rows: | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M_DEFAULT | Specifies the default value; the default value is 15. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 2 | Specifies the number of rows of calibration points to be used for camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ColumnNumber

Specifies the number of columns of calibration points to be used for camera calibration.

Depending on the camera calibration mode of the destination camera calibration context or image, the operation requires that enough calibration points be present. For camera calibration contexts allocated using McalAlloc() with M_UNIFORM_TRANSFORMATION and M_LINEAR_INTERPOLATION camera calibration modes, the minimum number of calibration points required is 3. For the M_PERSPECTIVE_TRANSFORMATION camera calibration mode, the minimum number of calibration points required is 4. Accuracy increases with the number of points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the number of columns: | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_DEFAULT | Specifies the default value; the default value is 15. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| *Value* >= 2 | Specifies the number of columns of calibration points to be used for camera calibration. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

TransformationType

Specifies the type of transformation. This parameter must be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying the type of transformation: | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_IDENTITY | No transformation is performed. This is used when **McalWarp**() approximates a new camera calibration context based on a source camera calibration context. In such cases, WarpParam1Id and WarpParam2Id must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WARP_LUT + | Performs the warping through LUTs. In such cases, WarpParam1Id and WarpParam2Id must be buffers allocated with the M_LUT attribute. | c | | e | f | g | a | | i | | j | k | l | m | n | b |
| M_WARP_POLYNOMIAL | Performs the warping using a 3x3 coefficient matrix. This includes first-order polynomial warping or perspective polynomial warping. In such cases, WarpParam1Id must indicate a buffer having the M_ARRAY attribute and WarpParam2Id must be set to **M_NULL**. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

Combination value for M_WARP_LUT.

You can add the following value to the above-mentioned value to specify the number of fractional bits in the coordinates of the source point.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

| For specifying fractional bits | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Combination value | Description | (c) | (d) | (e) | (f) | (g) | (a) | (h) | (i) | (o) | (j) | (k) | (l) | (m) | (n) | (b) |
| M_FIXED_POINT + *n* | Specifies the number of fractional bits for source coordinates. To do so, add the define M_FIXED_POINT + n to M_WARP_LUT where *n* >= 0. If nothing is added to M_WARP_LUT, it is assumed that there are no fractional bits in the coordinates of the source point. | c | | e | f | g | a | | i | | j | k | l | m | n | b |

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

## Compilation information

| | |
|---|---|
| **Header** | Include AIL.h. |
| **Library** | Use ail11.lib; ailcal11.lib. |
| **DLL** | Requires ail11.dll; ailcal11.dll. |